University of Bergen
Department of Informatics

Neural Structures in Breast Cancer: Segmentation and Survival Outcomes

Author: Alina Artemiuk
Supervisors: Nello Blaser, Kenneth Finne, Heidrun Vethe

UNIVERSITETET I BERGEN
Fakultet for naturvitenskap og teknologi

August, 2025

Abstract

Recent studies suggest that nerves within the tumor microenvironment play an active role in the progression of breast cancer. Neural tumor interactions can promote tumor growth, invasion, and metastasis, as well as influence the response to treatment. Despite their potential importance, nerves are difficult to detect in tissue images due to their thin morphology and fragmented appearance after tissue sectioning.

This thesis presents a computational pipeline that combines deep learning-based image segmentation and statistical survival analysis to study nerve fibers in breast cancer tissue. I developed a U-Net-based segmentation model to detect nerves in tumor tissue examined by high-dimensional imaging technology Imaging Mass Cytometry (IMC). I then quantified the spatial distribution of the detected nerves and the types of cells that interact with them, and subsequently analyzed these features using the Cox proportional hazards model. The results show that nerve-related measures alone have limited prognostic value, but incorporating information about the cell types present improves predictive performance. These findings suggest that the cellular composition within nerve regions may be an important factor in patient survival and represents a promising direction for future research on nerve-cell interactions.

Acknowledgements

First of all, I want to extend my thanks to Nello Blaser, my supervisor, for his valuable guidance and helpful feedback throughout the project. I am incredibly grateful for his assistance during the summer, which greatly contributed to the completion of this thesis.

My thanks also go to my co-supervisors, Kenneth Finne and Heidrun Vethe from Haukeland University Hospital. I appreciate the knowledge I gained in the field of biology and the opportunity to work on such a meaningful topic alongside experts in this domain.

I would also like to acknowledge the University of Bergen for providing access to their graphics processors. This significantly accelerated the training of deep learning models and allowed me to conduct a greater number of experiments.

And last but not least, I am deeply grateful to my family and friends. I thank my parents for their patience and encouragement, my grandmother for her constant support, and my friend Sophia, who stood by me during an exceptionally difficult year.

`Alina Artemiuk`
Friday 15th August, 2025

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

Breast cancer is the most common type of cancer among women and is the leading cause of cancer-related deaths [22]. Although early detection and treatment have improved survival rates, the decline in mortality has slowed in recent years [22]. This has prompted researchers to investigate new biological factors that may influence disease progression. One such factor appears to be the presence of nerves in the tumor microenvironment [25].

Recent studies suggest that the nervous system, previously considered a passive bystander, is an active participant in tumor progression [22]. Neural tumor interactions can promote tumor growth, invasion, metastasis, drug resistance, and impair the immune system's ability to combat cancer [25]. Increased nerve fiber density and greater nerve thickness within tumors have been associated with a more aggressive form of the disease and a poorer prognosis [22]. Certain nerve subtypes appear to promote tumor growth, while others have a predominantly protective effect [12]. Cancer treatments such as chemotherapy, targeted therapy, and immunotherapy can also affect the nervous system, resulting in neurological impairments that may further influence tumor progression [25]. Taken together, these findings suggest that treatment strategies targeting neural tumor interactions may offer new opportunities for improving the prognosis of cancer patients [25].

The detection of nerve structures in tissue images, however, poses significant challenges. The reason for this is their morphology. Nerve axons are elongated, thin, tail-like

structures that occupy a very small total area. During tissue sectioning, the cutting plane may intersect them at multiple points along their length, breaking up the continuous structure into small cross-sections that look like scattered clusters of dots. As a result, they are particularly difficult to distinguish from noise. Identifying these structures manually is a time-consuming, labor-intensive, and error-prone task. Commonly used cell segmentation methods are also unsuitable, as they rely on detecting cell nuclei, which in neurons are located in the cell body (called soma) far from the axon. Therefore, there is a need for a computational method that can accurately detect such thin, non-nucleated structures.

In this thesis, the problem is tackled using a deep learning-based approach. I explore different data preprocessing strategies and employ the widely used U-Net architecture, originally developed for biomedical image segmentation. Building on these segmentation results, I conduct a survival analysis using Cox regression to examine the relationship between nerve-related features and patient survival. By combining image analysis with statistical modeling, this work aims to provide both a robust computational tool for nerve detection and a quantitative evaluation of their prognostic value in breast cancer.

1.2 Objectives

The main goal of this thesis is to develop a pipeline for the detection and quantitative analysis of non-nucleated nerve structures in breast cancer tissue examined using a high-dimensional imaging technique called Imaging Mass Cytometry (IMC). This goal can be broken down into the following specific objectives:

1. Develop a deep learning-based segmentation model for accurate detection of nerve fiber elements.
2. Quantify the spatial distribution of nerve segments across tissue compartments (tumor and stroma) and analyze their cellular composition.
3. Investigate how the presence and spatial distribution of nerve structures correlate with patient survival.

1.3 Thesis Outline

This thesis is organized into five chapters. Chapter 1 defines the problem, states my objectives, and outlines the thesis structure. Chapter 2 presents the necessary background, covering Imaging Mass Cytometry and its challenges, data preprocessing methods, convolutional neural networks with the U-Net architecture, evaluation metrics for image segmentation, and survival analysis with the Cox proportional hazards model. Chapter 3 describes the data and methods used for segmentation, spatial nerve analysis, and survival analysis. Chapter 4 reports the results obtained using these methods. Finally, Chapter 5 summarizes the main findings, discusses strengths and limitations, and proposes directions for future research.

Chapter 2

Background

In this chapter, I provide the background knowledge required to follow this thesis. I begin with an overview of Imaging Mass Cytometry (IMC) and describe the specific challenges associated with IMC data. I then outline image preprocessing methods that address these challenges. This is followed by a discussion of convolutional neural networks (CNNs), the U-Net architecture, loss functions, and evaluation metrics for image segmentation. Finally, I present the fundamentals of survival analysis, focusing on the Cox proportional hazards (CPH) model.

2.1 Imaging Mass Cytometry

Imaging Mass Cytometry (IMC) is an imaging technique that enables the simultaneous analysis of up to 40 proteins in a single tissue sample at single-cell resolution. By using metal-tagged antibodies, IMC generates an image with up to 40 channels that captures both protein expression and spatial distribution within biological tissues. This technique allows identification of individual cell subpopulations, cell-cell interactions, and provides a deeper understanding of the complex structure of the tumor microenvironment [16].

2.1.1 IMC Workflow

The IMC workflow consists of five sequential stages: tissue preparation, antibody staining, laser ablation, mass spectrometry and image reconstruction. Together, these steps generate high-resolution images that map the spatial distribution of proteins within tissue samples. The process, based on [16], is summarized below and illustrated in Figure 2.1.

1. Tissue preparation First, tissue samples are processed into thin sections and placed on glass slides.

2. Antibody staining Next, a panel of antibodies targeting proteins relevant to breast cancer is selected. Each antibody is tagged with a unique rare-earth-metal isotope of known atomic mass. The tissue sections are then treated with a cocktail of these metal-tagged antibodies, which bind specifically to their target proteins.

3. Laser ablation After staining, the slides are placed into the ablation chamber of the Hyperion tissue imager. Inside this chamber, a UV laser scans the tissue spot by spot and line by line. Each laser pulse evaporates a tiny area of $1\mu m^2$, releasing plumes consisting of tissue particles, antibody residues, and metal isotopes [42].

4. Mass spectrometry At this stage, the ablation plumes are carried by a stream of inert gas into a CyTOF (Cytometry by Time-Of-Flight) mass spectrometer [42]. Inside this instrument, the particles from the plume are first ionized (turned into charged atoms) and then accelerated through a vacuum tube by an electric field. The speed of the ions as they travel through the tube depends on their mass-to-charge ratio.

At the end of the tube is a detector. The detector records both the arrival time and the quantity of each ion. Based on this information, the mass-to-charge ratio of each ion can be determined, thereby making it possible to identify the corresponding metal isotope used to label the antibody.

Since each antibody is tagged with a known metal isotope, the detection of a particular isotope reveals which protein is present, while the number of detected ions reflects the amount of antibody bound to the tissue, and thus the abundance of the target protein.

5. Image reconstruction Finally, the data from the mass spectrometer is processed to reconstruct a multi-channel image that captures the spatial distribution and abundance of proteins within the tissue sample. An example of a resulting IMC image can be seen in Figure 2.2.

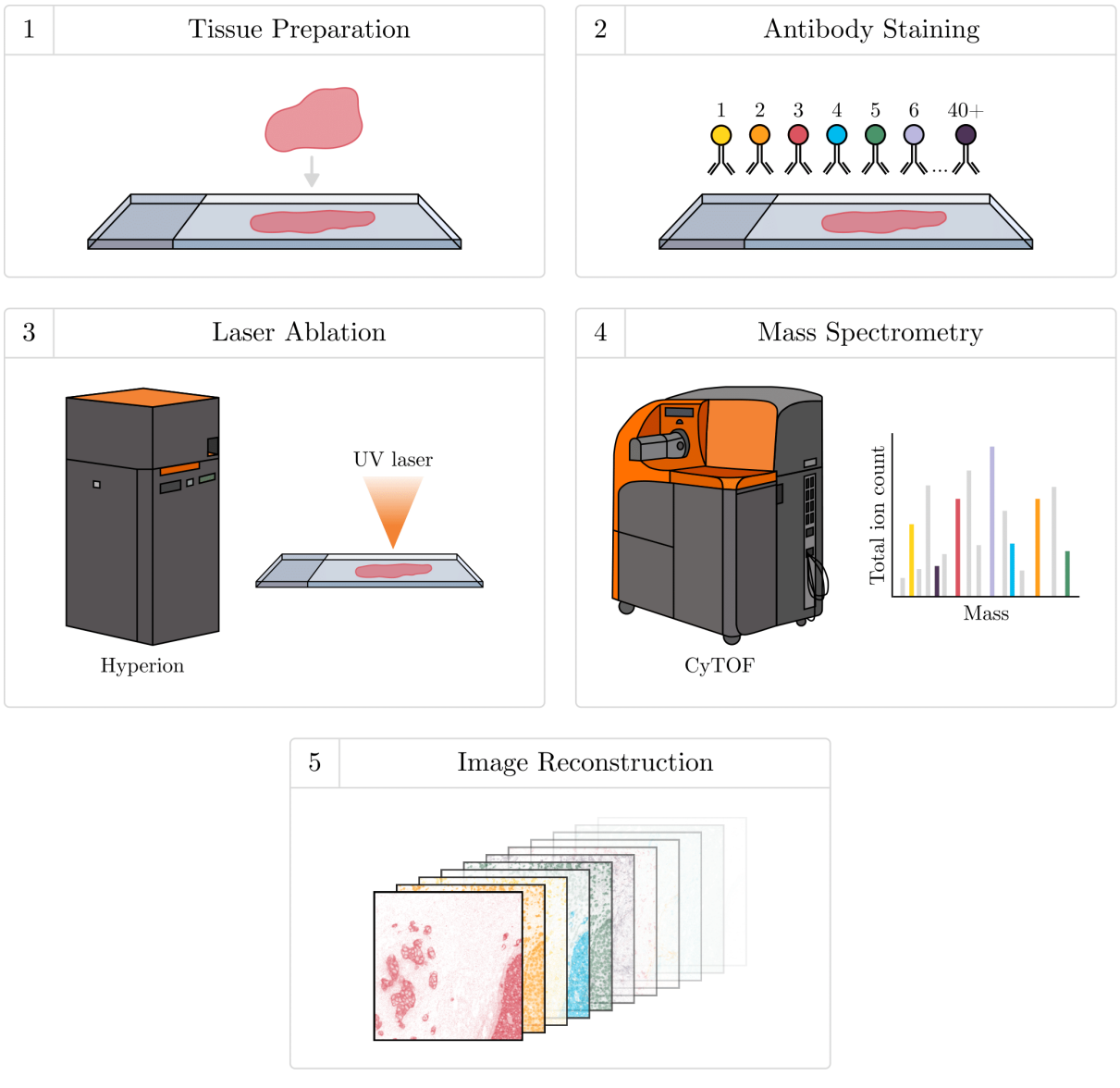


Figure 2.1: A schematic overview of the Imaging Mass Cytometry workflow.

Credit: The figure is inspired by [42], [8] and [31].

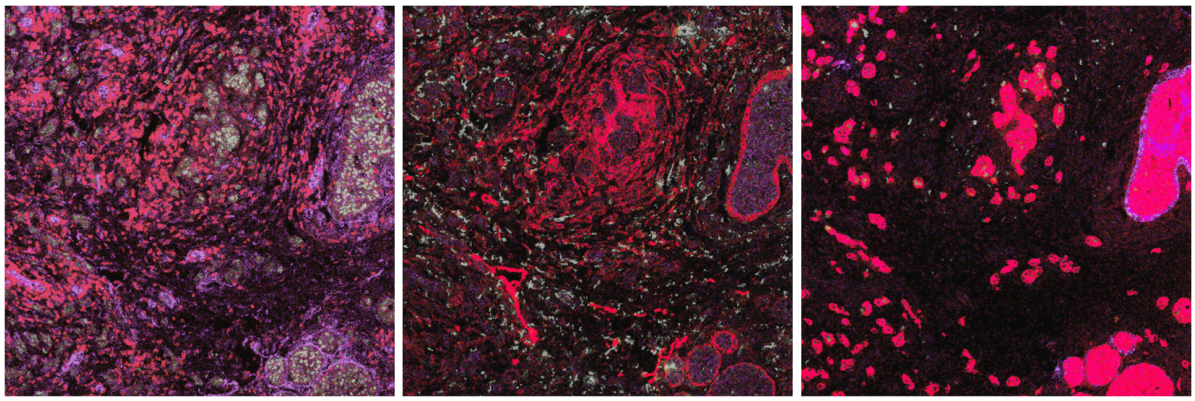


Figure 2.2: IMC image of a breast cancer patient's tissue section showing the overlay of multiple markers.

2.1.2 Challenges of IMC Data

The IMC data presents several challenges that need to be addressed before proceeding with further analysis steps [42]. These challenges are related to both the sample preparation process and the limitations inherent in imaging technology. The main issues include artifacts, low signal-to-noise ratio, background noise, batch effects, and limited spatial resolution.

The most common problem is the presence of artifacts. These include *hot pixels* (individual pixels with unusually high intensities caused by detector abnormalities) and *speckles* (small clusters of high-intensity pixels that do not correspond to biological structures, resulting from unspecific antibody binding or dust contamination) [42]. These artifacts can be mistaken for real biological signal and thus bias the analysis. One common approach to remove them is to use thresholding methods. However, since intensity levels vary between markers and tissues, thresholds must be chosen carefully to avoid removing real signal [36]. More advanced methods, such as IMC-Denoise, a deep learning denoising pipeline introduced by *Lu et at.* [36], can help reduce these artifacts more efficiently. It first detects hot pixels by comparing each pixel with its neighborhood, then uses a self-supervised neural network to filter out noise.

A low signal-to-noise ratio can also complicate the analysis. Weakly expressed markers make it difficult to distinguish between real biological signal and noise. This can lead to misclassification or an inability to detect structures of interest.

Background noise represents another challenge. It refers to unwanted variation in the image signal [42]. Background noise can be addressed using semi-automated ilastik-based

method described by *Ijsselsteijn et al.* [23]. Their workflow involves manual annotation and parameter tuning, which are time-consuming tasks that heavily rely on the user's domain knowledge and can lead to inconsistent and potentially biased results [23].

Batch effect can occur when samples are processed under different conditions. Variations in staining, tissue preparation or instrument performance between collection sessions can lead to differences in staining efficiency between samples, which can potentially affect downstream analysis [59, 42].

IMC also has a relatively low spatial resolution, about 1 μm^2 per pixel. This can make it difficult to accurately separate closely situated biological structures [3]. Boundaries in close proximity can merge, causing under-segmentation.

2.1.3 Data Preprocessing

Min-Max Normalization

Min-max normalization scales numerical data to a fixed range, typically between 0 and 1, by linearly transforming the values based on the minimum and maximum,

$$X_{\text{norm}} = \frac{X - \min(X)}{\max(X) - \min(X)} \in [0, 1]. \tag{2.1}$$

By putting all input features on the same scale, min-max normalization prevents features with higher values from dominating the learning process. This can accelerate convergence and improve model performance. On the downside, it does not handle outliers very well. If there are extreme outliers, they can stretch the range and squeeze most values closer to zero, resulting in reduced contrast of the image.

Log Transformation

The *logarithmic transformation* is commonly used to reduce the right skewness in the data. It narrows the range of large values and expands the range of small ones,

$$X_{\log} = \log(1 + X). \tag{2.2}$$

The offset of 1 avoids taking the logarithm of zero. In image processing, this transformation reduces the excessive brightness of bright areas and improves the visibility of dark areas.

Figure 2.3 shows how log transformation reduces the skewness of the distribution.

Figure 2.3: Effect of log transformation on skewed distribution.

Percentile Normalization

Percentile normalization, applied to IMC data in [5], helps mitigate the influence of extreme pixel values by scaling the data based on a specified percentile of its intensity distribution. Given an image X, the normalized output is computed as

$$X_{\text{perc}} = \frac{X}{P_q}, \tag{2.3}$$

where P_q is the q-th percentile of pixel intensities. In [5], the authors used the 99th percentile ($q = 99$).

Gaussian Blur

Gaussian blur, also known as Gaussian smoothing, is an image processing technique used for noise reduction. It works by replacing each pixel with a weighted average of its neighboring pixels [55]. The weights are determined using a Gaussian function that assigns higher values to pixels closer to the center of the local neighborhood and gradually lower values to those farther away.

The one-dimensional Gaussian function is a continuous probability density function defined by the equation

$$G_\sigma(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{x^2}{2\sigma^2}\right), \tag{2.4}$$

where x is the distance from the center of the neighborhood and σ is the standard deviation of the Gaussian distribution.

The two-dimensional Gaussian function used in image processing is derived by multiplying two one-dimensional Gaussian functions: one in the horizontal (x) and one in the vertical (y) directions. This results in the formula [55]

$$G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right). \tag{2.5}$$

The parameters x and y represent the horizontal and vertical distances from the center of the kernel, located at $(0, 0)$, where the Gaussian function reaches its maximum. The parameter σ controls the degree of blurring. A small σ gives more weight to pixels near the center, resulting in a subtle blur. In contrast, a larger σ spreads the weights over a larger area, leading to a stronger and more noticeable blurring effect. The effect of different σ values is shown in Figure 2.4.

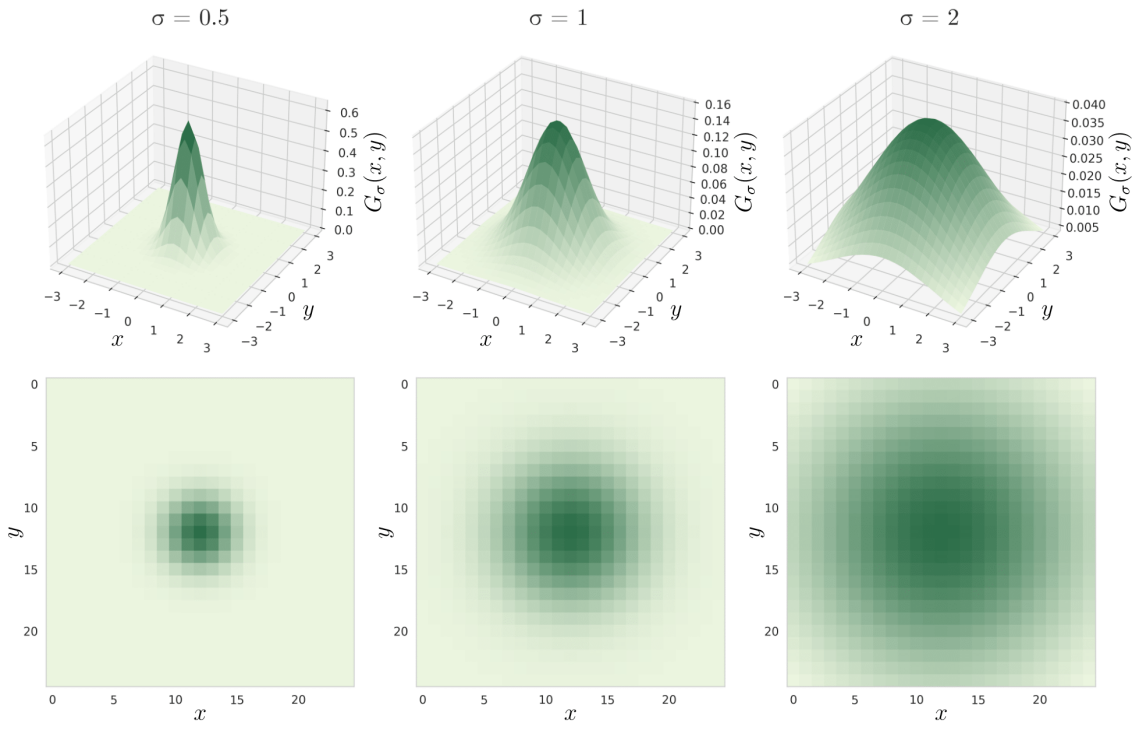


Figure 2.4: The two-dimensional Gaussian function with different standard deviations (σ) in 3D and 2D formats.

Credit: The figure is inspired by [64].

Although the Gaussian function is defined over an infinite domain, in practice, to make computations feasible, the kernel is truncated to a finite size [58]. Typically, the radius of the kernel is limited to three standard deviations (3σ), as approximately 99.73% of the area under the Gaussian curve lies within this range [58]. Pixels located beyond this distance have very little influence, so their contribution can be considered practically zero [58]. Based on this heuristic, the size of the kernel is set to $(2r + 1) \times (2r + 1)$, with a radius $r = 3\sigma$. The addition of 1 ensures that the kernel has a central pixel and is symmetric with respect to the origin. For example, if $\sigma = 1$, then the radius is 3, resulting in a kernel of size 7×7.

Once the kernel has been truncated, the resulting weights no longer sum to 1. If used as is, this can lead to unwanted changes in the brightness of the image. To prevent this, the kernel is normalized by dividing each weight by the sum of all weights in the kernel.

Histogram Equalization

Histogram equalization is a simple yet effective method for enhancing image contrast [67]. The method works by redistributing pixel intensities so that the resulting histogram approaches a uniform distribution [67]. The process is based on the original gray-level distribution of the image and aims to stretch and redistribute the most frequent intensity values over the entire available range [67].

According to information theory, a signal carries the most information when its values have a uniform distribution property [28]. By transforming the histogram into an approximately uniform one, histogram equalization increases the entropy of the image. This, in turn, improves contrast and reveals fine details that may be difficult to distinguish in the original image.

Let's formalize the process. Consider a grayscale image $X = \{x(i, j)\}$ of size $M \times N$, where $x(i, j)$ is the intensity value of the pixel at position (i, j) [67]. The total number of image pixels is $n = M \times N$, and the image has L ordered intensity levels $\{g_0, g_1, \ldots, g_{L-1}\}$, where $g_0 < g_1 < \cdots < g_{L-1}$.

Let n_k be the number of pixels in the image that have intensity value g_k. The probability density function (PDF) of the input image intensities is defined as the proportion of pixels that have the value g_k, namely

$$\mathrm{PDF}(g_k) = \frac{n_k}{n}, \quad 0 \leq k < L. \tag{2.6}$$

The cumulative distribution function (CDF) is then computed as the cumulative sum of PDF values up to and including g_k,

$$\mathrm{CDF}(g_k) = \sum_{i=0}^{k} \mathrm{PDF}(g_i), \quad 0 \leq k < L. \tag{2.7}$$

From Equations 2.6 and 2.7, it follows that $\mathrm{CDF}(g_{L-1}) = 1$.

Using the CDF, the histogram equalization transformation function $T(g_k)$ maps each input intensity level to a new one. This is defined as [67]

$$T(g_k) = g_0 + (g_{L-1} - g_0) \times \mathrm{CDF}(g_k), \quad 0 \leq k < L. \tag{2.8}$$

The equalized output image $Y = \{y(i,j)\}$ is then obtained by applying $T(g_k)$ to each pixel in the original image X [67]. This can be defined as

$$Y = T(X) = \{T(x(i,j)) \mid \forall x(i,j) \in X\}. \tag{2.9}$$

Figure 2.5 illustrates the effect of applying histogram equalization.

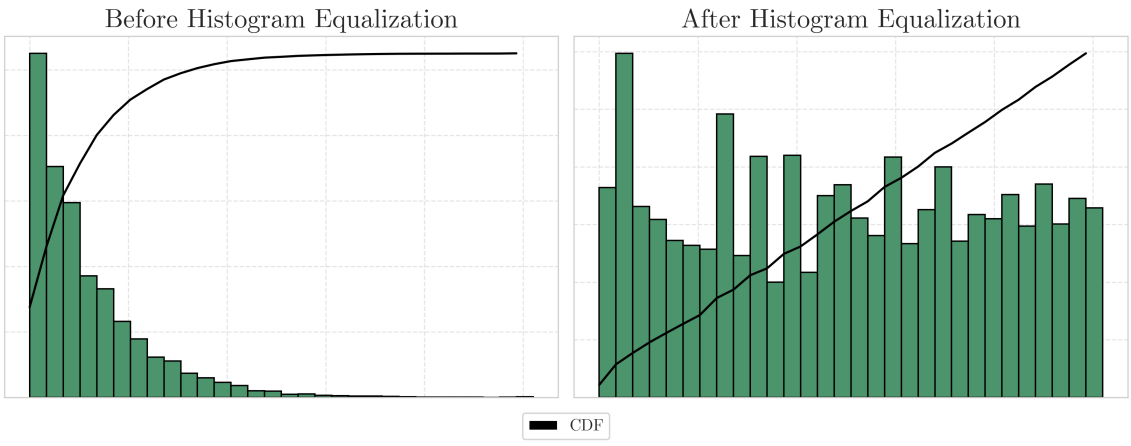


Figure 2.5: Histogram and CDF before and after histogram equalization.

2.2 Convolutional Neural Networks

This section is based on material from [68], [18] and [65]. It is assumed that the reader is familiar with basic neural network concepts, including multilayer perceptrons (MLPs)

and fully connected layers, where each neuron is connected to every neuron in the adjacent layer.

Convolutional neural networks (CNNs) are a type of neural network designed to process data with a grid-like structure, such as images. Images can be viewed as two-dimensional grids of pixels, where neighboring pixels are often related and visual features such as edges or textures are repeated in different areas. CNNs are well suited for this kind of data because they can capture local features while keeping the number of parameters relatively low.

Before CNNs, a common approach for image processing was to flatten the image into a one-dimensional vector and use a multilayer perceptron (MLP). This method, however, had significant drawbacks. MLPs struggle with large, high-dimensional data like images because they lack spatial awareness. Flattening breaks the spatial structure of the image and the number of parameters increases rapidly with increasing input data.

The following example from [7] clearly demonstrates this limitation. Consider grayscale images of size 28×28. A neuron in a fully connected hidden layer would receive 784 input values, which is computationally manageable. However, this does not scale well as images get larger. For a color image of size 200×200, the input layer would have $200 \times 200 \times 3 = 120{,}000$ inputs. With multiple layers and many neurons, the number of parameters increases significantly. This makes the model not only computationally expensive but also more likely to overfit to the training dataset.

CNNs avoid these issues by using a different strategy. Instead of connecting every input pixel to every neuron, they use filters. These filters slide across the image and apply the same set of weights at each location. For example, a 3×3 filter applied to a grayscale image has $3 \times 3 \times 1 = 9$ weights. For a color image with three channels, the same 3×3 filter would have $3 \times 3 \times 3 = 27$ weights. In both cases, the number of parameters is very small compared to a fully connected layer. The price paid for this drastic reduction in parameters is that each layer focuses only on a small area of the image and processes the same features in the same way, regardless of where they are located.

The design of CNNs was inspired by how the brain processes visual information. In the early 1960s, neurophysiologists David H. Hubel and Torsten N. Wiesel conducted experiments on the visual cortex (part of the brain responsible for processing visual information) of cats. They found that certain neurons respond selectively to simple visual features, such as edges or lines with particular orientations. These responses are then combined by other neurons to recognize more complex visual patterns. This hierarchical

organization inspired the structure of CNNs. Similarly, early layers of CNNs detect basic features like edges and textures, while deeper layers combine these simpler features to capture more abstract and complex patterns. A typical CNN structure is shown in Figure 2.6.

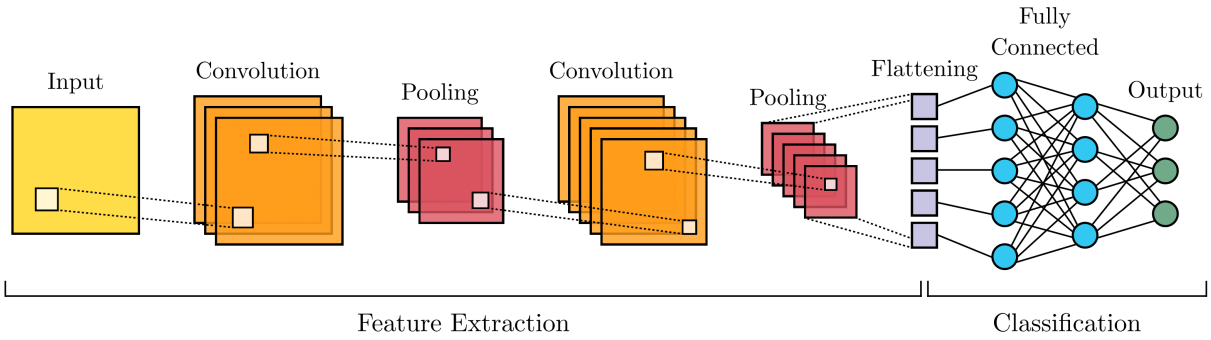


Figure 2.6: Schematic diagram of a basic CNN architecture, consisting of convolutional and pooling layers for feature extraction, followed by flattening and fully connected layers for classification.

In the following sections, I describe the main components of CNNs and how they process image data.

2.2.1 Convolutional Layer

As the name suggests, the convolution layer is a core component of CNNs. Its primary function is to extract meaningful features from input data, such as edges, textures, or shapes, while gradually reducing spatial dimensions. This allows the network to learn increasingly abstract features as the data moves through deeper layers.

A convolutional layer operates by applying a small sliding window, called a *filter* or *kernel*, to the input. The filter represents a small matrix of learnable weights that is placed at the upper-left corner of the input and slides across it both from left to right and top to bottom. At each position, the filter overlaps with a small region of the input called the *receptive field*. The values within the receptive field and the filter are multiplied element-wise and summed, producing a single scalar value. Repeating this process over the entire input yields a two-dimensional output called a *feature map*. The procedure can be repeated using different filters to form as many output feature maps as needed.

For a single-channel input, this process corresponds to a cross-correlation between an input matrix I and a filter F of size $f_h \times f_w$, defined as

$$M(i,j) = (I * F)(i,j) = \sum_{m=0}^{f_h-1} \sum_{n=0}^{f_w-1} I(i+m, j+n)\, F(m,n), \tag{2.10}$$

where $M(i,j)$ is the output value at position (i,j) in the feature map.

An example of this operation is illustrated in Figure 2.7.

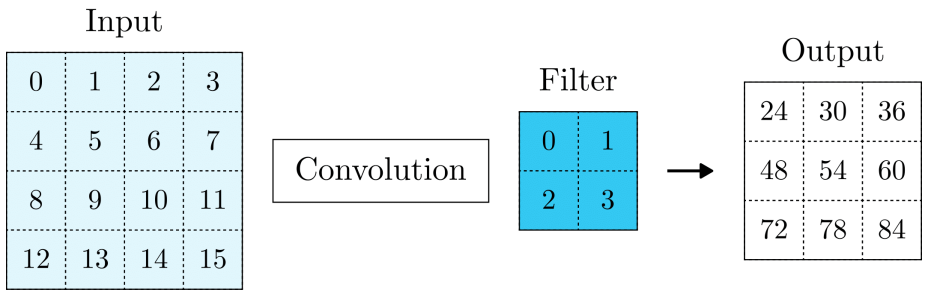


Figure 2.7: Cross-correlation of a single-channel input.

If there are multiple input feature maps, the filter will have to be three-dimensional. Each input feature map will be convolved with a distinct filter and the resulting intermediate feature maps will be summed element-wise to produce the final output feature map (see Figure 2.8).

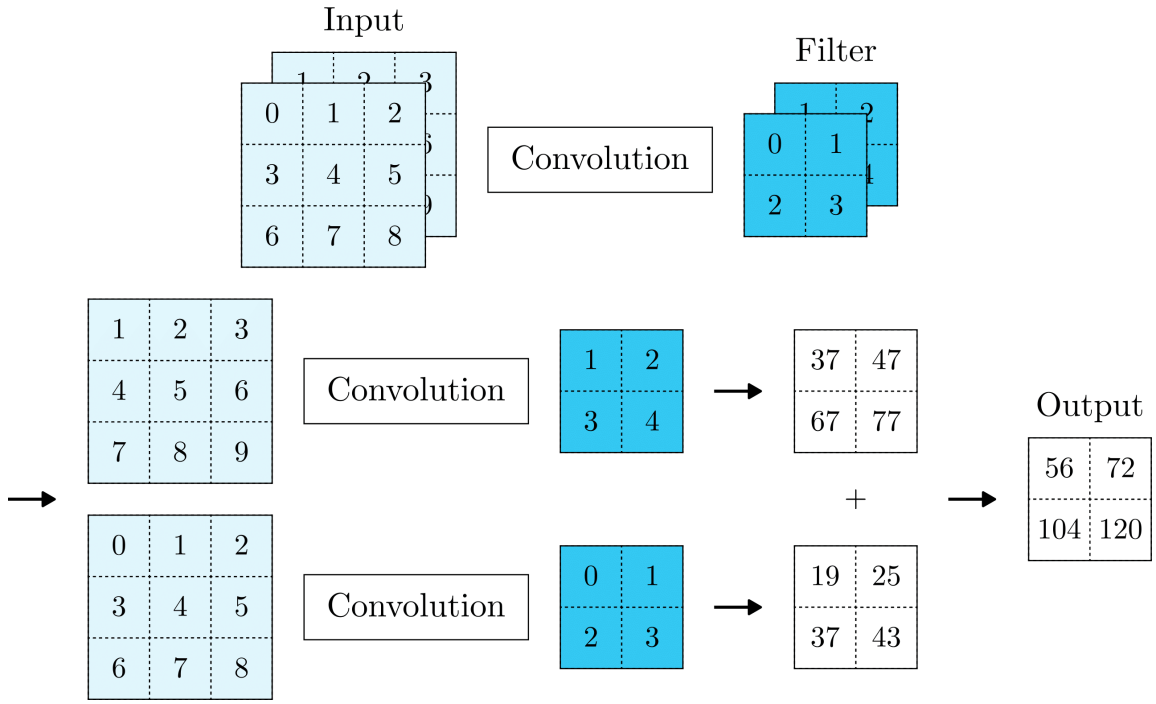


Figure 2.8: Cross-correlation with multiple input channels.

In order to improve computational efficiency or reduce the spatial resolution of the feature maps more significantly, the filter window may be moved more than one element at a time, skipping intermediate positions. The step size, referred to as the *stride*, specifies how far the window shifts vertically and horizontally after each operation. A stride of 1 moves the filter one pixel at a time, yielding the highest possible output resolution. Increasing the stride reduces the output size because the filter covers fewer positions. This lowers computational cost due to reduced overlap between filter applications and can also be advantageous when large filters are used, as these inherently capture a wide spatial area.

Another way to control the behavior of convolutional layers is through the *padding* hyperparameter. Since filters usually have a width and height greater than 1, applying many successive convolutions leads to a significant reduction in the spatial dimensions of the data. For example, applying ten layers of 5×5 filters to a 250×250 image would reduce the image to 210×210 pixels, discarding 30% of the image and, with it, potentially important boundary information. To mitigate this, CNNs use padding, which involves adding extra pixels (typically zeros) around the border of the input. This serves several purposes. First, it allows filters to be applied to border regions. Second, it makes sure that all pixels are used equally frequently. Third, it provides control over the spatial dimensions of the output.

Figure 2.9 depicts a two-dimensional cross-correlation operation with a padding of 1 and a stride of 2.

Given an input of size $I_h \times I_w$, a filter of size $f_h \times f_w$, vertical and horizontal strides s_h, s_w, and padding p_h, p_w, the output height and width are computed as

$$O_h = \left\lfloor \frac{I_h - f_h + 2p_h}{s_h} \right\rfloor + 1, \quad O_w = \left\lfloor \frac{I_w - f_w + 2p_w}{s_w} \right\rfloor + 1.$$

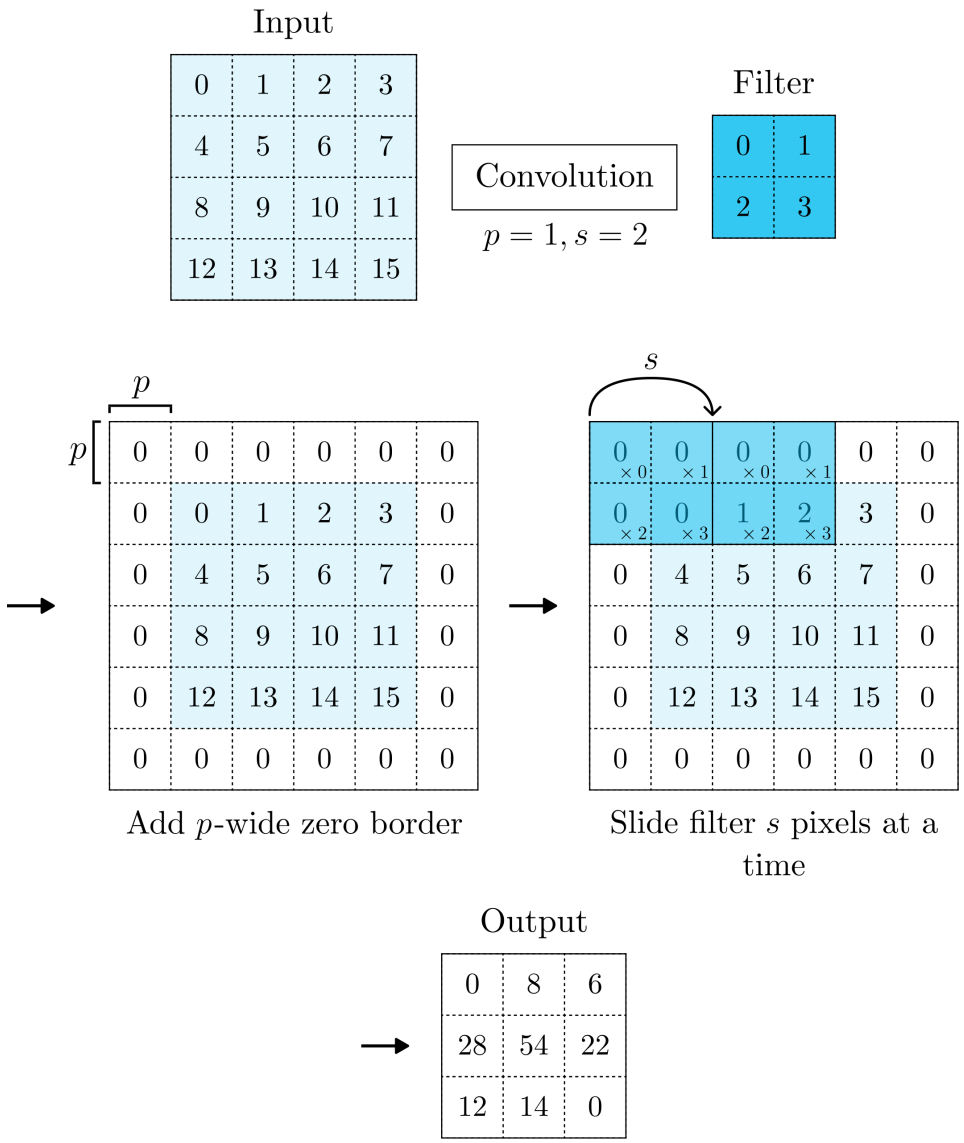


Figure 2.9: Convolution operation applied to a 4×4 input using a 2×2 filter, with padding $p = 1$ and stride $s = 2$. The filter slides over the padded input s steps at a time, producing a 3×3 output feature map.

2.2.2 Pooling Layer

Following feature extraction by convolutional layers, CNNs typically include pooling layers to gradually reduce the spatial dimensions of the resulting feature maps. By decreasing the spatial resolution, pooling lowers the computational cost, reduces the number of parameters and helps prevent overfitting [48].

Just like convolutional layers, pooling works by sliding a fixed-size window over the input according to a specified stride. At each step, the window aggregates information

in the region it covers and outputs a single value. However, unlike convolution layers, pooling layers do not have learnable parameters. Instead of learning what to extract, pooling layers apply a deterministic operation, such as selecting the maximum value or computing the average of the values within a sliding window. These operations are called *max-pooling* and *average pooling*, respectively.

Another difference from convolutional layers lies in how pooling handles multi-channel input. The pooling layer processes each input channel separately rather than combining them as convolutional layers do. As a result, the depth of the output feature map remains identical to that of the input.

Among the two popular pooling choices, max-pooling is generally preferred over average pooling. Max-pooling is typically performed using a window of size 2×2 and a stride of 2. This reduces the total number of activations by a factor of 4, while keeping the depth of the feature map unchanged. Figure 2.10 illustrates how this operation works in practice.

The output size $O_h \times O_w$ resulting from applying a pooling operation with window size $f_h \times f_w$, vertical stride s_h, and horizontal stride s_w, to an input of size $I_h \times I_w$, is computed as

$$O_h = \left\lfloor \frac{I_h - f_h}{s_h} \right\rfloor + 1, \quad O_w = \left\lfloor \frac{I_w - f_w}{s_w} \right\rfloor + 1.$$

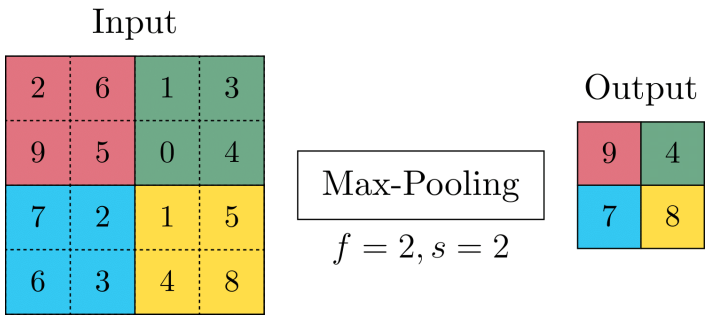


Figure 2.10: Max-pooling applied to a 4×4 input using a 2×2 filter with stride $s = 2$. From each non-overlapping region, the maximum value is selected, resulting in a 2×2 output.

2.3 Image Segmentation

Image segmentation is a computer vision technique that divides an image into several parts to facilitate analysis. The goal is to separate an image into distinct objects or regions that share common characteristics, such as color, intensity, shape, or other features.

Semantic segmentation is a method that focuses on how to divide an image into regions that belong to different semantic classes [68]. In this approach, each pixel is assigned a label indicating its belonging to one of the predefined classes. This means that all pixels of the same class receive the same label, without distinguishing between individual objects. In the context of the nerve detection task, semantic segmentation will result in a segmentation map with all pixels corresponding to nerve structures labeled as "nerve" with the remaining pixels labeled as "non-nerve."

2.3.1 Transposed Convolution

As discussed earlier, convolution and pooling layers are commonly used in CNNs to reduce the spatial dimensions of feature maps, allowing the network to learn increasingly abstract features. However, this downsampling comes at the cost of losing spatial information as the network becomes deeper. In tasks like semantic segmentation, where the output often needs to match the input size, so that each output pixel aligns with a specific input pixel, this loss can be problematic [68]. To address this, CNNs use a different type of layer called a *transposed convolution*.

The operation of a transposed convolutional layer is similar to that of the regular convolutional layer, except that it performs the convolution operation in the opposite direction [13]. In contrast to the regular convolution that reduces input size by sliding the filter and aggregating values, transposed convolution increases the size of the output by broadcasting input elements via the filter [68]. In doing so, it helps recover spatial information lost during earlier downsampling steps and restore the original resolution [68].

Different from regular convolution, where padding and stride are defined with respect to the input, in transposed convolution, these hyperparameters are defined in relation to the desired output [68]. Padding in transposed convolution determines how many rows and columns will be cropped from the output. For example, setting a padding of 1 removes one row and one column from each side of the transposed convolution output. The stride hyperparameter controls how much the output is expanded. A stride of 1 keeps the spatial size unchanged, while a larger stride increases the output size by inserting spaces between input elements before applying the filter.

A visual example of the transposed convolution operation is provided in Figure 2.11.

Given an input feature map of size $I_h \times I_w$, a filter of size $f_h \times f_w$, a vertical and horizontal stride s_h, s_w, and a vertical and horizontal padding p_h, p_w, the height O_h and width O_w of the output feature map produced by a transposed convolution are computed as

$$O_h = (I_h - 1) \times s_h + f_h - 2p_h, \quad O_w = (I_w - 1) \times s_w + f_w - 2p_w.$$

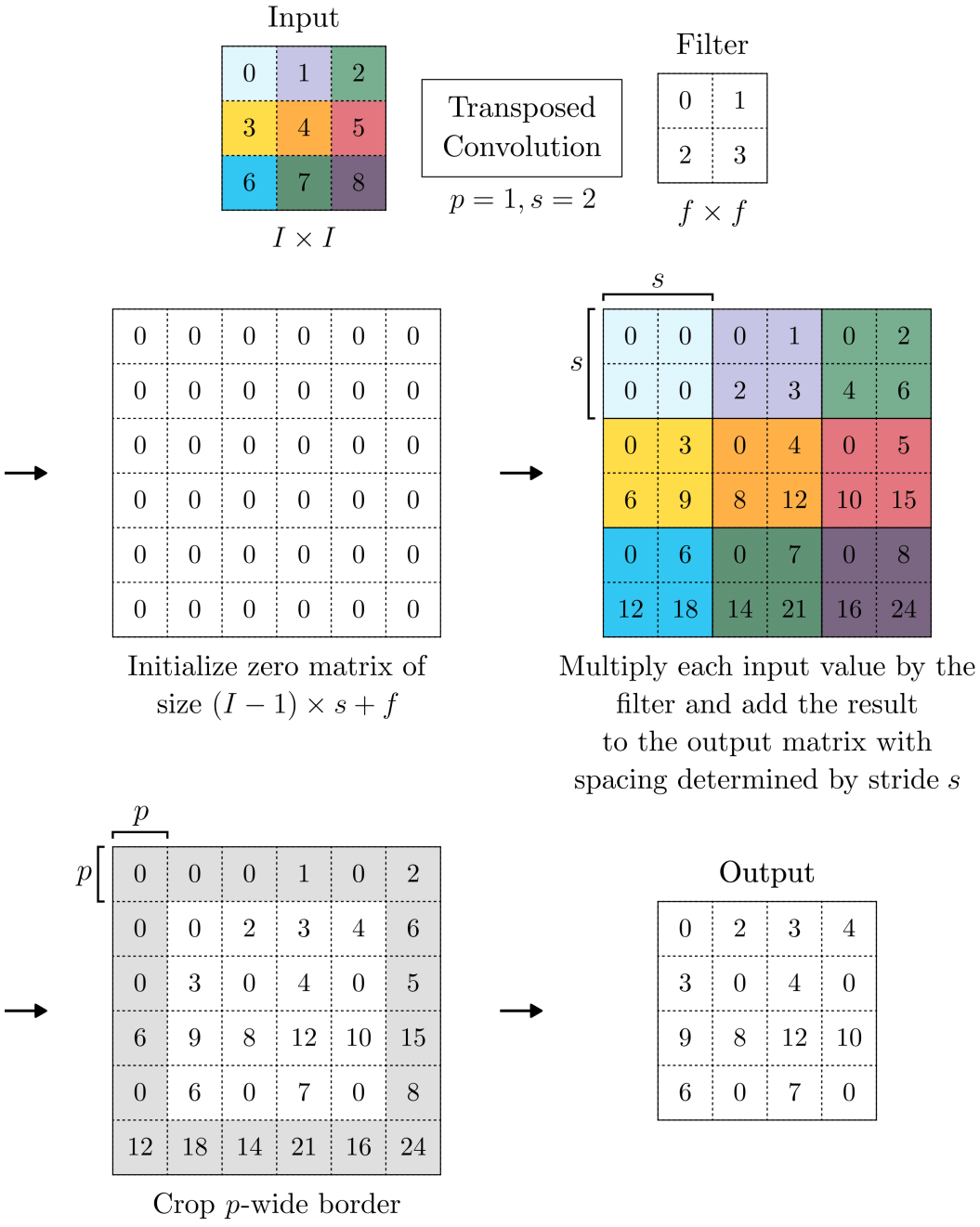


Figure 2.11: Transposed convolution applied to a 3×3 input using a 2×2 filter, with padding $p = 1$ and stride $s = 2$, resulting in a 4×4 output feature map. This is the reverse of the standard convolution (in terms of spatial dimensions) applied to a 4×4 input with the same filter size, padding, and stride, shown in Figure 2.9.

2.3.2 U-Net Model

In this thesis, I utilize the *U-Net* model, a fully convolutional neural network architecture developed by *Ronneberger et al.* [53] for biomedical image segmentation. Its architecture resembles a U-shape consisting of a *contracting path* (encoder) and an *expanding path* (decoder). This structure allows the network to efficiently capture both global and local features of the input image, making it particularly suitable for tasks that require precise localization, such as segmentation of neural structures.

The network architecture is illustrated in Figure 2.12. The contracting path (left side) reduces the image size to extract important features, while the expanding path (right side) restores the image resolution to create a detailed segmentation map. Skip connections between blocks pass high-resolution features from the encoder to decoder, which helps recover spatial information lost during downsampling and improve gradient flow.

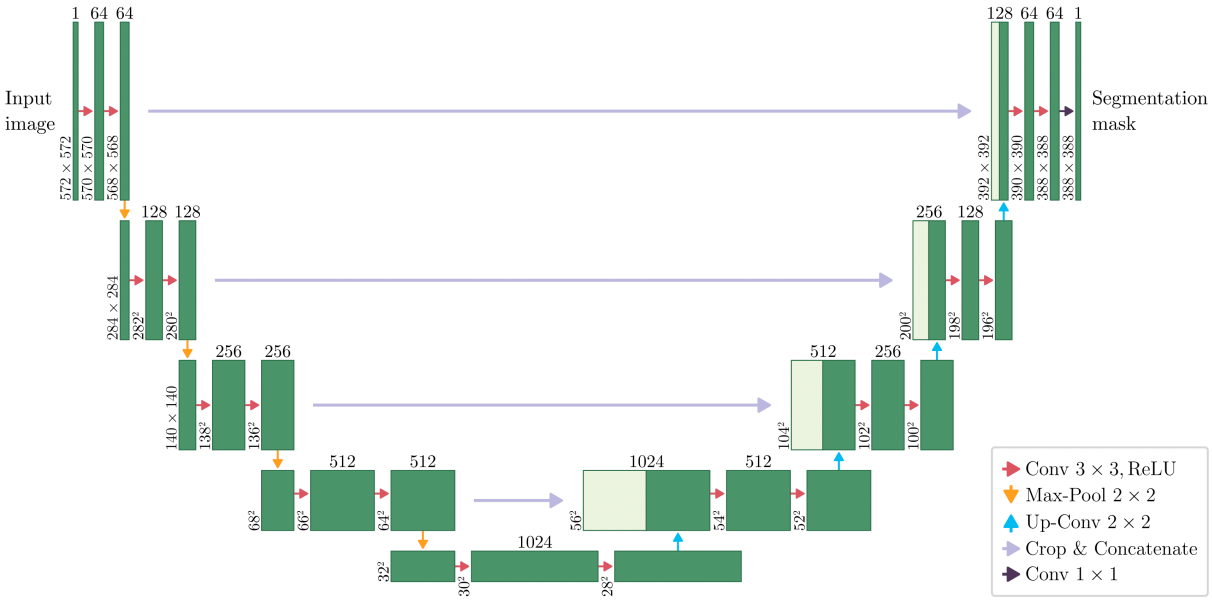


Figure 2.12: The original U-Net architecture. Each green box represents a multi-channel feature map, with the number of channels indicated on top. The x-y dimensions of each feature map are indicated at the lower left corner of the box. Arrows denote the operations such as convolution, max-pooling, up-convolution, and crop and concatenate. Light green boxes are feature maps from the contracting path that are cropped and concatenated to the corresponding feature maps in the expanding path.

Credit: Diagram redrawn from [53].

The following sections describe the main components of the U-Net architecture in more detail.

Contracting Path

The *contracting path* serves as the feature extractor and is responsible for capturing the context of the image at different resolutions through a sequence of convolutional blocks. Each block consists of two 3×3 valid convolutional layers, each followed by a ReLU activation function to introduce non-linearity.

After these convolutional layers, the output is passed through a downsampling layer with a 2×2 max-pooling operation with stride 2, where the spatial dimensions of the feature maps are reduced by half. At each downsampling step, the number of feature channels is doubled, increasing the network's ability to learn more complex patterns.

Bottleneck

At the bottleneck of the U-Net, the network reaches its lowest spatial resolution, having captured the most meaningful and high-level features of the input image. This stage serves as the transition point between the contracting and expanding paths, where the network has extracted all the necessary abstract features and is now ready to begin reconstructing the segmentation map.

Expanding Path

The *expanding path* is responsible for converting the abstract features into a segmentation map. It consists of repeated upsampling operations applied to feature maps, followed by 2×2 up-convolutions that reduce the number of feature channels by half [53].

At each step, the upsampled feature maps are concatenated with the corresponding cropped feature maps from the contracting path. Since the model uses valid convolutions, which result in the loss of border pixels at each layer, the feature maps from the contracting path must be cropped before concatenation to match the spatial dimensions of the feature maps in the expanding path.

After concatenation, the feature maps are processed by two 3×3 valid convolutional layers with ReLU, mirroring the convolutional blocks used in contracting path.

Skip Connections

Skip connections play a vital role in improving the precision of semantic segmentation results. They carry feature maps from the contracting path and concatenate them with the feature maps in the expanding path. This helps to preserve spatial information that may have been lost during downsampling. By concatenating high-resolution feature maps from the encoder with the upsampled feature maps in the decoder, the network can more accurately reproduce fine image details. In addition, skip connections help mitigate the vanishing gradient problem by making it easier for the gradients to flow from output layers to earlier layers of the network [18]. In Figure 2.12, the skip connections are indicated by the light purple arrows.

1×1 Convolution

At the final layer, the output is passed through a 1×1 convolution layer to map the feature maps from decoder output to the desired number of classes. This is followed by an activation function to produce the final segmentation map.

2.3.3 Loss Functions

A *loss function* is an algorithm to evaluate how well a network models the input data [49]. It measures the difference between the model's predictions and the actual values. This difference guides the optimization process and helps the model improve its accuracy. The ultimate goal is to find a set of trainable parameters that enable the model to make the most accurate predictions possible [51].

The choice of loss function depends on the model's architecture and the specific task at hand [62]. In neural structure segmentation, where nerve segments are much smaller than the surrounding background, traditional loss functions like Cross-Entropy can struggle due to class imbalance. The background, being the majority class, dominates the loss, leading the model to focus on it and ignore the smaller nerve segments. This imbalance can slow the convergence and cause the model to perform poorly on the minority class [51]. To address this issue, loss functions designed for imbalanced data should be considered. They help the model focus on the minority class by reducing the dominance of the majority class. In the following sections, I provide some commonly used loss functions for handling imbalanced data.

Weighted Cross-Entropy Loss

Weighted Cross-Entropy (WCE) *Loss* addresses class imbalance by increasing the loss contribution of the minority (foreground) class using a weighting factor α. The formula for WCE is given by

$$\mathcal{L}_{\text{WCE}}(y, p) = -\frac{1}{N} \sum_{i=1}^{N} \left[\alpha y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \right], \tag{2.11}$$

where y_i and p_i denote the ground truth label and predicted probability for pixel i, respectively, and N represents the total number of pixels [60]. The weight α is applied to the foreground class to compensate for class imbalance, while the background class remains unweighted. The value of α is dynamically calculated based on the predicted probabilities at the current training step as [60]

$$\alpha = \frac{N - \sum_{i=1}^{N} p_i}{\sum_{i=1}^{N} p_i}. \tag{2.12}$$

Focal Loss

Focal Loss enables training of high-accuracy models in the presence of a vast number of easy background examples [33]. While Weighted Cross-Entropy addresses the problem of class imbalance by assigning higher weights to the minority class, Focal Loss goes a step further by also taking into account the difficulty of individual examples. In large class imbalance settings, easily classified negatives may still dominate the loss and gradients [33]. To mitigate this, Focal Loss introduces a modulating factor that reduces the loss contribution from easy, well-classified examples and encourages the model to focus on hard, misclassified examples.

Let $y_i \in \{0, 1\}$ denote the ground truth label and $p_i \in [0, 1]$ the predicted probability for pixel i. Focal Loss features two hyperparameters: the class-balancing factor α and the focusing parameter γ. The parameter $\alpha \in [0, 1]$ addresses class imbalance by assigning weight α to the positive (foreground) class and a corresponding weight of $1 - \alpha$ to the negative (background) class. The focusing parameter $\gamma \geq 0$ controls how much the model reduces the influence of easy, well-classified examples during training by using the modulating terms $(1 - p_i)^\gamma$ for positive examples and p_i^γ for negative ones.

For N pixels, the Focal Loss is expressed as

$$\mathcal{L}_{\text{Focal}}(y, p) = -\frac{1}{N} \sum_{i=1}^{N} \left[\alpha\, y_i\, (1 - p_i)^{\gamma} \log(p_i) + (1 - \alpha)\, (1 - y_i)\, p_i^{\gamma} \log(1 - p_i)\right]. \quad (2.13)$$

When the model predicts correctly, that is, p_i is close to 1 for a positive example ($y_i = 1$) or close to 0 for a negative example ($y_i = 0$), the modulating factor approaches 0 and the loss is down-weighted [33]. For incorrect predictions where p_i differs significantly from the true label y_i, the modulating factor approaches 1. In this case, the loss is unaffected, the example fully contributes to the total loss.

With $\gamma = 0$, the Focal Loss is equivalent to the Weighted Cross-Entropy Loss [33]. As γ increases, the effect of the modulating factor also increases, placing more emphasis on hard, misclassified examples. According to the original paper by *Lin et al.* [33], $\gamma = 2$ was found to work best in their experiments.

Dice Loss

Dice Loss is commonly used in image segmentation tasks with imbalanced data. Unlike the previously discussed loss functions, Dice Loss does not require assigning weights to samples of different classes to establish the right balance between foreground and background [41]. Instead, it directly optimizes the spatial overlap between the predicted and ground truth regions. The *Dice Similarity Coefficient* (DSC), which will be discussed further in Section 2.3.4, measures this overlap. While the original DSC is computed using binary predictions and labels, Dice Loss uses relaxed predictions represented as probabilities in the range $[0, 1]$ instead of hard binary values 0 or 1 [52]. This makes the loss function differentiable and allows it to be optimized using gradient descent [52].

The Dice Loss is derived by subtracting the relaxed DSC from one, so that minimizing the loss corresponds to maximizing the overlap. It is defined as

$$\mathcal{L}_{\text{Dice}}(y, p) = 1 - \frac{2 \sum_{i=1}^{N} y_i p_i + \epsilon}{\sum_{i=1}^{N} y_i + \sum_{i=1}^{N} p_i + \epsilon}, \quad (2.14)$$

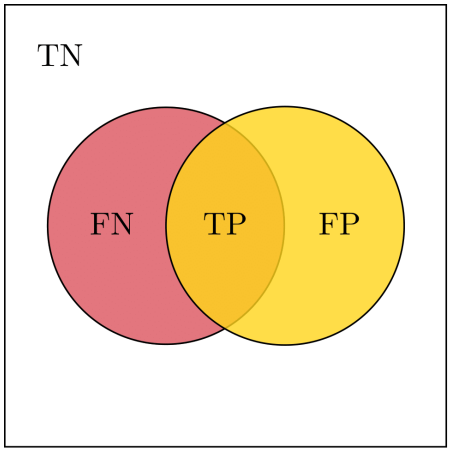
where y_i and p_i are ground truth and predicted probability for pixel i, respectively, and N represents the total number of pixels [60]. ϵ is a small constant added to prevent division by zero in cases where both sums are zero.

2.3.4 Segmentation Evaluation

The choice of evaluation metric can greatly influence how model performance is interpreted. Evaluation is only informative when the metrics are aligned with the specific task at hand. In this work, the segmentation task involves highly imbalanced data, where positive regions make up only a small portion of the image. In such cases, metrics such as Accuracy and Specificity (True Negative Rate) can be misleading, as a model may achieve high scores by predicting only the dominant negative (background) class.

To account for the imbalance, I focused on metrics that prioritize accurate detection of positive areas. In particular, I chose *True Positive Rate* (TPR) and *Dice Similarity Coefficient* (DSC). TPR measures how well the model detects positive regions, while DSC evaluates how closely the predicted regions match the ground truth. Both metrics are derived from the pixel-level classification outcomes. These outcomes are broken down into four categories:

- *True Positives* (TP): positive pixels correctly predicted as positive;
- *True Negatives* (TN) negative pixels correctly predicted as negative;
- *False Positives* (FP): negative pixels incorrectly predicted as positive;
- *False Negatives* (FN): positive pixels incorrectly predicted as negative.



		Prediction	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

Figure 2.13: Four components of the confusion matrix. The overlap between the ground truth (pink) and predicted mask (yellow) corresponds to true positives (TP). Pixels missed by the model are false negatives (FN, pink), while incorrect predictions are false positives (FP, yellow). All other pixels belong to the true negative (TN) region. The matrix on the right summarizes these relationships in standard confusion matrix form.

Credit: The figure is inspired by [47].

True Positive Rate

True Positive Rate (TPR), also known as sensitivity or recall, quantifies the ability of the segmentation model to correctly identify positive pixels. It measures the proportion of correctly predicted positive predictions among all actual positives. TPR is defined as

$$\text{TPR} = \frac{\text{Correctly classified actual positives}}{\text{All actual positives}} = \frac{\text{TP}}{\text{TP} + \text{FN}}. \tag{2.15}$$

A high TPR means that the model is able to correctly detect most of the regions belonging to the positive class, while a low TPR suggests that many regions are overlooked.

Dice Similarity Coefficient

Dice Similarity Coefficient (DSC), also known as Dice Score or Dice-Sørensen Coefficient, is the most widely used metric for evaluating medical image segmentation [45]. It quantifies segmentation accuracy by measuring the overlap between the predicted and ground truth regions. The DSC ranges from 0, indicating no overlap, to 1, which means perfect overlap.

The Dice metric is calculated as [32]

$$\text{DSC}(G, P) = \frac{2 \times |G \cap P|}{|G| + |P|} = \frac{2 \times \text{TP}}{(\text{TP} + \text{FN}) + (\text{TP} + \text{FP})} = \frac{2 \times \text{TP}}{2 \times \text{TP} + \text{FN} + \text{FP}}, \tag{2.16}$$

where G represents the actual positive regions and P the predicted ones.

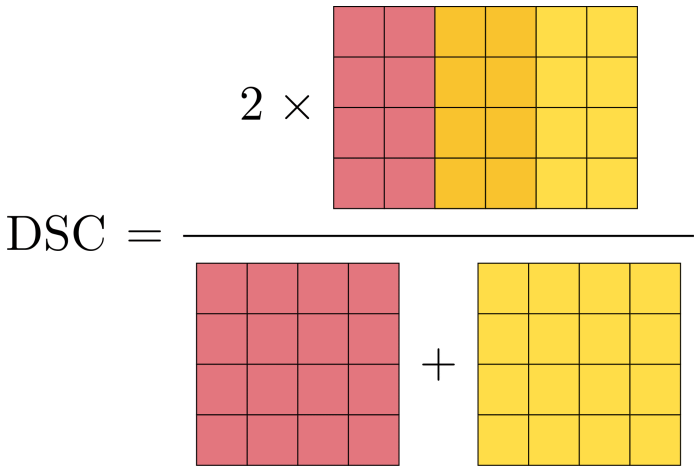


Figure 2.14: Visual representation of the Dice Similarity Coefficient (DSC). The overlapping region between the ground truth (pink) and predicted mask (yellow) is shown in orange.

Credit: The figure is inspired by [6].

2.4 Survival Analysis

In some studies, the exact timing of an event is not important. The outcome can simply be a binary "yes" or "no" and analyzed using logistic regression [26]. For example, in a study checking if a vaccine prevents infection, it may not matter if someone got sick on the second or tenth day after being exposed; the key question is whether they got infected or not. In other situations, especially those concerned with chronic conditions, the timing of the event is critically important [26]. In stroke research, for example, a recurrence that occurs within a few weeks after the first stroke has a completely different meaning than one that occurs many years later. Ignoring the timing and focusing only on whether the event happened discards valuable information and may lead to incorrect conclusions [26].

Survival analysis is used when the time until an event occurs is of primary interest. The main outcome variable in this analysis is the time until the event and is often referred to as the *failure time*, *survival time*, *event time*, or *time-to-event* [26]. Depending on the study, this variable may be measured in days, weeks, months, or other time units.

An *event* refers to a predefined experience of interest that marks the end of a follow-up period of a person or object [29]. In healthcare, typical events include death, relapse and recovery. In manufacturing, an event could be the failure of a machine component, and in the financial sector, it could be a loan default. Events of this nature are often referred to as *failures*, as they are usually associated with negative outcomes [29]. However, this is not always the case; survival time can also refer to the time until a positive event occurs [29]. For example, if the event of interest is returning to work after medical treatment, then "failure" represents a favorable outcome [29].

In many studies, not all subjects experience the event of interest before it ends. This situation is called *censoring*. Censoring occurs when the exact time of the event is unknown, but it is known that a subject remained event-free up to a certain time point. There are generally three reasons why censoring may occur [29]:

1. End of study: The subject does not experience the event before the end of the study.

2. Loss to follow-up: The subject becomes unreachable or unavailable for observation during the study period.

3. Withdrawal or competing event: The subject either voluntarily withdraws from the study or experiences an event that prevents the occurrence of the event of interest. For example, if the event of interest is death due to cancer and a patient dies in a traffic accident, the death due to the accident is considered a competing event [11].

Figure 2.15 graphically illustrates possible subject outcomes in a survival analysis. Of the six subjects, only B and E experience the event within the follow-up period. Subjects A and C complete the study without experiencing the event and are therefore censored due to the end of the study. The remaining two subjects, D and F, are also censored, but for different reasons: D is lost to follow-up, and F withdraws from the study.

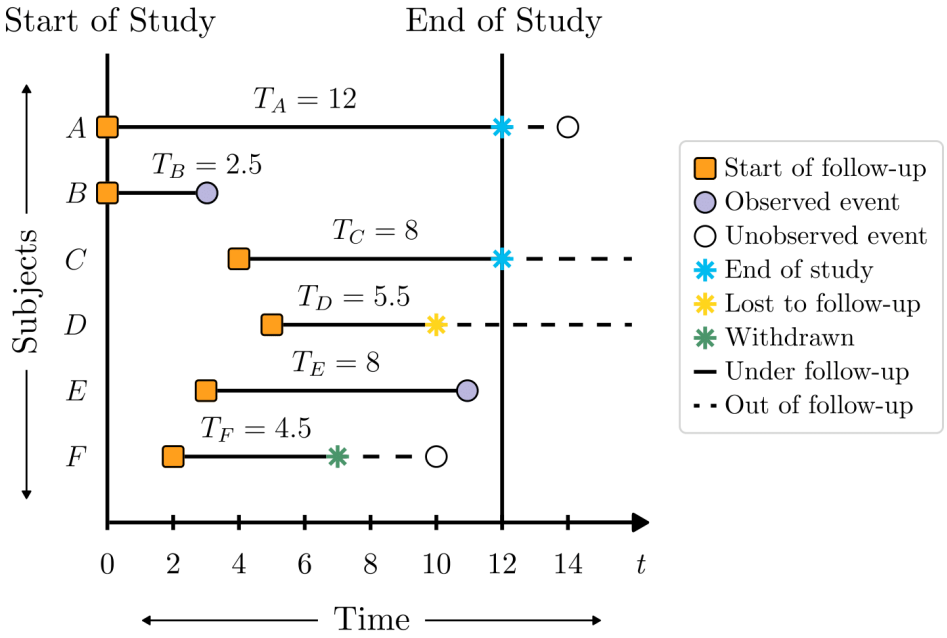


Figure 2.15: Possible subject outcomes in a survival analysis. Squares indicate the start of follow-up, filled circles represent observed events, and empty circles indicate unobserved events. Asterisks mark right-censored observations due to the end of the study, loss to follow-up, or withdrawal. Solid lines indicate the periods during which subjects were under follow-up, and dashed lines indicate the periods after censoring during which subjects were no longer observed.

Credit: The figure is inspired by [29].

It is possible that a censored subject experiences the event of interest after the follow-up period has ended. In this example, that is the case for subjects A and F. However, since these events occur after censoring, they are not included in the analysis. The only information available is that these subjects remained event-free for at least as long as they were followed. This scenario illustrates *right censoring*, which is the most common type

of censoring in survival analysis [29]. In such cases, the exact survival time is unknown because the observation period ends before the event occurs.

The other two types of censoring that can occur are *left censoring* and *interval censoring*. *Left censoring* happens when the event of interest has already occurred before the subject entered the study. In such cases, the exact time of the event is unknown, but it is known that it happened prior to the start of follow-up. For example, if a study is tracking recovery time after a medical procedure and some patients underwent the procedure before entering the study, their recovery time is left-censored.

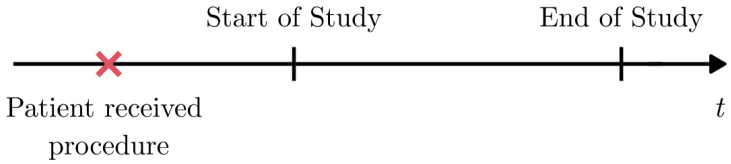


Figure 2.16: Left censoring: the medical procedure was performed before the start of the study, so the exact time of the start of recovery is unknown.

Credit: Example from [44].

Interval censoring is present when it is known that an event occurred within a certain time window, but the exact time is unknown [29]. This is usually the case in studies where subjects are assessed only at fixed time points (e.g., health check-ups) [26]. Consider a case in which a subject tests negative for a virus at time t_1, but is positive at the next visit at time t_2. Although the exact time of infection is unknown, it is known to have occurred within the interval (t_1, t_2). In this case, the subject is considered interval-censored between t_1 and t_2.

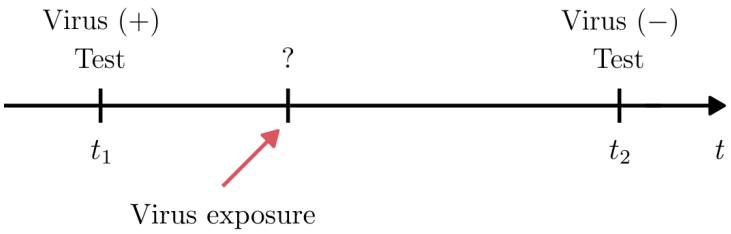


Figure 2.17: Interval censoring: the event occurred between two visits, but the exact time of virus exposure is unknown.

Credit: Example from [9].

2.4.1 Cox Regression

Survival and Hazard Functions

In survival analysis, the two main functions used to describe time-to-event data are the *survival function* and the *hazard function*. Survival models, including the Cox proportional hazards model, are usually formulated in terms of these two functions.

The *survival function*, denoted by $S(t)$, gives the probability that a subject survives longer than some specified time t [29]. Formally, it is defined as

$$S(t) = P(T > t), \tag{2.17}$$

where $T \geq 0$ is a random variable representing a subject's survival time. The survival function is non-increasing over time. $S(t)$ is always 1 at $t = 0$; all subjects survive at least to time zero [21]. As $t \to \infty$, $S(t) \to 0$, that is, if the study period were infinitely long, the probability of survival would eventually drop to zero [29].

The *hazard function*, denoted by $h(t)$, describes the instantaneous potential for the event to occur at time t, given that the subject has survived up to time t [29]. In contrast to the survival function, which focuses on not having an event, the hazard function focuses on the rate at which events occur [29].

Mathematically, the hazard function is defined as

$$h(t) = \lim_{\Delta t \to 0} \frac{P(t \leq T < t + \Delta t \mid T \geq t)}{\Delta t} \geq 0, \tag{2.18}$$

where Δt denotes a short interval of time [29]. The numerator represents the conditional probability that the event occurs within the interval $[t, t + \Delta t)$, given survival up to time t. Dividing by Δt turns this probability into a rate per unit of time, which can take any non-negative value; this rate depends on the time unit used (e.g., days, weeks, months, or years) [29]. Taking the limit as $\Delta t \to 0$ yields the instantaneous rate of occurrence of the event at time t.

In contrast to a survival function, which always starts at 1 and goes down to 0, the hazard function is not limited to a monotonic shape [29]. It can start at any non-negative value and may increase, decrease, or fluctuate over time. For any given time t, the hazard function $h(t)$ is always non-negative and has no upper bound [29].

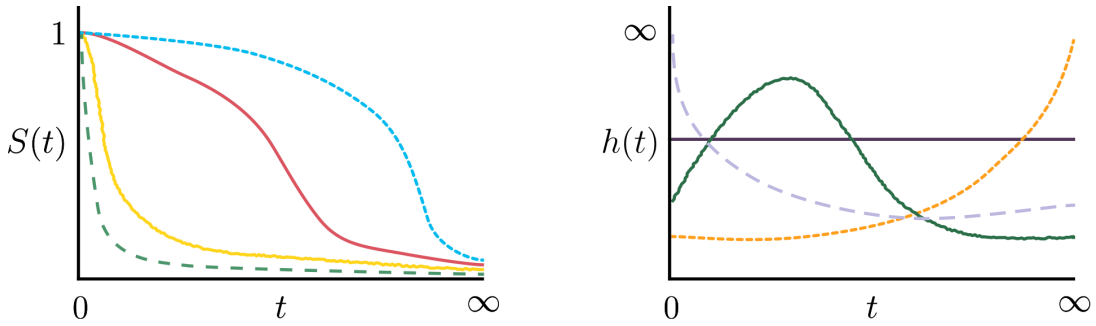


Figure 2.18: Survival functions (left) and hazard functions (right).

Credit: The figure is inspired by [29].

Cox Proportional Hazards Model

The *Cox proportional hazards* (CPH) *model* is a semi-parametric regression model based on the hazard function, used to examine how different factors affect survival time. These factors, called covariates or predictor variables, can be continuous measurements, such as age, or categorical variables, such as treatment group [15].

Let $x_i = (x_{i1}, \ldots, x_{in})$ denote the vector of n covariates for subject i, where x_{ij} is the value of the j-th covariate. The CPH model defines the hazard function as

$$h(t \mid x_i) = h_0(t) \exp\left(\sum_{j=1}^{n} \beta_j x_{ij}\right) = \underbrace{h_0(t)}_{\substack{\text{baseline} \\ \text{hazard}}} \overbrace{\exp\left(\beta^\top x_i\right)}^{\text{relative hazard}}, \tag{2.19}$$

where $h_0(t)$ is the *baseline hazard function* and $\beta = (\beta_1, \ldots, \beta_n)^\top$ is the vector of regression coefficients [29]. The baseline hazard plays a role similar to the intercept in a linear regression in that it represents the hazard for individuals whose covariate values are either zero (in the case of continuous variables) or set to the reference category (for categorical variables) [46]. The exponential term, $\exp(\beta^\top x)$, referred to as the *relative hazard function*, describes the (relative) effects of predictors on the risk of the event [26].

The model is termed *semi-parametric* because it combines both parametric and non-parametric elements. It makes a parametric assumption concerning the effect of the predictors on the hazard function (through the exponential term), but makes no assumptions about the shape of the baseline hazard function $h_0(t)$ [26]. This is advantageous in practice because the form of the true hazard function is often unknown or too complex to model [26]. Also, the main interest in most cases is in understanding how the covariates

affect the hazard rather than in the shape of $h_0(t)$, and the Cox approach allows for this by leaving $h_0(t)$ unspecified [26].

The term *proportional* in proportional hazards regression refers to the assumption that hazard ratios between individuals with different covariate values remain constant over time [27]. For example, if the hazard of death at time t for treated patients is half that of control patients at time t, then the same ratio will be in effect at any other time [26]. In other words, treated patients have a consistently lower risk of death throughout the entire follow-up period [26] (see Figure 2.19).

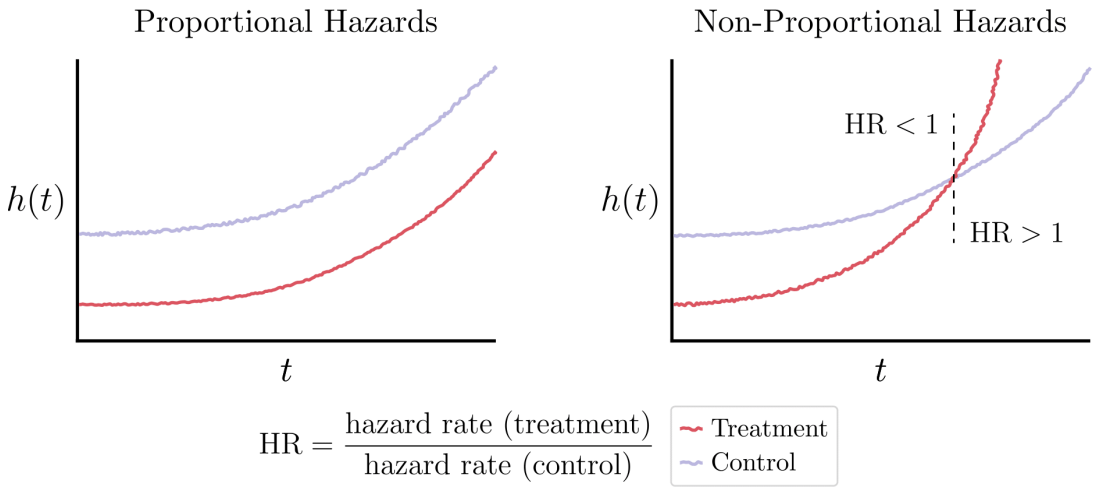


Figure 2.19: The proportional hazard assumption. In the proportional case, the treatment group (pink) consistently has a lower hazard than the control group (purple), and the hazard ratio remains constant over time. In the non-proportional case, the curves cross, violating the proportional hazards assumption.

Credit: The figure is inspired by [40].

This assumption can be expressed formally by comparing two individuals i and i', with covariate vectors $x_i = (x_{i1}, \ldots, x_{in})$ and $x_{i'} = (x_{i'1}, \ldots, x_{i'n})$. The ratio of their hazard rates is

$$\frac{h(t \mid x_i)}{h(t \mid x_{i'})} = \frac{h_0(t) \exp\left(\sum_{j=1}^{n} \beta_j x_{ij}\right)}{h_0(t) \exp\left(\sum_{j=1}^{n} \beta_j x_{i'j}\right)} = \exp\left[\sum_{j=1}^{n} \beta_j (x_{ij} - x_{i'j})\right]. \tag{2.20}$$

The only time-dependent term, the baseline hazard $h_0(t)$, cancels out, so the hazard ratio depends only on the difference in covariate values between individuals.

When the differences in the hazards between groups with different values of covariates are not proportional over time, the assumption of proportional risks is violated and

the Cox model becomes not appropriate [27]. In such situations, other approaches can be used. These include dividing the follow-up time into intervals and fitting separate Cox models for each period, using an extended Cox model that includes time-dependent covariates, or employing other methods that don't rely on the proportional hazards assumption [29].

Partial Likelihood

The regression coefficients β of the CPH model are estimated by maximizing a partial likelihood function [29]. The derivation of this function can be described as follows. Let $t_1 < t_2 < \ldots < t_k$ denote the unique ordered failure times of the n individuals, and let ℓ_j be the label of the subject who fails at t_j [10]. Thus, $\ell_j = i$ if and only if $T_i = t_j$ [10]. For now, assume there are no tied failure times (tied censoring times are allowed), so that $k = n$ [26]. At each failure time t_j, the set of individuals at risk of failure, that is, those who have not yet failed or been censored, is called the *risk set*, denoted by $\mathcal{R}_j$ [26]. Formally, it is defined as

$$\mathcal{R}_j = \{i \mid T_i \geq t_j\}, \tag{2.21}$$

where T_i is the observed time (either failure or censoring) for subject i [63]. A visual representation of these definitions is given in Figure 2.20.

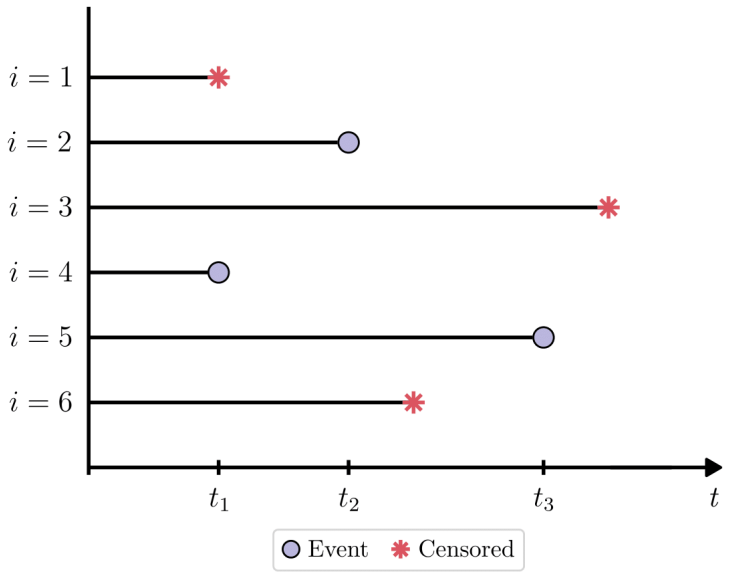


Figure 2.20: Failure and censoring of six individuals. Circles denote observed failures, and asterisks indicate censoring. Failure times are t_1, t_2, t_3. Risk sets are $\mathcal{R}_1 = \{1, 2, 3, 4, 5, 6\}$; $\mathcal{R}_2 = \{2, 3, 5, 6\}$; $\mathcal{R}_3 = \{3, 5\}$.

Credit: The figure is inspired by [10].

In *Cox and Oakes (1984)* [10], it is explained that when the baseline hazard function $h_0(t)$ is unknown, failure times t_j provide little or no information about regression coefficients β, since their distribution depends heavily on $h_0(t)$. Instead of looking at *when* events happen, Cox proposed focusing on the *order* in which individuals fail [10].

The probability of observing the sequence of failures, where subject ℓ_1 fails at time t_1, subject ℓ_2 fails at time t_2, and so on, can be expressed as

$$L(\beta) = P(L_1 = \ell_1, L_2 = \ell_2, \ldots, L_k = \ell_k), \tag{2.22}$$

where L_j is the random variable corresponding to the subject who fails at time t_j [35]. Cox computed this joint probability as a product of conditional probabilities at each failure time [35].

The conditional probability that ℓ_j fails at t_j given that one individual from the risk set $\mathcal{R}_j$ fails at that time t_j, is given by [35, 26]

$$\begin{aligned} P(\ell_j \text{ fails at } t_j \mid 1 \text{ failure from } \mathcal{R}_j \text{ at } t_j) &= \frac{P(\ell_j \text{ fails at } (t_j, t_j + \Delta t))}{P(1 \text{ failure from } \mathcal{R}_j \text{ at } (t_j, t_j + \Delta t))} \\ &\approx \frac{h(t_j \mid x_{\ell_j}) \cdot \cancel{\Delta t}}{\sum_{i \in \mathcal{R}_j} h(t_j \mid x_i) \cdot \cancel{\Delta t}} \\ &= \frac{h_0(t_j) \exp(\beta^\top x_{\ell_j})}{\sum_{i \in \mathcal{R}_j} h_0(t_j) \exp(\beta^\top x_i)} \\ &= \frac{\exp(\beta^\top x_{\ell_j})}{\sum_{i \in \mathcal{R}_j} \exp(\beta^\top x_i)}. \end{aligned} \tag{2.23}$$

Thus, the Cox's partial likelihood for β is

$$L(\beta) = \prod_{j=1}^{k} \frac{\exp(\beta^\top x_{\ell_j})}{\sum_{i \in \mathcal{R}_j} \exp(\beta^\top x_i)}. \tag{2.24}$$

In practice, it is easier to estimate the coefficients by working with the logarithm of the partial likelihood instead of the likelihood itself [68]. Taking the logarithm transforms the product into a sum, which simplifies calculations and avoids numerical issues that can arise from multiplying a large number of small probabilities. The corresponding

log-partial likelihood is

$$\ell(\beta) = \log L(\beta) = \sum_{j=1}^{k} \left[\beta^\top x_{\ell_j} - \log \left(\sum_{i \in \mathcal{R}_j} \exp(\beta^\top x_i) \right) \right]. \tag{2.25}$$

Partial Likelihood with Tied Event Times

The partial likelihood for the Cox model was developed under the assumption of continuous event times [37]. However, in real-world data sets, it is common to encounter tied event times, where multiple subjects experience the event at exactly the same time [37]. Ties may occur because continuous event times are grouped into intervals or because the event time scale is discrete [37].

Several methods have been developed to handle tied events in the CPH model. Before describing the one I used, it is necessary to introduce the notation used in its formulation.

As in the standard partial likelihood, let $t_1 < t_2 < \ldots < t_k$ denote the unique ordered failure times of the n individuals. Now, $k < n$ because multiple individuals may fail at the same time. At each failure time t_j, R_j represents the risk set, consisting of individuals still under follow-up just before t_j. Let $\mathcal{D}_j \subseteq \mathcal{R}_j$ denote the set of individuals who fail at time t_j, and define $d_j = |\mathcal{D}_j|$ as the number of tied failures. A visual illustration of this setup is provided in Figure 2.21.

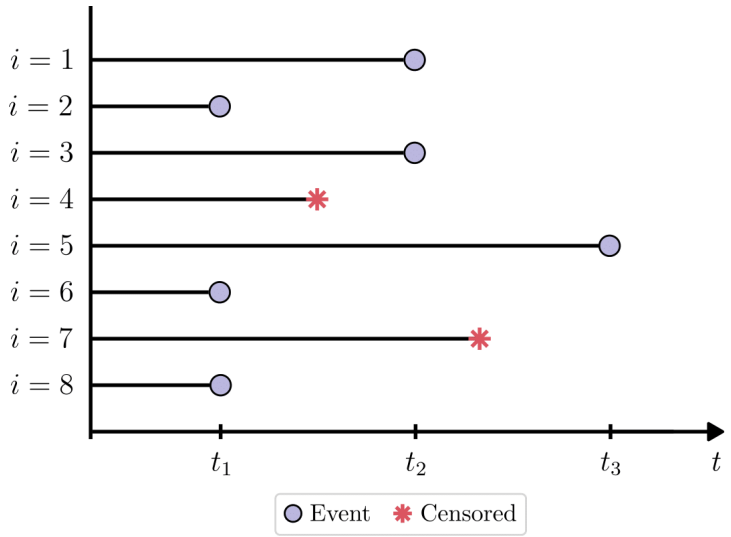


Figure 2.21: Failure and censoring of eight individuals, with tied failures occurring at t_1 and t_2. Circles indicate observed events, and asterisks denote censoring. Failure times are t_1, t_2, t_3. Risk sets and event sets are: $\mathcal{R}_1 = \{1, 2, 3, 4, 5, 6, 7, 8\}$, $\mathcal{D}_1 = \{2, 6, 8\}$; $\mathcal{R}_2 = \{1, 3, 5, 7\}$, $\mathcal{D}_2 = \{1, 3\}$; $\mathcal{R}_3 = \{5\}$, $\mathcal{D}_3 = \{5\}$.

Credit: The figure is inspired by [10].

The three commonly used approaches for handling tied event times are *exact*, *Breslow's*, and *Efron's* methods. Among the three, the exact method is the most accurate, as it accounts for all possible orderings of tied events at each failure time. Breslow's method is the simplest in terms of computation, but the least accurate [37]. Efron's method is a middle ground between Breslow's and the exact method. Estimates obtained with Efron's method are usually very close to those from the exact method, unless the number or proportion of tied events is extremely large relative to the size of the risk set [37, 26].

For large datasets with many ties, the Efron and Breslow approximations are usually preferred because they offer significant computational savings at a reasonably high accuracy. In contrast, for small datasets, the preference is given to the exact method. Since the dataset in this thesis is relatively small, I used the exact method to obtain the most accurate estimates.

Formally, the exact partial likelihood is given by [63]

$$L_{\text{Exact}}(\beta) = \prod_{j=1}^{k} \left[\sum_{(q_1,q_2,\ldots,q_{d_j})=(1,2,\ldots,d_j)} \prod_{u=1}^{d_j} \left(\frac{\exp\left(\beta^\top x_{q_u}\right)}{\sum\limits_{i\in\mathcal{R}_j} \exp(\beta^\top x_i) - \sum\limits_{s=u}^{d_j} \exp(\beta^\top x_{q_s})} \right) \right], \quad (2.26)$$

where $(q_1, q_2, \ldots, q_{d_j})$ are indices corresponding to one of the $d_j!$ permutations of the tied individuals in $\mathcal{D}_j$.

In the absence of tied event times, that is, when $d_j = 1$ for all $j = 1, \ldots, k$, (2.26) reduces to the standard partial likelihood (2.24) used in the continuous model [10].

Estimation of β coefficients

Parameter estimates are obtained by taking partial derivatives of the log-partial likelihood with respect to each coefficient β_j and setting them equal to zero [29],

$$\frac{\partial \ell(\beta)}{\partial \beta_j} = 0, \quad j = 1, \ldots, n. \quad (2.27)$$

This results in a system of equations that cannot be solved analytically for β, so, typically, the optimal values are found using numerical methods such as the Newton-Raphson algorithm [61]. This method is conceptually similar to gradient descent, except that it also uses second-order information [61].

The algorithm begins with an initial guess $\hat{\beta}^{(0)}$. At each iteration i, it computes the gradient (called the score vector) $U(\hat{\beta}^{(i)})$ and the negative second derivative (called the observed information matrix) $I(\hat{\beta}^{(i)})$ of the log-partial likelihood function with respect to the coefficients β [61, 26]. These are then used to update the coefficient estimates according to the Newton-Raphson update rule, defined as [61, 26]

$$\hat{\beta}^{(i+1)} = \hat{\beta}^{(i)} + I\left(\hat{\beta}^{(i)}\right)^{-1} U\left(\hat{\beta}^{(i)}\right). \tag{2.28}$$

The iterative process is repeated until the convergence criterion is satisfied.

Interpretation of Cox Regression Results

After fitting the CPH model, the next step is to interpret its output. This involves [29]

1. Assessing the effect of covariates using hazard ratios;
2. Checking if the covariates have a significant effect;
3. Obtaining a confidence interval for the effect;
4. Verifying the proportional hazards assumption to ensure model validity.

Each of these points is described in detail below.

1. Effect of covariates on hazard In the CPH model, the effect of covariates is expressed through the *hazard ratio* (HR) [29]. A hazard ratio compares the predicted hazard between two individuals who differ in one predictor, while all other predictors are held constant [46]. For a continuous covariate, it quantifies how the hazard changes with a one-unit increase in that variable [46]. For a categorical covariate, it compares individuals in a specific category with those in the reference category [46].

Consider the continuous case. Let individuals i and i' have covariate vectors $x_i = (x_{i1}, \ldots, x_{in})$ and $x_{i'} = (x_{i'1}, \ldots, x_{i'n})$, respectively, where all covariates are identical except for the k-th one. That is, $x_{ik} = x_k + 1$, $x_{i'k} = x_k$, and $x_{ij} = x_{i'j}$ for all $j \in \{1, \ldots, n\} \setminus \{k\}$ [46]. Their hazard functions are then

$$\begin{aligned} h(t \mid x_i) &= h_0(t) \times \exp(\beta_1 x_{i1} + \cdots + \beta_k(x_k + 1) + \cdots + \beta_n x_{in}), \\ h(t \mid x_{i'}) &= h_0(t) \times \exp(\beta_1 x_{i'1} + \cdots + \beta_k x_k + \cdots + \beta_n x_{i'n}). \end{aligned}$$

The hazard ratio between these two individuals is

$$\begin{aligned} \text{HR} = \frac{h(t \mid x_i)}{h(t \mid x_{i'})} &= \frac{h_0(t) \times \exp(\cdots + \beta_k(x_k + 1) + \cdots)}{h_0(t) \times \exp(\cdots + \beta_k x_k + \cdots)} \\ &= \frac{\exp(\cdots + \beta_k(x_k + 1) + \cdots)}{\exp(\cdots + \beta_k x_k + \cdots)} \\ &= \frac{\exp(\cdots + \beta_k x_k + \cdots) \times \exp(\beta_k)}{\exp(\cdots + \beta_k x_k + \cdots)} \\ &= \exp(\beta_k). \end{aligned} \tag{2.29}$$

This implies that

$$h(t \mid x_i) = \exp(\beta_k) \cdot h(t \mid x_{i'}). \tag{2.30}$$

Thus, $\exp(\beta_k)$ is the factor by which the hazard is multiplied if the k-th covariate increases by 1 unit, holding all other variables constant.

In the categorical case, the interpretation of the hazard ratio is analogous to that in the continuous case. For a binary covariate x_j, coded as 1 for exposed and 0 for unexposed individuals, $\exp(\beta_j)$ quantifies how the hazard of the event changes for individuals with the risk factor compared to those without it [54].

A HR greater than 1 means the event is more likely to occur, and a ratio less than 1 means an event is less likely to occur [15]. A hazard ratio equal to 1 means the covariate has no effect on the hazard of the event [15].

For HR > 1, $100\% \times (\text{HR} - 1)$ represents the percentage increase in hazard for $x_{ij} = x_j + 1$ compared to $x_{ij} = x_j$ for a continuous predictor, or for a specific category of x_{ij} compared to its reference category for a categorical predictor [46]. For HR < 1, $100\% \times (1 - \text{HR})$ represents the percentage decrease in hazard [46].

As an example, consider age at diagnosis as a continuous covariate in a CPH model assessing breast cancer survival. If the estimated regression coefficient for age is $\beta = 0.05$, the corresponding HR is $\exp(0.05) \approx 1.05$. This means that each additional year of age increases the risk of death by approximately 5%, assuming that all other covariates in the model remain constant (the patients being compared are identical in all measured characteristics except for age). Thus, older patients have a slightly higher risk of death compared to younger patients with the same clinical profile.

Conversely, suppose the covariate is categorical and inversely related (i.e., has a negative coefficient). For instance, let $\beta = -0.9$ for a binary variable indicating whether

the patient underwent chemotherapy (0 = no, 1 = yes). The hazard ratio in this case is $\exp(-0.9) \approx 0.41$. This means that patients who underwent therapy have 41% of the risk of those who did not. In other words, receiving chemotherapy is associated with a 59% lower risk of death, again, assuming that all other covariates remain constant.

2. Test for significance of effect To determine whether a covariate has a statistically significant effect on the hazard, *p-value* is used [29]. The p-value, which ranges from 0 to 1, represents the probability of obtaining the observed result or an even more extreme one if the null hypothesis is true [39]. In Cox regression, this means evaluating how likely it is to obtain a hazard ratio as extreme as the one observed, assuming that the covariate actually has no effect on the hazard.

To decide whether to reject or retain (not reject) the null hypothesis (which is that there is no association between the covariate and the hazard), the p-value is compared to a threshold α, called the *significance level* [39]. This threshold is determined before the test and is typically set at 0.05 [39]. A significance level of 0.05 corresponds to a 5% risk of concluding that an effect is present when it is actually absent [43].

If the p-value is less than or equal to the significance level ($p \leq \alpha$), the null hypothesis is rejected and the covariate is considered to have a statistically significant association with the hazard [43]. If the p-value is greater than the significance level ($p > \alpha$), the null hypothesis is not rejected, and the association is not considered statistically significant [43]. In such cases, it may be reasonable to refit the model without that covariate, as it is unlikely to provide meaningful information [43].

3. Confidence interval for effect In Cox regression, *confidence intervals* (CI) represent the degree of uncertainty of the estimated hazard ratios.

A *confidence interval* is a range of values within which the estimate is expected to lie a certain percentage of the time if the experiment were repeated or re-sampled under the same conditions [4]. To calculate a confidence interval, three components are required: the point estimate (in Cox regression, this is the estimated coefficient $\hat{\beta}$), its standard error, and the critical value corresponding to the desired confidence level.

The *standard error* (SE) represents the uncertainty in the estimated coefficient $\hat{\beta}$ for a covariate. It reflects how much the estimate might vary if the study were repeated with different samples. The SE is calculated as

$$\mathrm{SE}(\hat{\beta}) = \sqrt{\widehat{\mathrm{Var}}(\hat{\beta})}, \tag{2.31}$$

where $\widehat{\text{Var}}(\hat{\beta})$ is the estimated variance of the coefficient [29].

The *confidence level* refers to the probability that the confidence interval will contain the estimate if the test were repeated many times [4]. It is usually set to $1 - \alpha$, where α is the significance level used in statistical test [4]. If the alpha value used for statistical significance is 0.05, then the confidence level will be $1 - 0.05 = 0.95$ or 95%. This means that 95 out of 100 times the estimate will lie between the upper and lower values specified by the CI [4].

The *critical value*, denoted by z, indicates how many standard deviations away from the estimate are needed to achieve the desired confidence level for CI [4]. If the data follows a normal distribution, or if the sample size is large ($n > 30$) and is approximately normally distributed, the standard normal distribution is used to find the critical value [4]. In a two-tailed CI, significance level α is divided equally between the two tails (see Figure 2.22). To find the value of z the cumulative probability of $1 - \alpha/2$ is used [14]. For a 95% confidence level, this gives $1 - 0.025 = 0.975$, which corresponds to $z = 1.96$ [14].

The confidence interval for the hazard ratio is computed in two steps [29]:

1. First, a confidence interval is calculated for the regression coefficient $\hat{\beta}$, given by

$$\text{CI} = \hat{\beta} \pm z \cdot \text{SE}(\hat{\beta}), \tag{2.32}$$

   where $\text{SE}(\hat{\beta})$ is the standard error of the coefficient and z is the critical value (1.96 for 95% CI).

2. The confidence interval for the hazard ratio is then obtained by exponentiating the two limits, resulting in

$$\text{CI}_{\text{HR}} = \left[\exp\left(\hat{\beta} - z \cdot \text{SE}(\hat{\beta})\right),\ \exp\left(\hat{\beta} + z \cdot \text{SE}(\hat{\beta})\right)\right]. \tag{2.33}$$

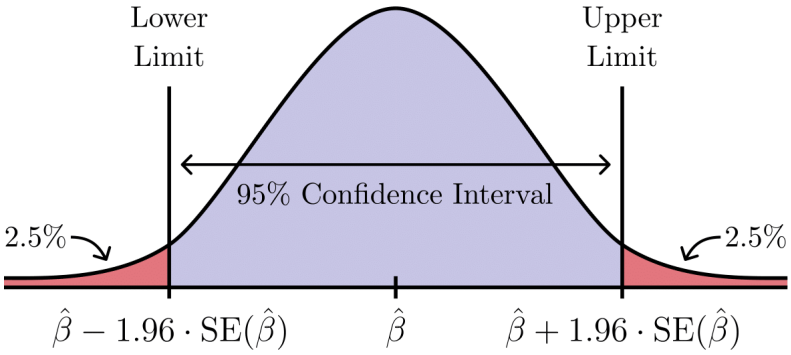


Figure 2.22: 95% confidence interval for the estimated coefficient $\hat{\beta}$.

The interpretation of confidence intervals is closely related to the interpretation of p-values. If the p-value is below the significance level α, the CI corresponding to that significance level will not include 1, indicating a statistically significant effect [2]. If the p-value is above α, the CI will contain 1, suggesting no significant association between the covariate and the hazard [2].

4. Test for proportional hazards assumption The proportional hazards (PH) assumption can be checked using graphical methods and statistical tests based on the *Schoenfeld residuals*.

The Schoenfeld residual, computed at each event time t_j, compares the *observed* covariates of the individual who fails at that time with what would be *expected* if the CPH model with constant β were correct [34].

Let x_{ℓ_j} denote the covariate vector of the individual ℓ_j who experiences the event at time t_j, and let $\mathcal{R}_j$ denote the risk set at that time. The Schoenfeld residual r_j is defined as the difference between the observed covariate vector x_{ℓ_j} and its expected value $\bar{x}(t_j)$, which is calculated as a hazard-weighted average of covariate values among individuals in the risk set $\mathcal{R}_j$ [29]. Formally, the residual vector is expressed as [34, 66]

$$\begin{aligned} r_j &= x_{\ell_j} - \sum_{i \in \mathcal{R}_j} x_i \cdot P(\text{subject } i \text{ fails at } t_j) \\ &= x_{\ell_j} - \sum_{i \in \mathcal{R}_j} x_i \cdot \frac{\exp(\beta^\top x_i)}{\sum_{i \in \mathcal{R}_j} \exp(\beta^\top x_i)} \\ &= x_{\ell_j} - \frac{\sum_{i \in \mathcal{R}_j} x_i \cdot \exp(\beta^\top x_i)}{\sum_{i \in \mathcal{R}_j} \exp(\beta^\top x_i)} \\ &= x_{\ell_j} - \bar{x}(t_j). \end{aligned} \tag{2.34}$$

If the residual for the k-th covariate, r_{jk}, is positive, it means that the observed value $x_{\ell_j k}$ is higher than the expected value $\bar{x}_k(t_j)$ under the CPH model [17]. Conversely, a negative value indicates that the observed covariate value is lower than expected.

Graphical method involves plotting the Schoenfeld residuals against time for each covariate. If the PH assumption holds, the residuals should be randomly scattered around a horizontal line. A plot showing a non-random pattern over time is evidence of a violation of the PH assumption [34].

Statistical test is a more objective approach than visual inspection [29]. It checks whether there is a statistically significant relationship between Schoenfeld residuals and time. The test involves three steps [29]:

1. Fit a CPH model and compute the Schoenfeld residuals for each predictor.
2. Rank the event times in ascending order.
3. Test the correlation between the Schoenfeld residuals and the ranked event times obtained in steps 1 and 2.

The null hypothesis is that there is no correlation [29]. If the p-value is below the significance level (typically 0.05), the null hypothesis is rejected, and it is concluded that the PH assumption is violated [29]. Conversely, if the p-value is greater than the significance level, the PH assumption is considered to be satisfied [24].

The example shown in Figure 2.23 demonstrates both the case when the PH assumption is met and the case when it is violated. The left subplot corresponds to a model with a covariate that satisfies the PH assumption. The Schoenfeld residuals are randomly scattered and the test yields p-value greater than the significance level $\alpha = 0.05$, thereby validating the assumption of constant effects over time. In contrast, the right subplot shows a model with a covariate that violates the PH assumption. The covariate is time-dependent as residuals follow a trend over time. The statistical test confirms PH violation with a p-value below α.

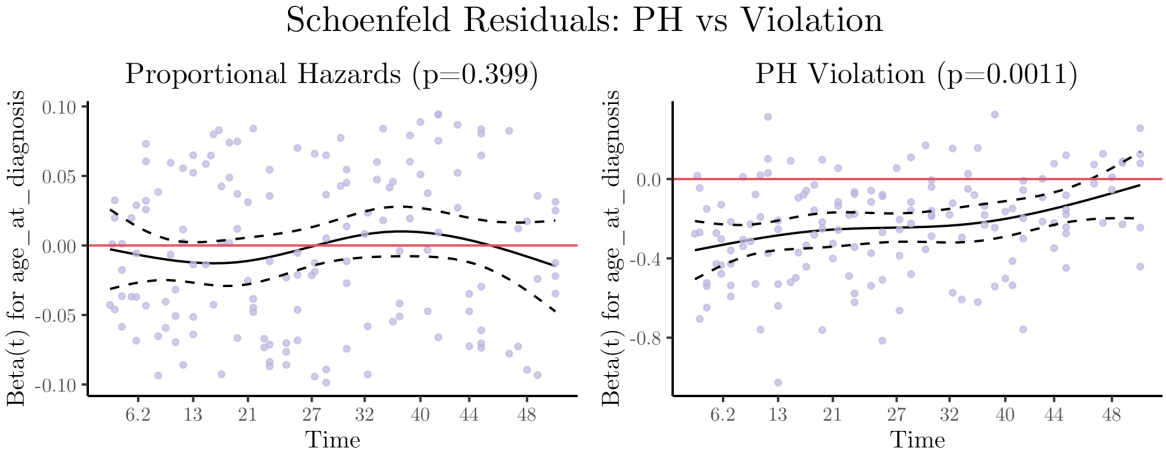


Figure 2.23: Schoenfeld residual plots. The covariate on the left satisfies the proportional hazards assumption, while the one on the right violates it.

2.4.2 Survival Model Evaluation

Evaluating the performance of survival models requires metrics that take into account both the timing of events and the fact that some events may not be observed. Traditional regression metrics, such as Mean Squared Error, are not suitable for evaluating Cox models because they do not account for the time component and cannot handle censored data [21]. A widely used evaluation metric in survival analysis is the *concordance index*, also known as Harrell's C or C-index [1].

The main idea behind the C-index lies in a pairwise comparison. Instead of evaluating the predictions for each subject separately, the accuracy of the model is assessed by comparing pairs of individuals. If one subject lives longer than the other, the model should predict a lower risk for the one who lives longer and, conversely, a higher risk for the one who dies earlier. The more often the model correctly ranks such pairs, the higher the C-index.

Formally, the C-index is defined as the proportion of all comparable pairs of individuals for which the model correctly ranks the predicted risks relative to the observed outcomes [21]. A pair is considered *comparable* if at least one of the individuals has experienced the event and if it is possible to determine who lived longer. This means that the other individual either experienced the event later or was censored after the event occurred to the first individual. A pair is *non-comparable* in the following situations [21]:

- Both individuals are censored: Since neither of them experienced the event, it is not possible to determine who would have experienced it first.
- Both individuals have the same observed time: The order cannot be determined.
- One subject is censored too early: If one was censored before the other experienced the event, it is unclear who would have experienced the event first.

A pair is said to be *concordant* if the subject who lived longer is predicted to have a lower risk than the one who died earlier [21]. It is *discordant* if the model predicts a higher risk for the subject who actually lived longer. If both individuals have the same predicted risk score and the pair satisfies the comparability condition, it is still included in the calculation and referred to as *half-concordant*. In this case, during the calculation of concordant pairs in the C-index, such a pair contributes 0.5 instead of 1 [21].

Figure 2.24 illustrates the process of deciding whether a pair of subjects (s_i, s_j) is concordant, discordant, half-concordant, or non-comparable based on their predicted risk scores (η_i, η_j), time-to-event (T_i, T_j), and censoring indicators.

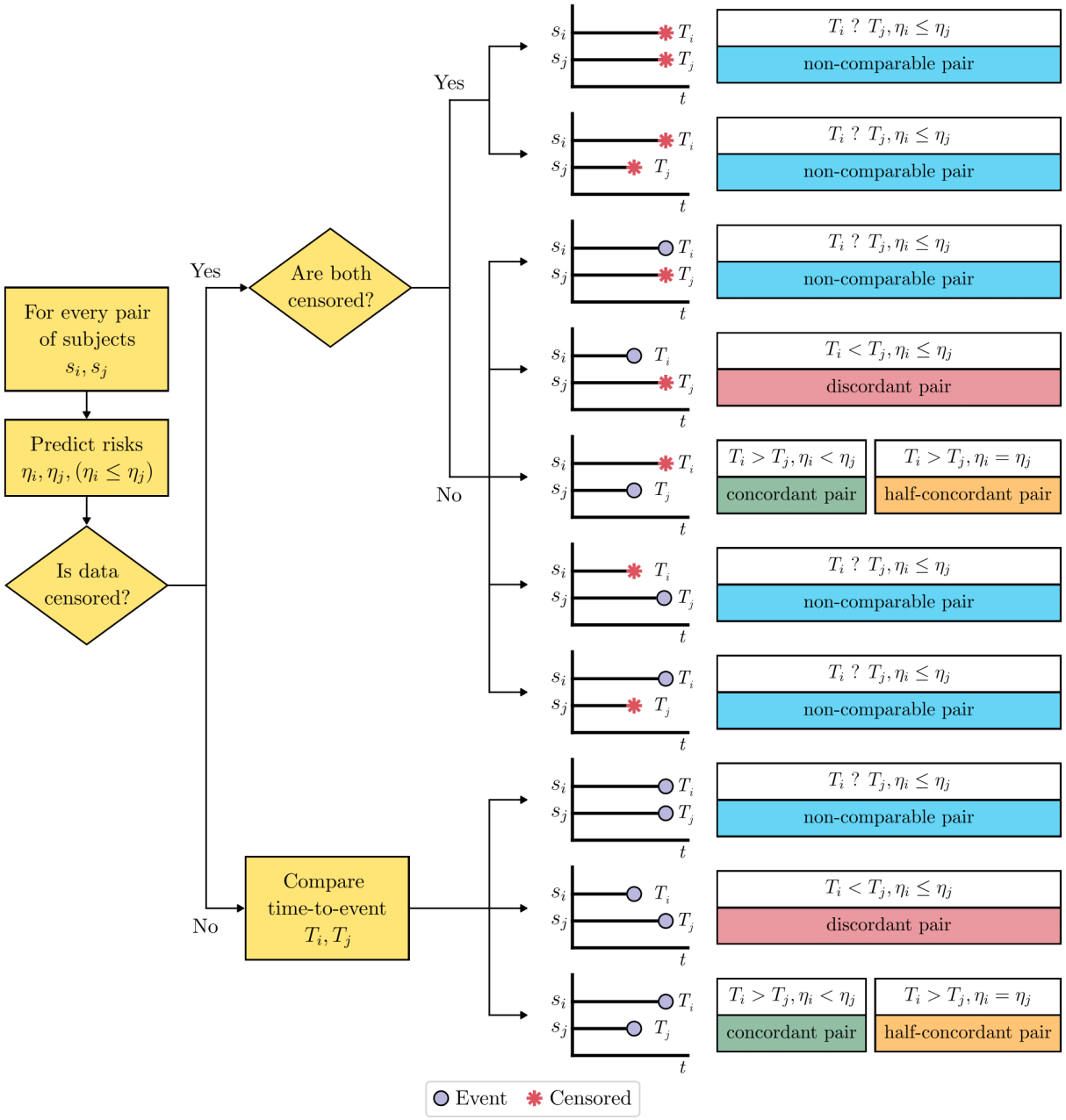


Figure 2.24: Classification of subject pairs for C-index computation. Circles denote observed events, asterisks indicate censored observations.

Credit: The flowchart is inspired by [1].

The C-Index is then calculated as

$$\text{C-index} = \frac{N_c + 0.5 \times N_t}{N_c + N_d + N_t}, \tag{2.35}$$

where N_c is the number of concordant pairs, N_d is the number of discordant pairs, and N_t is the number of tied (half-concordant) pairs.

Mathematically, this can be written as [56, 1] [1]

$$\text{C-index} = \frac{\sum_{i \neq j} \mathbb{I}(T_i > T_j) \cdot [\mathbb{I}(\eta_i < \eta_j) + 0.5 \cdot \mathbb{I}(\eta_i = \eta_j)] \cdot \Delta_j}{\sum_{i \neq j} \mathbb{I}(T_i > T_j) \cdot \Delta_j}. \tag{2.36}$$

In this expression, T_i and T_j denote the survival times for subjects s_i and s_j, respectively. The terms η_i and η_j represent their corresponding predicted risk scores. The variable Δ_j is the event indicator for subject j, which takes the value 1 if the event was observed and 0 if the observation was censored; it is used to exclude non-comparable pairs where the shorter survival time is censored. The function $\mathbb{I}(\cdot)$ is the indicator function, which returns 1 if the condition inside is true and 0 otherwise.

The interpretation of the C-index is straightforward. A value of 1 means that the model perfectly ranks individuals by risk, constantly predicting higher risk scores for those who experience the event earlier. A value of 0.5 indicates no predictive discrimination, meaning that the model is no better than a random guessing [21]. Values below 0.5 suggest that the model performs worse than random. In general, a C-index between 0.7 and 0.8 is considered acceptable, while a value between 0.8 and 0.9 is regarded as excellent [57].

[1] The C-index formulation given in [56, 1] does not account for tied predicted risk scores. I modified the formula by assigning a weight of 0.5 to pairs with equal predicted risk scores ($\eta_i = \eta_j$).

Chapter 3

Method

This chapter details the methods I used in this thesis. I begin by describing the dataset and how the segmentation masks used for model training were generated. Then, I describe the preprocessing techniques applied to prepare the images for training. I outline the modifications I made to the U-Net architecture, followed by the training process and the grid search I used to explore different model settings. After that, I explain how I selected the best-performing model and processed its outputs. Finally, I describe how I analyzed the spatial distribution of nerve structures and performed survival analysis to test their association with breast cancer-specific survival using Cox regression.

3.1 Data

The dataset used in this thesis was provided by co-supervisors at Haukeland University Hospital. It includes data collected from patients diagnosed with invasive breast carcinoma between 1996 and 2003 as part of the Norwegian Breast Cancer Screening Program (NBCSP). The dataset is composed of TIFF-format Imaging Mass Cytometry (IMC) images and TXT-format metadata.

The imaging dataset includes data from 488 tissue samples and consists of:

- IMC images: 38-channel TIFF files, each channel representing the expression of a specific biomarker.

- Nerve masks: Instance segmentation masks in which each nerve segment is labeled with a unique number; used as loose ground truth masks for training the nerve segmentation model.
- Compartment masks: Binary masks that separate tumor compartments from the stroma.
- Cell masks: Instance segmentation masks where each cell is labeled with a unique number corresponding to one of ten predefined cell types.

An explanation of how segmentation masks were generated is provided in Section 3.1.1.

In addition to imaging data, the dataset includes patient metadata, consisting of 14 columns and 451 entries, which describe key clinical and pathological characteristics of tumor samples. A detailed description of each variable is provided in Table 3.1.

Column Name	**Description**
`case`	Unique case identifier
`tma_case`	Tissue microarray case identifier
`image`	TIFF file name
`nerve_count`	Number of detected nerve segments
`n_stroma`	Number of tumor stroma-related nerves
`n_tumor`	Number of tumor cell-related nerves
`tumor_diameter`	Tumor diameter (in mm)
`histologic_grade`	Histological grade of the tumor
`age_of_diagnosis`	Age at diagnosis (in years)
`follow_up_months`	Duration of patient follow-up (in months)
`cause_of_death`	Survival status
`subtype2`	Breast cancer molecular subtype
`subtype3`	More specific classification within `subtype2`
`batch`	Batch identifier

Table 3.1: Description of patient metadata variables.

The dataset also includes cell metadata, which provides detailed morphological and categorical information about individual cells within the tissue samples. The cell metadata consists of 1,942,523 rows and 9 columns, corresponding to 486 tissue samples. Each row represents a single cell with attributes that describe its size, shape and classification. A summary of the cell metadata variables is provided in Table 3.2.

Column Name	Description
tma_case	Tissue microarray case identifier
obj_num	Unique object number assigned to the cell
area	Cell area in pixels
axis_major_length	Major axis length of the cell (longest diameter)
axis_minor_length	Minor axis length of the cell (shortest diameter)
eccentricity	Cell shape eccentricity
width_px	Image width in pixels
height_px	Image height in pixels
cell_type	Identified cell type

Table 3.2: Description of cell metadata variables.

The dataset further includes a marker panel file containing 4 columns that describe each IMC channel. The order of the entries in this file matches the order of the channels in the IMC images. Table 3.3 provides an overview of these variables.

Column Name	Description
metal	Rare-earth-metal isotope used to label the antibody
antibody	Antibody that binds to a target protein
cell	Cell type
category	Classification of the marker

Table 3.3: Description of marker panel variables.

3.1.1 Origin of Segmentation Masks

Nerve Masks

Of the eight nerve-related IMC channels present in the dataset, only a protein called peripherin was used to create nerve masks. Peripherin is a protein that is predominantly expressed in neurons of the peripheral nervous system (hence its name) [38]. According to the co-supervisors, it was the only channel in the dataset that produced the expected staining pattern and was therefore chosen for nerve detection. Regarding the other nerve markers, there were doubts as to whether the antibodies worked as intended, so they were not considered.

The segmentation workflow involved two open-source software tools: ilastik for pixel classification and CellProfiler for object segmentation.

1. Pixel classification with ilastik A random forest pixel classifier was trained on a subset of samples where positive nerve segments were easy to identify. In these selected cases, pixels were manually annotated as either positive (nerve) or negative (not nerve). The trained model was then applied to all samples to produce probability maps that indicated the likelihood of a given pixel corresponding to a nerve fiber.

2. Segmentation with CellProfiler The probability maps were imported into CellProfiler, where the intensity thresholds and object size parameters were manually adjusted. This resulted in labeled instance masks in which each nerve segment was assigned a unique object number.

Compartment Masks

The compartment masks were generated using the same method as the nerve masks. Pixel classification in ilastik was based on pan-cytokeratin expression. Areas with a positive signal were marked as tumor regions, while areas without signal were considered stroma. The resulting probability maps were imported into CellProfiler for adjusting the intensity thresholds. This yielded binary masks isolating tumor and stromal areas.

Cell Masks

Cell masks were produced using the DeepCell Mesmer deep learning model, designed for cell segmentation of multiplexed tissue imaging data. The model takes as input two channels: a nuclear channel for locating the nucleus, and a membrane or cytoplasmic channel for defining the cell boundary [19]. The output is an instance segmentation mask in which each cell is assigned a unique integer label shared by all its pixels.

3.2 Data Preprocessing

3.2.1 Image Resizing and Data Splitting

To meet U-Net's input size requirements, I cropped all images in the dataset from 850×850 pixels to 848×848 pixels. Since U-Net includes four max-pooling layers that reduce

the spatial dimensions by a factor of two at each layer, the size of the input image had to be divisible by 16. Between cropping and padding, I chose cropping to simplify the workflow.

After resizing, I divided the data into three sets: 80% for training, 10% for validation and 10% for testing. To reduce potential bias in model training and validation, I performed a stratified split based on the number of nerve segments. This ensured that each subset contained a similar proportion of samples with no, few and many nerve segments.

3.2.2 Preprocessing of IMC Images

To address common issues in IMC data, such as noise and artifacts (unusually high intensities), I applied several preprocessing methods to the peripherin channel before training the segmentation models. Specifically, I tested six different approaches, described below.

1. **Min-Max Normalization**: Used to check whether simply bringing pixel values into a consistent [0,1] range would be sufficient for the model to learn meaningful features.
2. **Log Transformation**: Used to make subtle details more apparent.
3. **Percentile Normalization**: Intended to reduce the influence of extreme pixel values by scaling the image intensities using the 99th percentile.
4. **Intensity Clipping**: Caps high-intensity pixels to a computed threshold. The details of this method are provided below this list.
5. **Intensity Clipping + Min-Max Normalization**: Combines clipping with min-max scaling to both suppress extreme values and rescale the image to consistent [0, 1] range.
6. **Gaussian Blur + Histogram Equalization + Min-Max Normalization**: A combination of Gaussian blur with a kernel size of 3 for slight noise reduction, histogram equalization to enhance contrast and reveal fine details, and min-max normalization to bring pixel values into a uniform range.

The purpose of intensity clipping is to reduce the influence of excessively bright pixels, which may result from detector abnormalities, dust contamination, or other technical issues. Simply discarding such pixels can be risky, as some of them may actually correspond to true biological signal. An alternative would be to treat such values as outliers and clip them using a percentile threshold, typically set at the 99th percentile. However, this approach is also not ideal, because in images with very few high-intensity pixels the 99th percentile may simply be zero. This would effectively remove all bright pixels, making the method equivalent to complete removal, which is not desirable.

Taking these issues into account, I opted for a somewhat better approach. I still clipped to the 99th percentile, but used whichever was higher: the 99th percentile of the image itself or the 99th percentile computed across an entire training dataset. Furthermore, I excluded zero-valued pixels from the calculation, so that the estimated thresholds are not lowered by background regions. This way, even if the 99th percentile of the image's pixel values is very low, high-intensity signal is not lost but instead clipped to a more reasonable value derived from the training dataset.

The mathematical formulation of intensity clipping is given below.

First, a global threshold T_{global} is computed as the 99-th percentile of non-zero intensities across the entire training dataset. This is defined as

$$T_{\text{global}} = P_{99}\left(\left\{ x \,\middle|\, x > 0,\ x \in \bigcup_{i=1}^{N} X^{(i)} \right\}\right),$$

where $X^{(i)}$ denotes the i-th image in the training dataset, N is the size of the training dataset, and x are pixel intensity values.

Next, for each image X, a local threshold is calculated as the 99-th percentile P_{99} of its non-zero pixel values,

$$T_{\text{local}} = P_{99}\left(\{x \,|\, x > 0,\ x \in X\}\right).$$

The final clipping threshold for that image is then defined as the maximum of the global and local thresholds, namely

$$T_{\text{clip}} = \max\left(T_{\text{global}},\ T_{\text{local}}\right).$$

Finally, all pixel intensities that exceed T_{clip} are replaced by this value, resulting in

$$X_{\text{clipped}} = \min\left(X,\ T_{\text{clip}}\right).$$

3.3 Image Segmentation

3.3.1 Model Architecture

In my implementation, two changes were made to the original U-Net architecture.

The first modification involves the use of same padding instead of valid padding. The original U-Net uses valid padding, which causes the output image to be smaller than the input. To achieve a complete segmentation image, the model would need to be run multiple times on overlapping image tiles. By switching to same padding, the output image retains the same dimensions as the input, removing the need for tiles.

The second modification is the inclusion of batch normalization after each convolutional layer. Although the original U-Net architecture does not use batch normalization, its addition has been shown to improve segmentation performance [20]. Batch normalization accelerates the convergence of deep networks by keeping consistent data values across layers [68]. Additionally, it provides inherent regularization, which helps reduce overfitting [68].

The utilized U-Net architecture and its convolution block are illustrated in Figures 3.1 and 3.2.

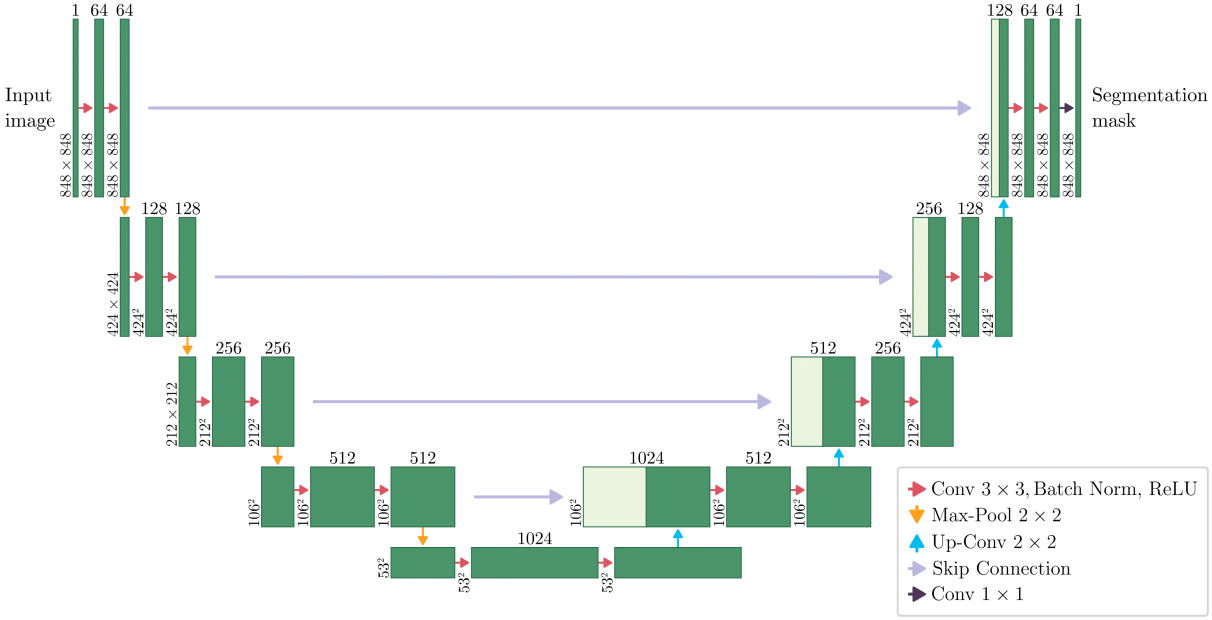


Figure 3.1: U-Net architecture used in this work.

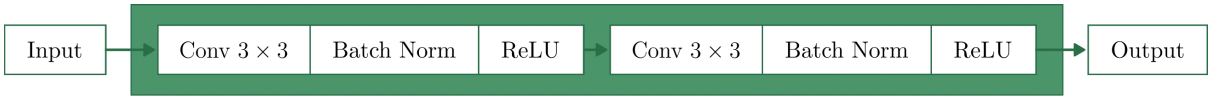


Figure 3.2: A convolutional block consisting of two 3×3 convolution layers, followed by batch normalization and a ReLU activation. The arrows indicate the flow of features through the network.

3.3.2 Model Training

Each model was trained using the peripherin channel as input and the nerve masks generated by the ilastik/CellProfiler pipeline as loose ground truth. I used the training set for optimization and monitored model performance on the validation set.

For all experiments, I used the Adam optimizer, a batch size of 16, and a maximum of 500 training epochs. Given the small size of the dataset, I applied data augmentation to artificially expand the training data and improve the model's ability to generalize to unseen data. The augmentation techniques included random horizontal and vertical flipping, as well as 90-degree rotations.

To prevent overfitting, I used early stopping with a patience of 20 epochs, which stopped training if the performance of the model on the validation data no longer improved. In addition, I used a learning rate scheduler that reduced the learning rate by half when the validation loss plateaued for 10 consecutive epochs.

I conducted a grid search over multiple training configurations. Specifically, I varied three components: the loss function, the learning rate, and the preprocessing method applied to the input images. The loss functions tested were Weighted Cross-Entropy Loss, Focal Loss, and Dice Loss. For Focal Loss, I set the class-balancing factor $\alpha = 0.99$ to strongly emphasize the minority nerve class and used the focusing parameter $\gamma = 2$, as this value yielded the best results in the original paper [33]. For the learning rates, I chose values of 1×10^{-3}, 1×10^{-4}, and 1×10^{-5}. The preprocessing methods included min-max normalization, log transformation, percentile normalization, intensity clipping, Gaussian blur, histogram equalization, and combinations thereof (described in Section 3.2.2). In addition to these, I trained models using the raw peripherin channel without any preprocessing. These models served as a baseline for further evaluation of whether preprocessing actually affects model performance.

3.3.3 Model Selection

The goal of model selection was to identify the best-performing segmentation model from a set of trained candidates. To ensure a fair and consistent comparison, all models were evaluated on the validation set following a standardized procedure. This involved generating predictions using test-time augmentation, applying multiple threshold values to probability maps to identify the optimal one, and computing evaluation metrics. The procedure is detailed below.

1. Prediction with test-time augmentation To improve prediction robustness and accuracy, I used test-time augmentation, which combines predictions from several augmented versions of the same image. Specifically, for each image, its 90°, 180° and 270° rotated versions were generated. Each rotated image, along with the original version, was passed through the model to produce a probability map using a sigmoid activation function. The resulting prediction masks were then rotated back to their original orientation and averaged pixel-wise to produce a final probability map.

2. Thresholding of probability maps Instead of applying a fixed threshold to all models with different configurations, I evaluated several threshold values for converting probability maps into binary segmentation masks. Specifically, I tested values of 0.5, 0.6, 0.7, 0.8, and 0.9. Pixels with values above the threshold were classified as nerve regions, while the rest were labeled as background.

3. Computation of evaluation metrics For each threshold, I computed the Dice Similarity Coefficient (DSC) and True Positive Rate (TPR) using only samples in the validation set that contained nerve segments. Samples without any nerve structures were excluded from the evaluation, as DSC and TPR metrics are only meaningful if there is a positive ground truth; if a sample does not contain nerve pixels, these metrics become undefined or misleading. For example, in a sample without nerve structures, a perfectly correct prediction (i.e., all zeros) results in undefined DSC and TPR values. Although the prediction is correct, the metrics do not reflect this. Similarly, if the model falsely predicts nerve pixels in such a sample, DSC becomes 0, and TPR remains undefined. In both cases, even if the model makes reasonable or correct predictions, the inclusion of such samples distorts the metrics. Therefore, it made sense to exclude them from the metric computation.

For each model, I selected the threshold that achieved the highest DSC among the five tested values as the optimal one. When choosing the best model overall, I considered both its ability to detect nerve structures present in the imperfect "ground truth" masks and its potential to identify additional nerves that might have been missed. While TPR reflects a model's ability to identify positive (nerve) pixels, it does not account for false positives and can therefore be misleading when used alone. For example, a model that predicts all pixels as positive would achieve a TPR of 1, but this does not imply high accuracy unless the vast majority of pixels are in fact positive (which is certainly not the case in nerve segmentation). DSC, by contrast, accounts for both true positives and errors (false positives and negatives), making it a more reliable metric for model selection. Therefore, I first prioritized achieving a high DSC and then, among models with roughly similar DSC, selected the one with the highest TPR.

3.3.4 Mask Processing

After generating binary nerve masks for the entire dataset using the model that achieved the highest scores on the validation set, I performed the following steps to isolate the nerve regions for spatial analysis and subsequent survival analysis:

1. Connected components labeling To separate nerve segments, I applied connected components labeling using 8-connectivity. In this method, each pixel is grouped with all directly adjacent neighbors, including diagonal ones. This results in a mask where each connected region is assigned a unique label. Figure 3.3 provides a visual example of this method.

2. Removal of small objects To reduce noise and eliminate possible false positives, I removed the smallest connected components. Due to the tail-like structure of nerves and the fact that they can pass through the tissue at angles not aligned with the cutting plane, some of these very small areas may, in fact, represent true nerve regions that are only partially captured in a tissue section. However, they make up only a tiny portion of the total nerve area, so their exclusion has negligible impact on the overall nerve density.

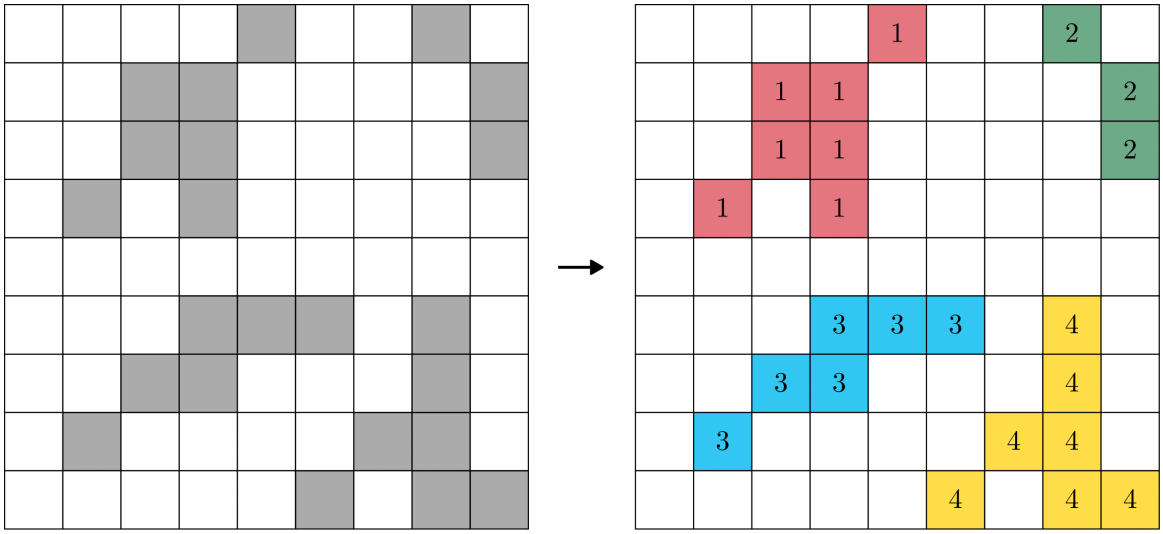


Figure 3.3: Example of connected components labeling using 8-connectivity. On the left: a binary mask with dark pixels representing foreground regions to be labeled. On the right: the same mask after labeling.

3.4 Spatial Distribution of Nerve Structures

Tumor-Stroma Overlap

To explore where nerves are predominantly located within the tumor microenvironment, I first quantified how much of each nerve segment lies in tumor versus stroma. Specifically, for each nerve segment i, I calculated the proportion of its area overlapping with each compartment. These proportions are defined as

$$t_i = \frac{\text{Number of tumor-overlapping pixels in segment } i}{\text{Total number of pixels in segment } i},$$

$$s_i = \frac{\text{Number of stroma-overlapping pixels in segment } i}{\text{Total number of pixels in segment } i},$$

where t_i and s_i denote the tumor and stroma fractions of segment i, respectively.

Figure 3.4 provides a visual example for better understanding how fractions are calculated. The nerve on the left (with label 1) consists of 7 pixels, all of which overlap with the tumor compartment, resulting in $t_1 = 1$, $s_1 = 0$. The nerve on the right (with label 2) contains 15 pixels, of which 8 are located in tumor and 7 in stroma, yielding

$$t_2 = \frac{8}{15} \approx 0.53, \quad s_2 = \frac{7}{15} \approx 0.47.$$

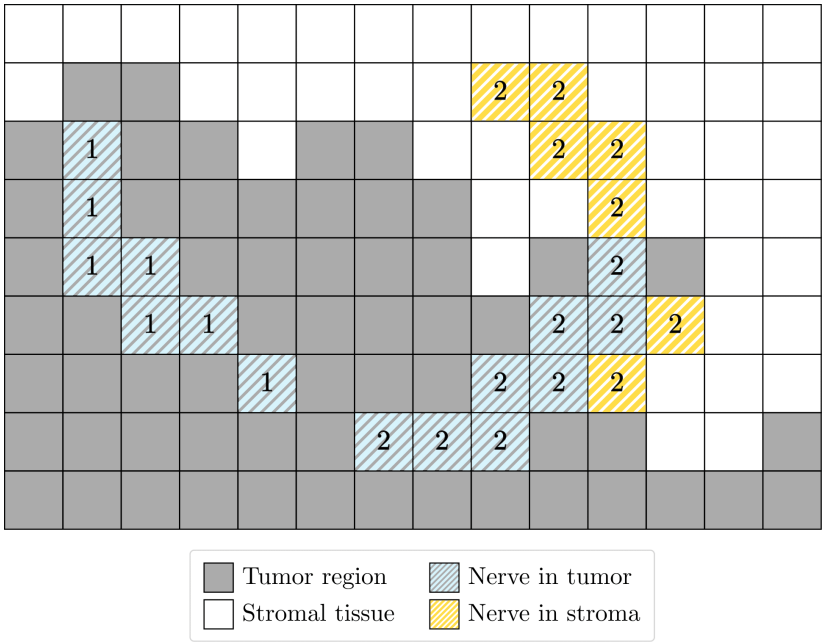


Figure 3.4: Gray pixels represent tumor area, white pixels represent stroma, and colored pixels denote nerve segments. Blue-striped pixels indicate nerve segments within the tumor region, and yellow-striped pixels indicate segments within the stroma.

After computing these fractions for all nerve segments in a sample, I added them together to obtain tumor- and stroma-associated nerve counts. This can be written as

$$T = \sum_{i=1}^{n} t_i, \quad S = \sum_{i=1}^{n} s_i,$$

where n is the number of nerve segments in the tissue sample, T denotes the tumor-associated nerve count and S the stroma-associated.

In the given example, it results in

$$T = t_1 + t_2 = 1 + 0.53 = 1.53, \quad S = s_1 + s_2 = 0 + 0.47 = 0.47.$$

Cellular Composition

To analyze the cellular composition within each nerve segment across the tissue samples, I used the cell masks. For each nerve segment i, I counted the number of pixels associated with each cell type c, where c corresponds to one of the ten types: `B_cells`, `Cancer_cell`, `Endothelial`, `Immune_other`, `Macrophages_CD163+`, `Macrophages_CD163-`, `Stromal`, `T_cytotoxic`, `T_helper`, and `T_regulatory`. Pixels within the segment that did not

overlap with any segmented cell were labeled as "unclassified" and excluded from further analysis.

Next, I normalized these counts by the total number of pixels in each nerve segment to determine the relative area occupied by each cell type within that segment. Formally, for each nerve segment i and cell type c, this is defined as

$$p_{i,c} = \frac{\text{Number of pixels of cell type } c \text{ in segment } i}{\text{Total number of pixels in segment } i}.$$

Here, $p_{i,c}$ represents the proportion of the area of segment i that is covered by cell type c.

Following the same approach as in the compartment analysis, I aggregated the results at the tissue sample level by summing the proportions of each cell type across all nerve segments in the sample. Formally, this can be expressed as

$$P_c = \sum_{i=1}^{n} p_{i,c},$$

where P_c represents the abundance of cell type c in the nerve regions of the tissue sample, and n is the total number of nerve segments.

3.5 Survival Analysis

After quantifying nerve segments and their spatial distribution, I performed survival analysis using Cox regression. The analysis had two main objectives: first, to determine whether the presence and distribution of nerves are associated with breast cancer-specific survival, and second, to compare the prognostic value of nerve counts obtained from my U-Net-based segmentation with those derived from nerve masks generated by the ilastik/CellProfiler pipeline.

The survival data in this thesis includes three outcome categories. The survival status variable `cause_of_death` in the patient metadata (Table 3.1) is labeled 0 if the patient left the study alive, 1 if the patient died from other causes, and 2 if the patient died from breast cancer. Patients who left the study alive represent right-censored data. Both death from other causes and death from breast cancer are failure events; however, since the aim of this work is to assess the relationship between nerves and breast cancer survival, death

from other causes represents a competing event, while death from breast cancer is the event of interest.

In survival analysis with competing events, a common approach when using the Cox model is to estimate hazards separately for each failure type, treating other event types as censored in addition to the usual censored observations [29]. When only one failure type is of primary interest, the analysis is typically restricted to estimating hazards for that type alone, with competing failure types treated as censored [29]. Therefore, in this work, I focused the analysis exclusively on mortality related to breast cancer. I considered both subjects who left the study alive and those who died from other causes as censored observations.

3.5.1 Model Training

Cox regression was implemented in R programming language using `survival` package. I opted for R because, unlike Python libraries commonly used for survival analysis, such as `scikit-survival` and `lifelines`, which only support the Breslow and Efron methods, R also supports the exact method for handling tied events.

For survival model fitting, I merged the training and validation sets used during segmentation model training into a single dataset. The validation set had already been used during the segmentation model selection process and therefore no longer served as an unbiased dataset. The test set, which remained untouched during both segmentation model training and selection, was set aside exclusively for survival model evaluation.

In total, I fitted 11 Cox regression models with different covariates to explore factors that may be linked to breast cancer survival. The predictor variables used in each model are summarized in Table 3.4.

Model No.	Predictor
1	nerve_count
2	n_stroma, n_tumor
3	nerve_count, age_of_diagnosis, tumor_diameter, histologic_grade
4	n_stroma, n_tumor, age_of_diagnosis, tumor_diameter, histologic_grade
5	pred_nerve_count
6	pred_n_stroma, pred_n_tumor
7	pred_nerve_count, age_of_diagnosis, tumor_diameter, histologic_grade
8	pred_n_stroma, pred_n_tumor, age_of_diagnosis, tumor_diameter, histologic_grade
9	age_of_diagnosis, tumor_diameter, histologic_grade
10	B_cells, Cancer_cell, Endothelial, Immune_other, Macrophages_CD163+, Macrophages_CD163-, Stromal, T_cytotoxic, T_helper, T_regulatory (using provided nerve masks)
11	B_cells, Cancer_cell, Endothelial, Immune_other, Macrophages_CD163+, Macrophages_CD163-, Stromal, T_cytotoxic, T_helper, T_regulatory (using U-Net predicted nerve masks)

Table 3.4: List of predictor variables included in each Cox model.

Models 1-4 use nerve counts derived from the provided nerve masks. Specifically, Model 1 includes the total number of nerves, while Model 2 uses counts of nerves associated with the tumor and stroma. Models 3 and 4 extend Model 1 and 2 by adding clinical predictors (age at diagnosis, tumor diameter and histologic grade). Models 5-8 use the same predictor combinations as Models 1-4 but with nerve counts derived from U-Net segmentation masks. Model 9 includes only clinical predictors to assess their standalone prognostic value. Models 10 and 11 use the proportions of different cell types within nerve segments, with Model 10 based on values computed from the provided nerve masks and Model 11 based on those derived from the U-Net predicted masks.

3.5.2 Model Evaluation

To evaluate model performance, I first applied each fitted Cox regression model to the test set to generate the risk scores. Following this, I calculated the concordance index to determine how well the predicted risks aligned with the observed survival outcomes.

Chapter 4

Results

In this chapter, I present the results obtained using the methods described in Chapter 3. I begin with an overview of the dataset, followed by data cleaning and exploratory analysis. Then, I examine how different preprocessing techniques affect the appearance of the peripherin channel. Next, I present the segmentation results and compare the performance of models trained with different configurations. Finally, I conclude with an overview of the Cox regression findings.

4.1 Data

The following figures provide examples of the imaging and segmentation data used in this thesis. Figure 4.1 shows an example of a 38-channel Imaging Mass Cytometry (IMC) image. Figure 4.2 shows a compartment mask that separates tumor and stroma regions, and Figure 4.3 presents an example of a cell segmentation mask, in which each cell is associated with one of ten cell types.

Figure 4.4 shows the input-output pairs used to train nerve segmentation models. The first column displays raw peripherin channel images, which served as input data for the model. The second column shows the corresponding nerve masks used as loose ground truth labels. Since the peripherin signal is barely visible at full resolution without preprocessing, for this figure I selected three samples with the highest number of nerve segments in the dataset and zoomed in to make the nerve structures more noticeable.

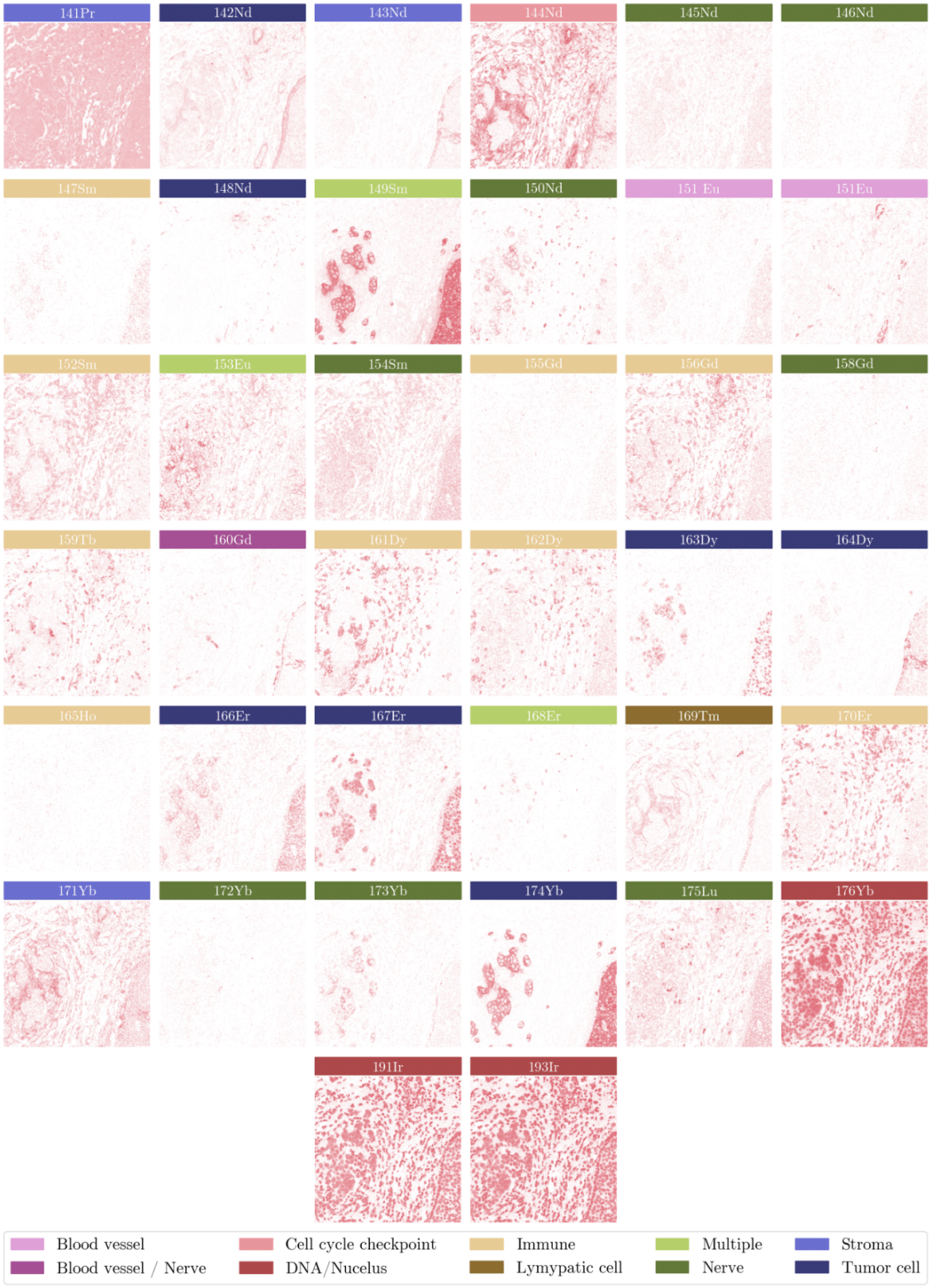


Figure 4.1: Example of 38-channel IMC image. Labels indicate the metal isotope used to tag a specific antibody, and panel colors correspond to the category of the marker. Here, the peripherin channel is located in the penultimate row, second column, with label 172Yb.

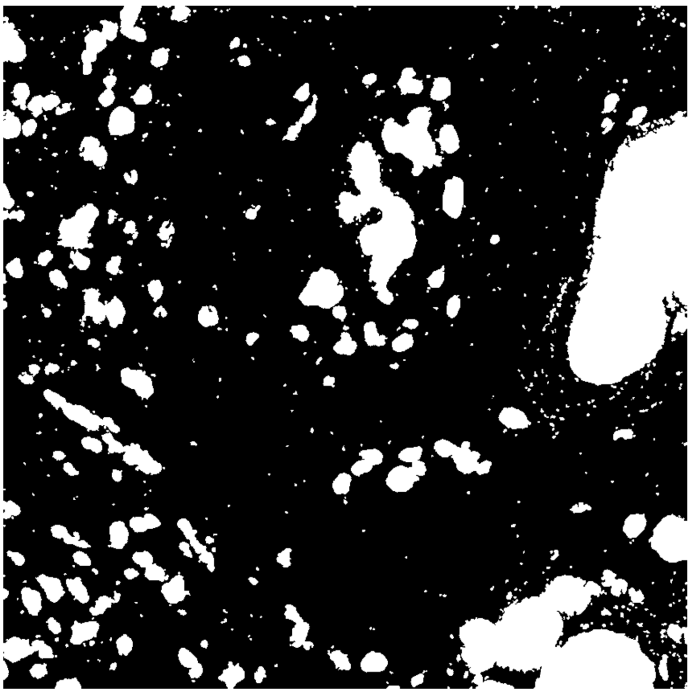


Figure 4.2: Example of compartment mask.

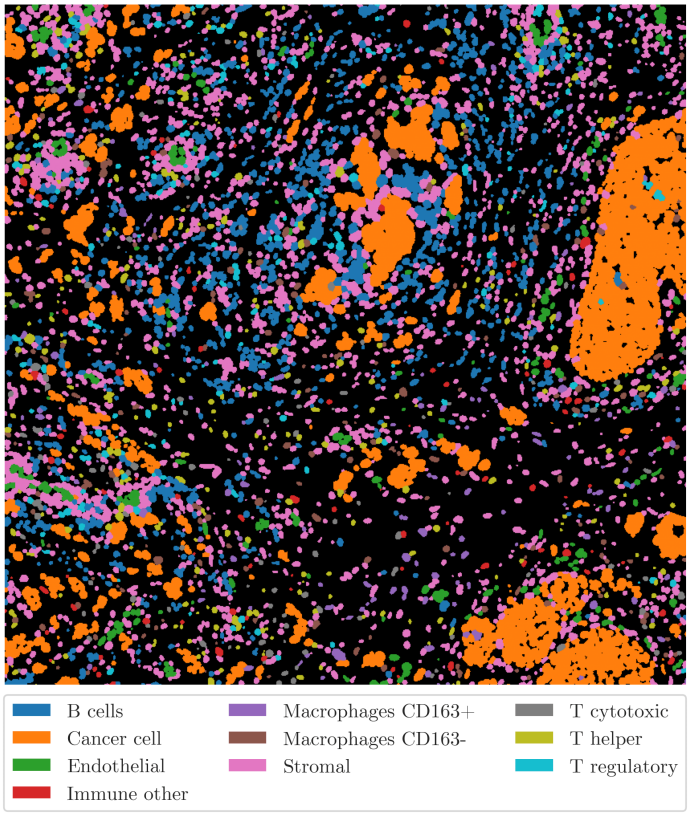


Figure 4.3: Example of cell segmentation mask.

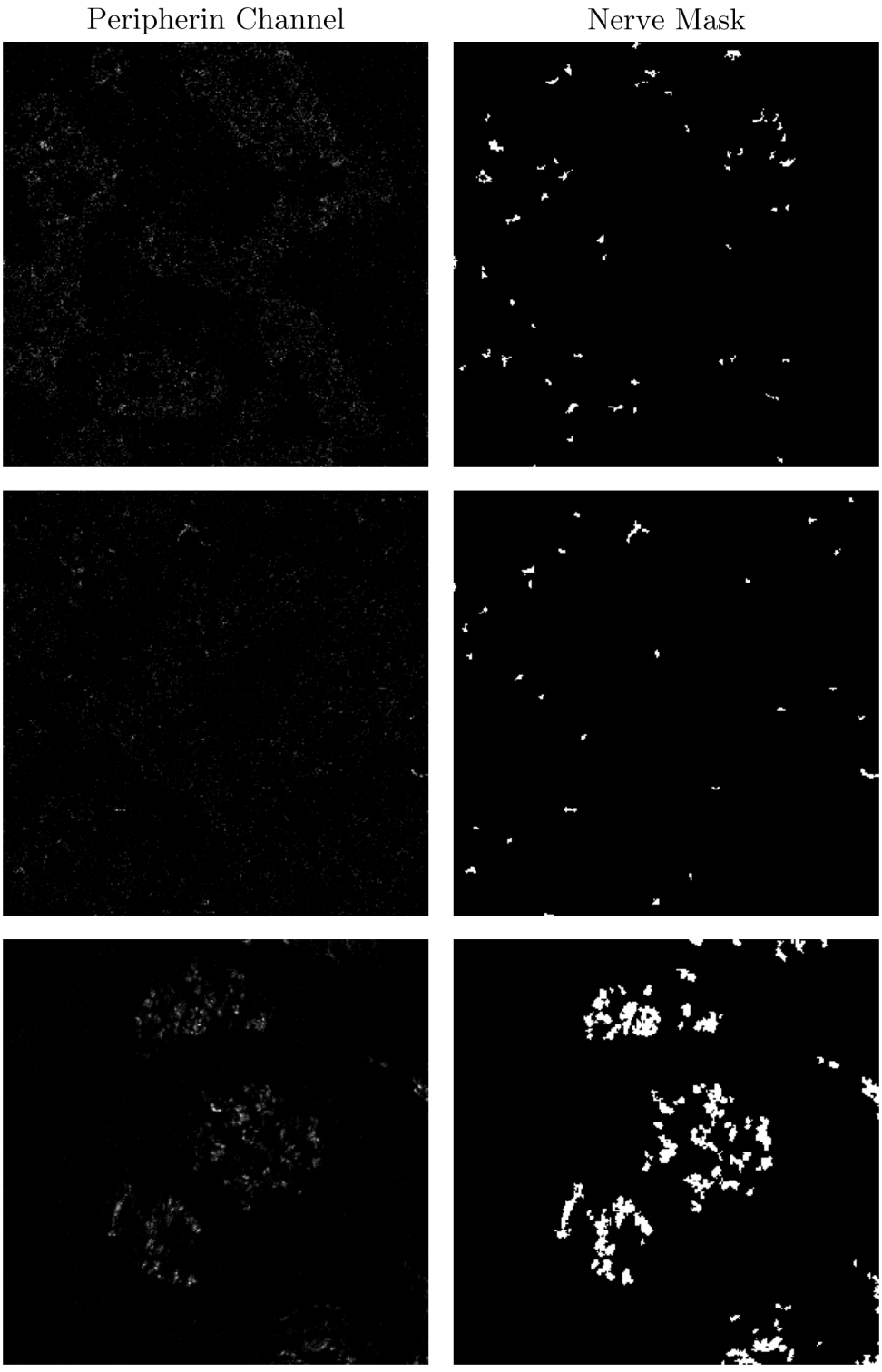


Figure 4.4: Input-output pairs used for training segmentation models. The first column shows raw peripherin channel images (input), and the second column shows the corresponding nerve masks (loose ground truth).

4.1.1 Data Cleaning

The first step was to check the completeness and consistency of all the data. The imaging dataset contained 488 tissue samples, with some patients having more than one sample, while the patient metadata included 451 records, each associated with one tissue sample.

First, I removed duplicate entries from patient metadata and excluded image samples with artifacts caused by hardware issues during the IMC run. Next, I looked at the image sizes. Most of the images were 850 × 850 pixels, and only 13 had different dimensions. To make the dataset consistent and easier to work with, I kept only the images with a size of 850 × 850 pixels.

Finally, I removed all image files that could not be matched to a record in patient metadata. After all these cleaning steps, the dataset was reduced to 427 patients.

4.1.2 Exploratory Data Analysis

Figure 4.5 shows the distribution of nerve segment counts in the loose ground truth nerve masks across the entire dataset. 28.8% of the samples do not contain nerves, 41.9% of the samples have 1 to 3 nerve segments, while the rest contain between 4 and 244.

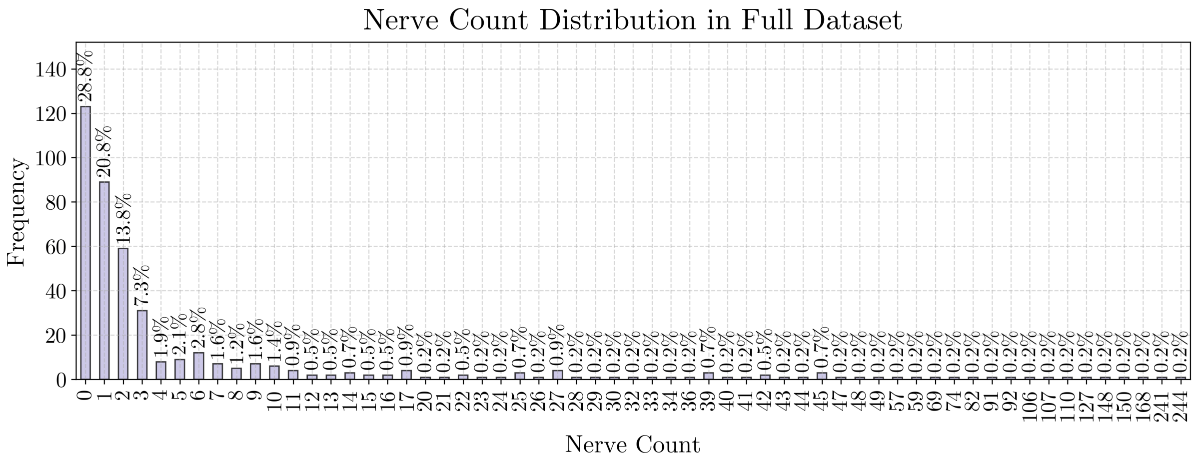


Figure 4.5: Distribution of nerve segment counts in the entire dataset. The x-axis indicates how many nerves were detected in each sample, and the y-axis shows how many samples have that number of segments. The percentages above the bars represent the proportion of samples with the corresponding nerve segment count.

Most nerve segments are small, with pixel counts concentrated below 10 (see Figure 4.6).

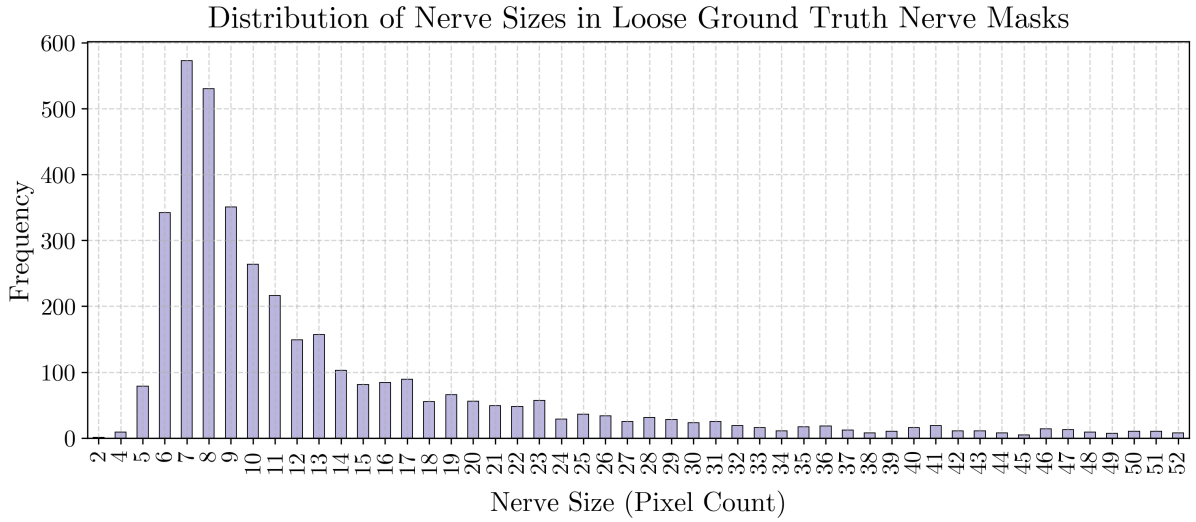


Figure 4.6: Distribution of nerve segment sizes in the loose ground truth nerve masks (first 50 nerve sizes by pixel count).

After splitting the data, the training set contains 341 samples, the validation set 43 samples, and the test set 43 samples. Figures 4.7, 4.8, and 4.9 show the distribution of samples in each subset based on the number of nerve structures in the nerve masks.

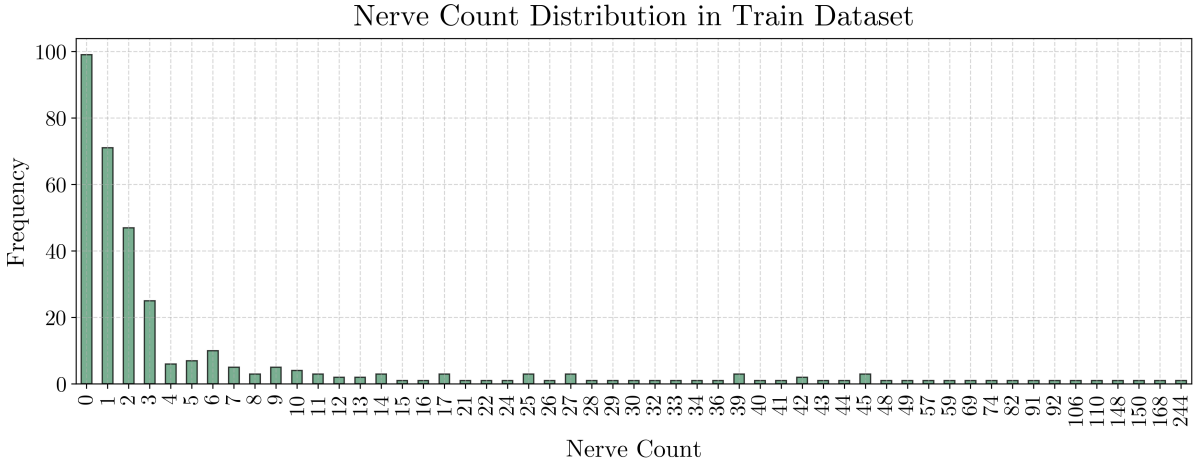


Figure 4.7: Distribution of nerve segment counts in the training dataset.

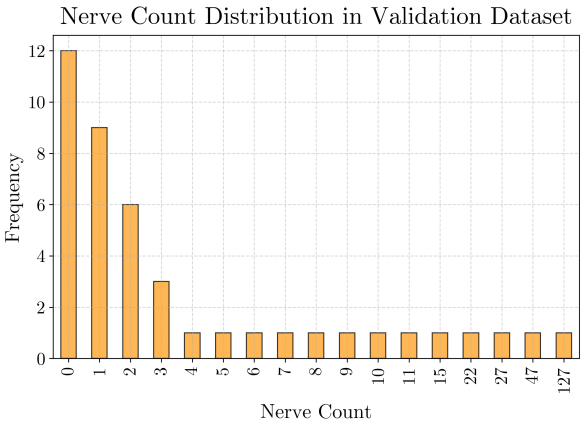


Figure 4.8: Distribution of nerve segment counts in the validation dataset.

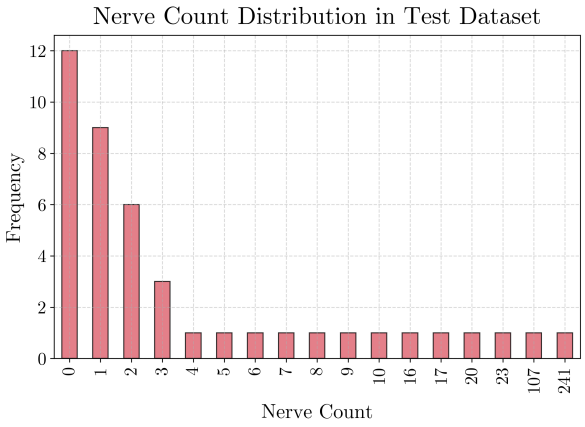


Figure 4.9: Distribution of nerve segment counts in the test dataset.

The peripherin channel, used for the detection of nerve fibers in tissue samples, is represented by integer-type images with pixel intensity values ranging from 0 to 13,934. The pixel intensity distribution is highly right-skewed, with the vast majority of pixels exhibiting very low intensities and a small proportion of pixels with higher intensity. The 99th percentile across all pixels is equal to 1, and increases to 4 when zero values are excluded. The intensity distribution with the 50 most frequent values is shown in Figure 4.10.

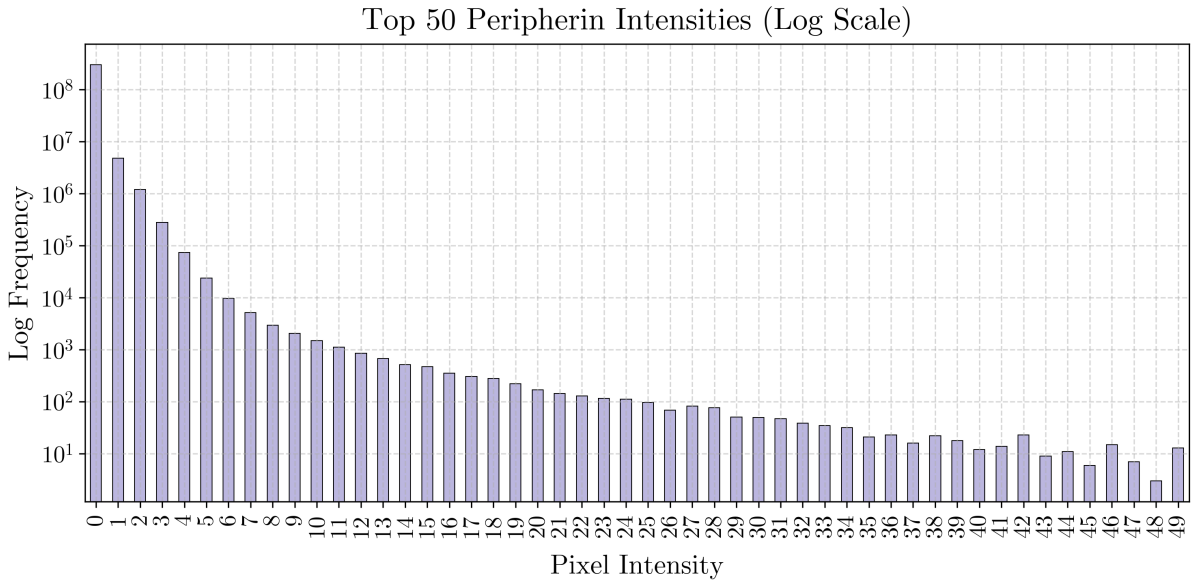


Figure 4.10: Distribution of the 50 most frequent pixel intensity values in the peripherin channel in the full dataset. The x-axis shows pixel intensity, and the y-axis represents frequency on a logarithmic scale.

4.1.3 Data Preprocessing

Figures 4.11 and 4.12 illustrate how the peripherin channel from the first and second rows of Figure 4.4 appears after applying different preprocessing techniques. Visually, min-max normalization and percentile normalization have no effect on the appearance of the image. In contrast, log transformation and intensity clipping, both with and without subsequent min-max normalization, make nerve-like structures more visible but also amplify image noise. The combination of Gaussian blur, histogram equalization, and min-max normalization reduces noise more significantly and, compared to the other methods, most clearly reveals structures that resemble nerves.

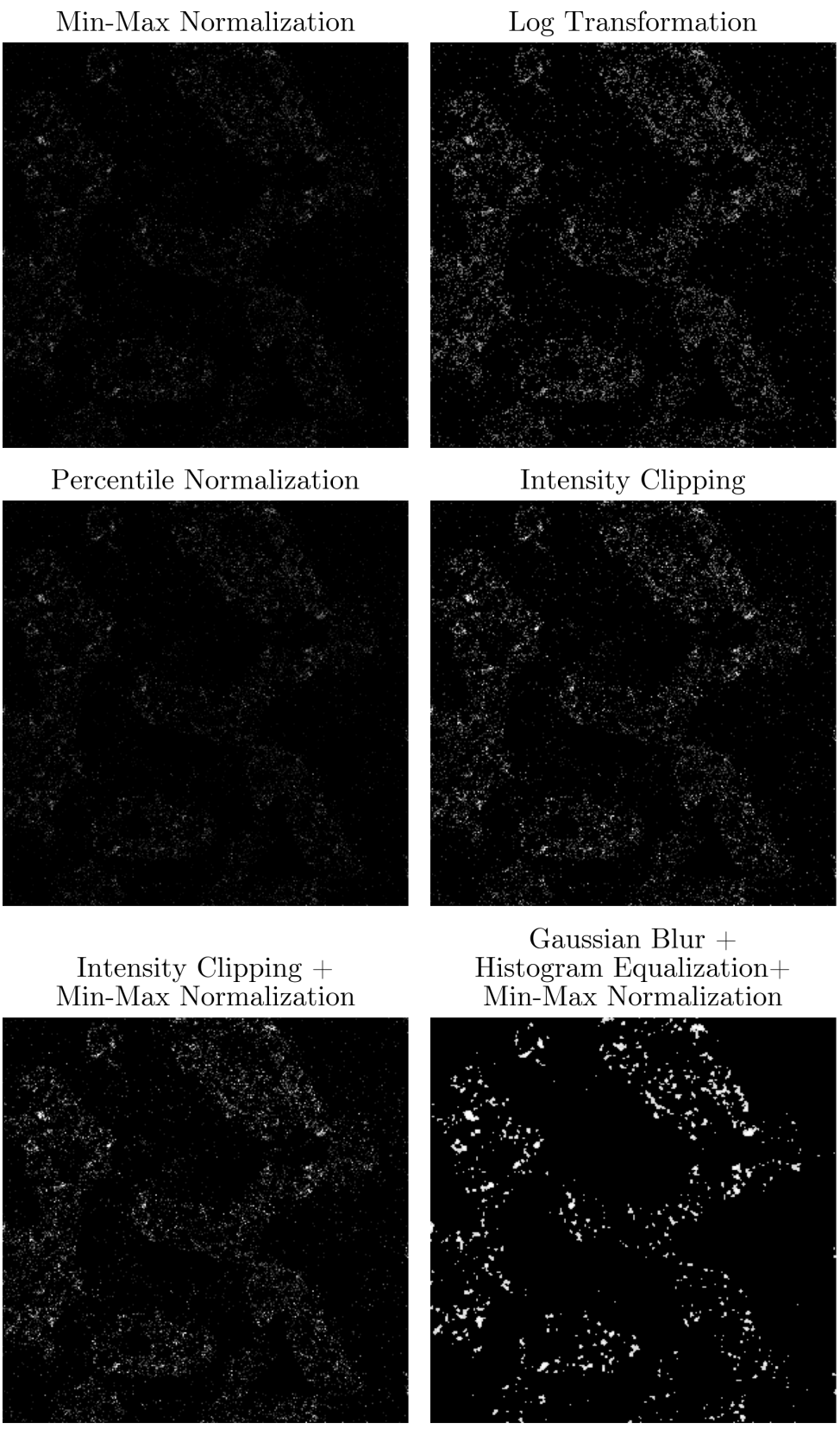


Figure 4.11: Example 1: Comparison of different preprocessing methods applied to the peripherin channel. Each subfigure shows the same image after applying normalization or enhancement techniques.

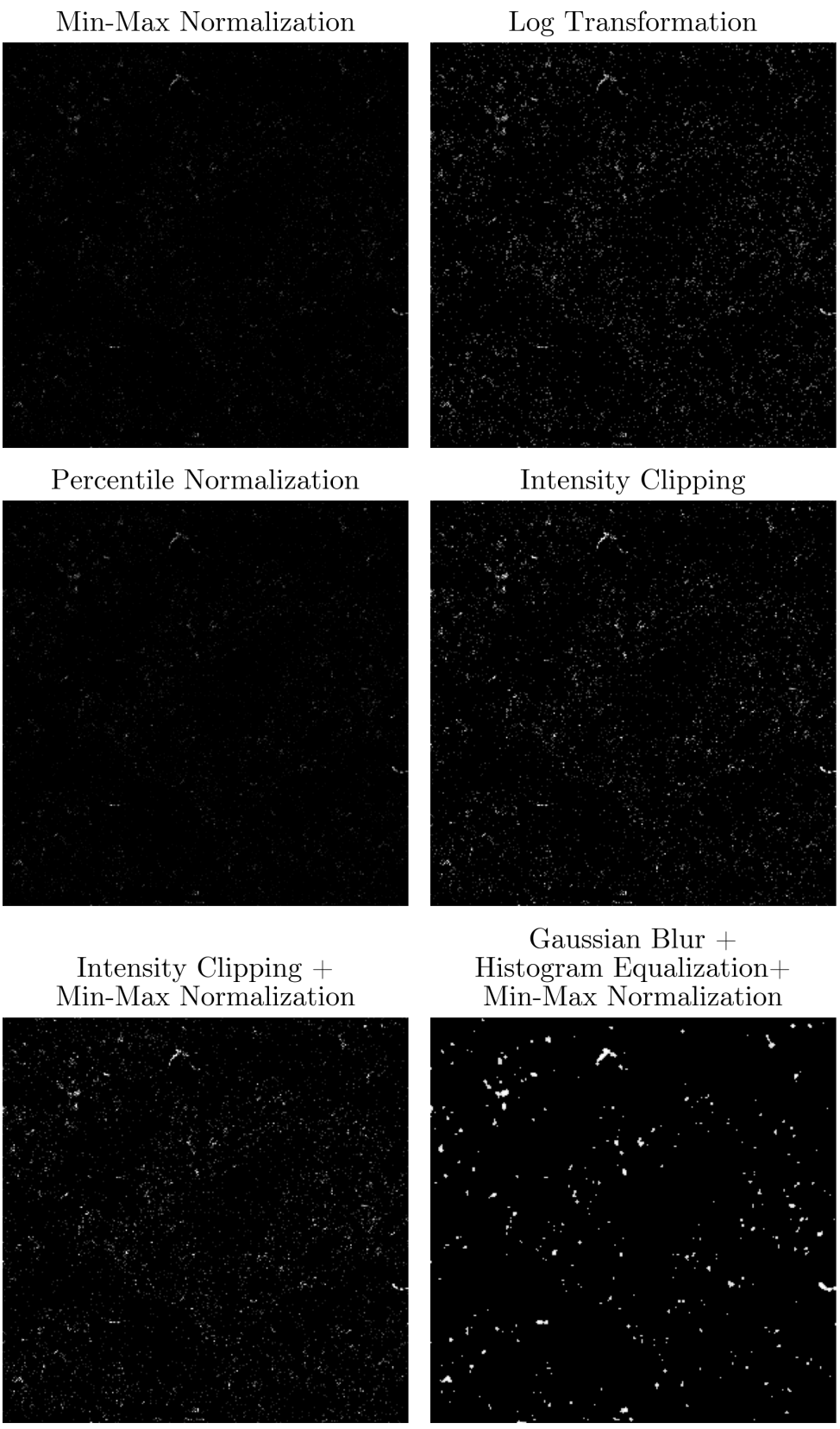


Figure 4.12: Example 2: Comparison of different preprocessing methods applied to the peripherin channel. Each subfigure shows the same image after applying normalization or enhancement techniques.

4.2 Image Segmentation

Training and validation loss curves for all 63 trained models are provided in Appendix A. Across all tested configurations, the training loss and the validation loss decreased sharply during the early epochs before reaching a plateau. Models trained with Focal Loss showed the smoothest and most stable loss curves, with a minimal difference between training and validation. Weighted Cross-Entropy Loss also converged well across preprocessing methods, although sometimes more slowly. Dice Loss demonstrated greater variability, particularly for raw, min-max normalized and percentile normalized inputs. In these cases, the validation loss fluctuated a lot and stayed apart from the training loss. This indicates that these models do not generalize well to new, unseen data.

Tables 4.1 and 4.2 summarize the 15 best- and worst-performing segmentation models on the validation set, respectively, ranked by the Dice Similarity Coefficient (DSC). For each model, the tables list the preprocessing method applied to the input images, the type of loss function used during training, the learning rate, the threshold value used for binarization of the predicted probability maps, as well as the resulting Dice Similarity Coefficient (DSC) and True Positive Rate (TPR) values.

#	Data Preprocessing	LF	LR	Threshold	DSC	TPR
1	LT	DL	1×10^{-4}	0.5	0.6431	0.6619
2	LT	DL	1×10^{-3}	0.5	0.6366	0.6470
3	LT	FL	1×10^{-4}	0.8	0.6286	0.7244
4	LT	FL	1×10^{-5}	0.8	0.6266	0.7049
5	$\boldsymbol{LT}$	$\boldsymbol{FL}$	$\mathbf{1 \times 10^{-3}}$	**0.8**	**0.6130**	**0.7649**
6	LT	WCE	1×10^{-5}	0.9	0.6092	0.6551
7	IC	FL	1×10^{-4}	0.8	0.6043	0.6153
8	$IC + MMN$	FL	1×10^{-3}	0.8	0.6022	0.6178
9	$IC + MMN$	FL	1×10^{-4}	0.8	0.5985	0.6717
10	IC	DL	1×10^{-3}	0.5	0.5973	0.5734
11	IC	FL	1×10^{-3}	0.8	0.5945	0.5739
12	IC	DL	1×10^{-4}	0.5	0.5919	0.5692
13	$GB + HE + MMN$	WCE	1×10^{-5}	0.9	0.5912	0.7605
14	$IC + MMN$	FL	1×10^{-5}	0.8	0.5900	0.6340
15	MMN	FL	1×10^{-4}	0.8	0.5891	0.6646

Table 4.1: 15 best-performing models on the validation set, sorted by Dice Similarity Coefficient (DSC) in descending order. LF = Loss Function, LR = Learning Rate, MMN = Min-Max Normalization, LT = Log Transformation, IC = Intensity Clipping, GB = Gaussian Blur, HE = Histogram Equalization, WCE = Weighted Cross-Entropy Loss, FL = Focal Loss, DL = Dice Loss. The selected model is outlined with a frame.

#	Data Preprocessing	LF	LR	Threshold	DSC	TPR
1	–	DL	1×10^{-4}	0.5	0.0242	0.0165
2	–	DL	1×10^{-3}	0.5	0.0270	0.0190
3	PN	WCE	1×10^{-5}	0.6	0.0323	0.0316
4	–	WCE	1×10^{-5}	0.5	0.0327	0.0232
5	–	WCE	1×10^{-4}	0.5	0.0397	0.0275
6	–	FL	1×10^{-5}	0.5	0.0423	0.0386
7	–	FL	1×10^{-3}	0.5	0.0466	0.0410
8	–	WCE	1×10^{-3}	0.5	0.0545	0.0480
9	PN	DL	1×10^{-3}	0.5	0.0598	0.0418
10	–	DL	1×10^{-5}	0.5	0.0611	0.0770
11	$GB + HE + MMN$	DL	1×10^{-5}	0.9	0.0618	0.9886
12	–	FL	1×10^{-4}	0.5	0.0892	0.0687
13	PN	WCE	1×10^{-3}	0.5	0.1865	0.1946
14	PN	DL	1×10^{-5}	0.9	0.2716	0.7769
15	PN	FL	1×10^{-5}	0.5	0.3405	0.3845

Table 4.2: 15 worst-performing models on the validation set, sorted by Dice Similarity Coefficient (DSC) in ascending order. LF = Loss Function, LR = Learning Rate, – = No Preprocessing, MMN = Min-Max Normalization, PN = Percentile Normalization, GB = Gaussian Blur, HE = Histogram Equalization, WCE = Weighted Cross-Entropy Loss, FL = Focal Loss, DL = Dice Loss.

Baseline models trained directly on raw input data performed the worst, confirming that some form of preprocessing is indeed essential for accurate segmentation. Models that used log transformation (LT) or intensity clipping (IC), both with and without min-max normalization (MMN), consistently achieved the highest scores. Percentile normalization (PN), however, offered little benefit. When comparing the top-performing models from each preprocessing approach, the one that used percentile normalization got the lowest DSC score of 0.5.

Among all candidates, model 5 from Table 4.1 offered the best balance between DSC and TPR and was therefore selected as the best. Its DSC is only about 2% lower than that of model 1 (the one with the highest DSC overall), while its TPR is the highest among all 15 top-ranked models.

Figure 4.13 presents the training and validation losses of the chosen model (model 5), which was trained using Focal Loss with log transformation preprocessing. Both curves decline rapidly during the initial epochs and stabilize after approximately 10 epochs. Their close alignment indicates good generalization and the absence of overfitting.

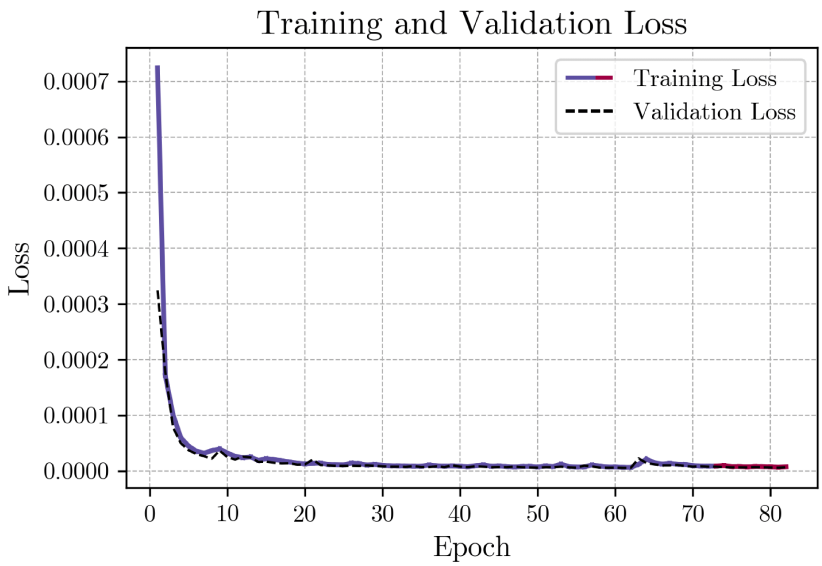


Figure 4.13: Training and validation loss curves of the best-performing model. The training loss is color-coded based on learning rate changes triggered by the scheduler, and the validation loss is shown as a dashed black line.

Figure 4.14 presents the distribution of the sizes of the nerve segments in the masks generated by the selected best model.

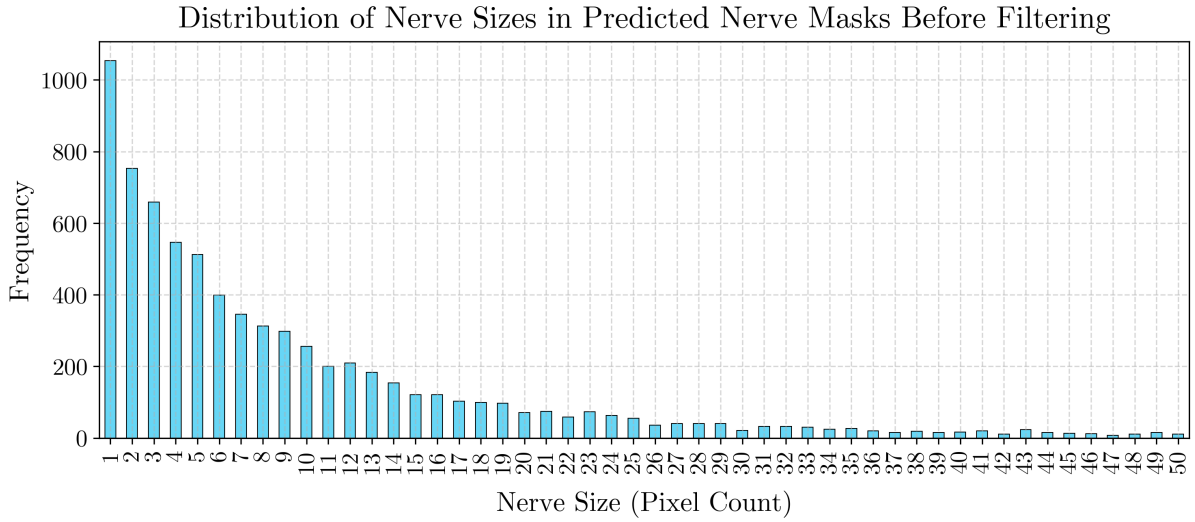


Figure 4.14: Distribution of nerve segment sizes in predicted nerve masks before filtering (first 50 nerve sizes by pixel count).

As noted in Section 3.3.4, after generating nerve masks using the best model, I applied a postprocessing step to remove small connected components in order to reduce noise and minimize false positives. To determine an appropriate threshold, I looked at the sizes of

segments in the loose ground truth masks (Figure 4.6). There, the smallest sizes are 2, 4, and 5 pixels, with sizes 2 and 4 occurring only 10 times in total. Setting a threshold too high could remove real nerve structures that may have been missed by the pipeline used to generate the loose ground truth masks. In contrast, setting it too low would retain segments that are unlikely to represent complete nerve structures and are more likely to be the result of segmentation noise or artifacts. Based on these considerations, I opted for a middle ground and set the minimum size to 4 pixels. Figure 4.15 shows the distribution of nerve sizes after this filtering step.

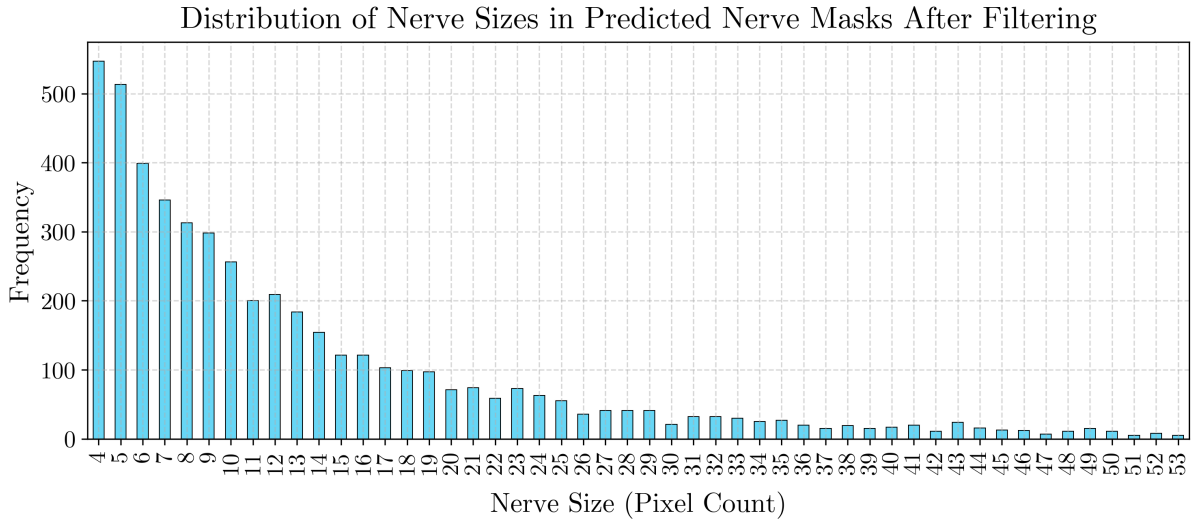


Figure 4.15: Distribution of nerve segment sizes in predicted nerve masks after filtering (first 50 nerve sizes by pixel count).

Figure 4.16 shows the nerve masks produced by the best-performing model for three validation samples with the highest number of nerve segments.

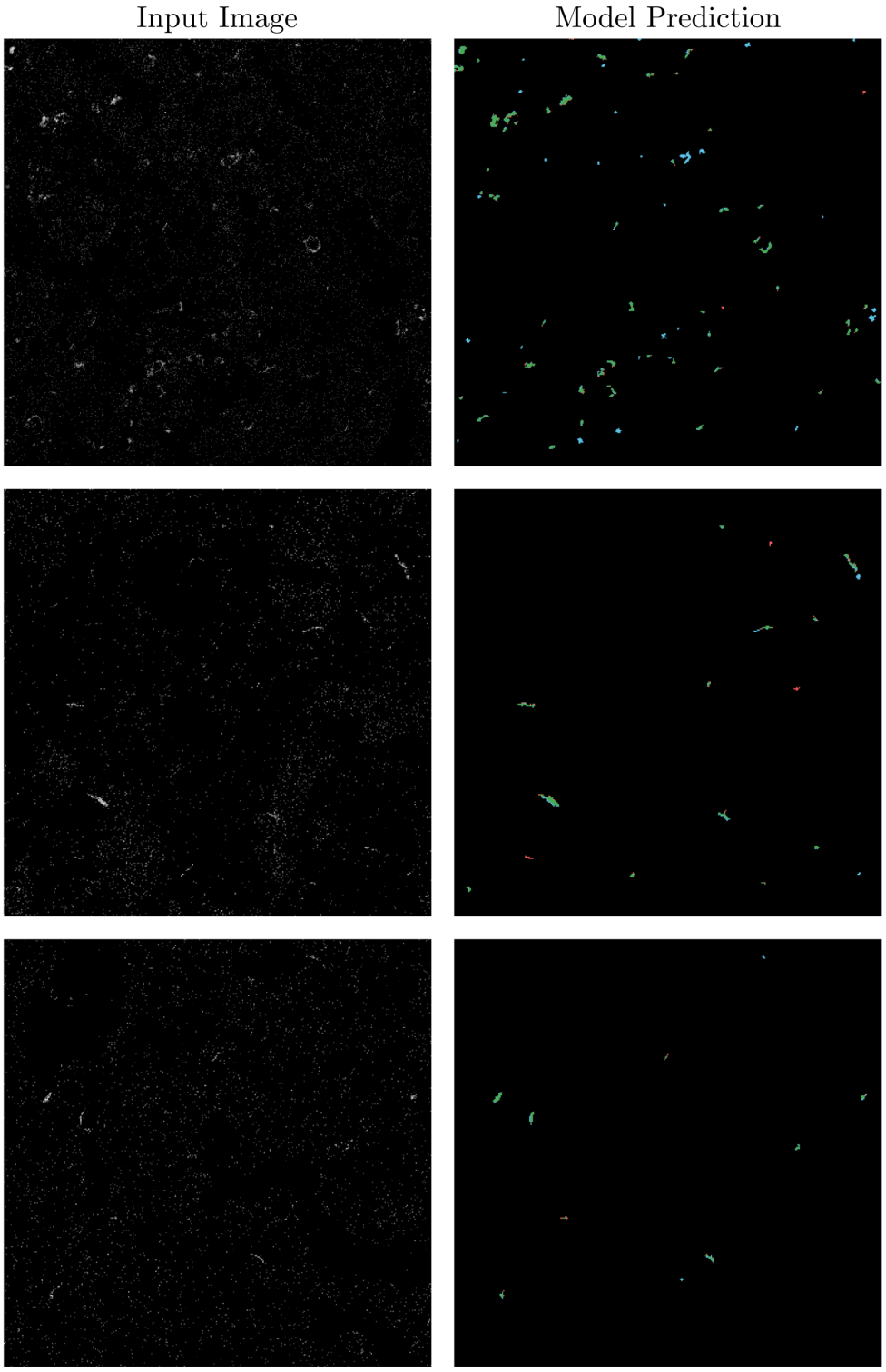


Figure 4.16: Left: input image (log-transformed peripherin channel). Right: predicted mask compared to the loose ground-truth mask. Green = true positives, red = false negatives, blue = false positives.

4.3 Survival Analysis

Cox models were fitted on data from 384 patients, of whom 58 experienced the event of interest (death from breast cancer). Table 4.3 illustrates the output format of the fitted models. This table in particular presents the results for the model that achieved the highest concordance index on the data used for fitting among those incorporating neural features derived from U-Net segmentation. The columns indicate the predictor variables, their estimated coefficients with standard errors (SE), the corresponding hazard ratios (HR), the p-values for statistical significance, the 95% confidence intervals (CI) for the hazard ratios, and the model's concordance index (C-index) with its standard error. The outputs for all other tested models are provided in Appendix B.

Predictor	**Coef (SE)**	**HR**	***p*-value**	**95% CI**	**C-index (SE)**
`pred_nerve_count`	-0.0014 (0.0047)	0.9986	0.7702	[0.9894, 1.008]	0.677 (0.035)
`age_of_diagnosis`	0.0211 (0.0223)	1.0214	0.3431	[0.9777, 1.067]	
`tumor_diameter`	0.0335 (0.0068)	1.0341	<0.0001	[1.0204, 1.048]	
`histologic_grade`	0.4097 (0.1769)	1.5064	0.0206	[1.0649, 2.131]	

Table 4.3: Cox regression results for Model 7 using the exact method for handling ties. SE = Standard Error, HR = Hazard Ratio, CI = Confidence Interval.

Nerve-related predictors only Across all models that included only nerve-related predictors (models 1, 2, 5 and 6), whether derived from the provided nerve masks or from the U-Net predicted masks, no statistically significant association with survival was observed ($p > 0.05$). The hazard ratios for these predictors are very close to 1, indicating that there is no change in the hazard with an increase in the number of nerve segments. The 95% confidence intervals for these predictors all include 1, further confirming the lack of association between the covariates and the hazard. The corresponding C-index values are low, ranging from approximately 0.48 to 0.50, suggesting that these four models do not perform better than random guessing.

The Schoenfeld residual plots for these models (Figures C.1, C.2, C.5 and C.6) show no evidence of time-dependent effects. In all cases, the black smoothing lines are almost flat over time. All p-values are greater than the significance level of 0.05, further supporting that the proportional hazards assumption is satisfied.

Nerve-related + clinical predictors When I added clinical predictors such as tumor diameter, histological grade, and age at diagnosis to these four models, predictive performance slightly improved. In the resulting models (models 3, 4, 7, and 8), tumor diameter and histological grade consistently showed statistically significant associations with survival, with $p \leq 0.05$ and hazard ratios greater than 1. Specifically, the hazard ratio is around 1.03 for tumor diameter (95% CI excluding the null value of 1) and around 1.5 for histological grade (95% CI excluding 1). This means that each 1 mm increase in tumor diameter increases the hazard by about 3%, while a one-grade increase in histological grade increases the hazard by roughly 50%. Nevertheless, nerve-related predictors in these models remained insignificant. The C-index, with the inclusion of these clinical covariates, increased to approximately 0.67-0.68. Although this is a better result compared to models based solely on nerve-related predictors, it is still slightly below the commonly accepted threshold of 0.7.

As shown in Figures C.3, C.4, C.7, and C.8, the Schoenfeld individual tests are not statistically significant ($p > 0.05$) for any of the covariates, indicating that the proportional hazards assumption holds for these models.

Clinical predictors only Model 9, which included only clinical predictors, performed similarly to the models that combined these variables with nerve-related predictors (models 3, 4, 7, and 8). Tumor diameter and histological grade remained statistically significant ($p \leq 0.05$) with hazard ratios of approximately 1.03 and 1.5, respectively. Age at diagnosis showed no statistically significant association with the hazard, with a p-value of 0.34. The C-index for this model reached 0.677, suggesting that clinical variables alone have a moderate level of discriminatory power.

The Schoenfeld residual plots for model 9 (Figure C.9) show no violation of the proportional hazards assumption. The residuals are randomly scattered over time and all p-values are above 0.05.

Prognostic value of cellular composition Model 10, which used cell-type predictors extracted from ilastik/CellProfiler-based nerve masks, achieved a C-index of 0.588. In contrast, model 11, based on U-Net predicted nerve masks, reached a slightly higher C-index of 0.62. Although both models relied on the same set of cell-type predictors, their results differ significantly. The predictors that appeared significant in one model did not appear significant in another. Specifically, in model 10, the only statistically significant predictor is the abundance of CD163+ macrophages, with a hazard ratio of 2.04 and

a p-value of 0.022. In contrast, in model 11, this same predictor appeared to be not significant ($p = 0.095 > 0.05$, HR = 1.49, 95% CI: [0.9327, 2.372]).

Model 11 also identified only one statistically significant predictor. Specifically, endothelial cell abundance showed a p-value of 0.0125, a hazard ratio of 2.88, and a remarkably wide 95% confidence interval [1.256, 6.598]. These findings again differ significantly from those of model 10 based on cell-type predictors extracted from the loose ground truth nerve masks. In that model, endothelial cell abundance has a lower hazard ratio of 1.36 and showed no statistically significant association with survival ($p = 0.689$, 95% CI: [0.2989, 6.214]).

Graphical inspection revealed no pattern over time in either of the cell-type-based model (Figures C.10 and C.11). However, when considering statistical tests, all covariates exceed the significance level of 0.05, with the exception of endothelial cells in model 10. The p-value for this covariate is 0.0385, indicating a violation of the proportional hazards assumption.

Predictive performance of models In addition to the summary statistics presented above, Table 4.4 provides the detailed concordance results for all tested Cox models evaluated on the test set. For each model, the C-index and its standard error (SE) are reported alongside the number of discordant, half-concordant, and concordant pairs.

Model No.	C-index (SE)	Discordant	Half-concordant	Concordant
1	0.6589 (0.0972)	57	17	118
2	0.6406 (0.1083)	66	6	120
3	0.2969 (0.0849)	135	0	57
4	0.2552 (0.0729)	143	0	49
5	0.5959 (0.1159)	70	19	103
6	0.5833 (0.1134)	78	4	110
7	0.2865 (0.0784)	137	0	55
8	0.2865 (0.0789)	137	0	55
9	0.2448 (0.0712)	145	0	47
10	0.8229 (0.0713)	32	4	156
11	0.7656 (0.1204)	45	0	147

Table 4.4: C-index and number of discordant, half-concordant, and concordant pairs for each Cox model using the exact method for handling ties. SE = Standard Error.

The best predictive performance is achieved by the cell-type-based models 10 and 11 with C-index values of 0.82 and 0.77, respectively. The next best-performing models

are models 1 and 2, which use total nerve counts and compartment-specific nerve counts derived from the loose ground truth masks. These models reached C-index values around 0.65, indicating predictive accuracy slightly below the commonly accepted threshold of 0.7. All remaining models showed poor predictive accuracy, with C-index values less than 0.6.

Chapter 5

Discussion

In this chapter, I first summarize the main findings of the thesis and interpret them in the context of the existing literature on neural tumor interactions in breast cancer. Then, I highlight the strengths of this work. After that, I reflect on its limitations and consider how they may have influenced the results. Finally, I outline potential directions for future research.

5.1 Thesis Objectives

In Section 1.2, I outlined the three main objectives of this thesis. Before discussing how I approached each of these objectives and assessing how successfully I achieved them, let's restate them here for clarity:

1. Develop a deep learning-based segmentation model for accurate detection of nerve fiber elements.
2. Quantify the spatial distribution of nerve segments across tissue compartments (tumor and stroma) and analyze their cellular composition.
3. Investigate how the presence and spatial distribution of nerve structures correlate with patient survival.

5.1.1 Segmentation Model Development

For the first objective, I implemented a widely used architecture for biomedical image segmentation, U-Net. The models were trained on peripherin channel images, using loose ground truth masks generated by a semi-automated segmentation pipeline applied to the same channel.

Given the nature of IMC data, I explored several preprocessing techniques to enhance nerve visibility, as well as different loss functions to address the extreme class imbalance between positive (nerve) and negative (background) pixels. The results, summarized in Tables 4.2 and 4.1, show that baseline models trained directly on the raw peripherin channel consistently yielded the lowest scores. Apparently, due to the extremely high variability in pixel intensity, U-Net was unable to extract meaningful patterns from raw, unprocessed images. In contrast, models trained on preprocessed data performed much better.

Notably, the six highest-ranked models in terms of DSC used logarithmic transformation as a preprocessing step. I was somewhat surprised by this, considering that logarithmic transformation is a rather simple method that often comes to mind first when dealing with extreme outliers. I expected that more complex preprocessing methods would yield the best results. However, as it turned out, this simple operation was sufficient for U-Net to locate nerves in noisy and artifact-prone IMC images.

When it comes to the choice of loss function, there was no clear overall winner. All three tested loss functions produced both good and poor results, depending on which preprocessing method was applied to the data before training.

The best-performing model turned out to be the one that used log-transformed input images and Focal Loss for training. It offered the best balance between the considered metrics. Specifically, it achieved a Dice Similarity Coefficient (DSC) of 0.6130 and a True Positive Rate (TPR) of 0.7649 on the validation set. In practical terms, a TPR of 0.7649 means the model detected about three-quarters of the nerve-related area in the loose ground truth masks. Meanwhile, the DSC of 0.6130 indicates that the predicted nerve regions matched the loose masks moderately well, although quite a few mismatches were still present.

It is difficult to draw a definitive conclusion whether these results are "good" or "bad" without having reliable masks for training and evaluation. When I inspected

the segmentation masks visually, they appeared quite accurate to me, but this alone is obviously not enough to draw any firm conclusions. In Figure 4.16, I provided three validation samples with the highest number of nerves to visually illustrate how the best model performs. In most cases, the model successfully detected nerves present in the loose ground truth masks. It also identified additional nerve-like structures that were absent in the training masks. It is quite possible that these are genuine nerves that semi-automated approach missed. However, without completely reliable masks or thorough examination by a specialist in the field, it is difficult to say which of them are genuine false positives and which are "false positives". In addition to false positives, some false negatives are also present. In some cases, the model failed to detect nerves present in loose masks. However, again, this does not necessarily imply poor model performance. It may simply be that some of them are the result of inaccuracies in the training masks themselves.

To summarize, I developed a segmentation pipeline capable of detecting nerve structures in IMC images with a good balance between sensitivity and overlap with the ground truth. Thus, the first objective can be considered largely achieved. Yet, this conclusion is only partial, as the lack of reliable ground truth made an objective assessment of performance impossible. When dealing with biological data, even small errors can have serious consequences. Therefore, the outcomes of such computational methods should be interpreted with extreme caution before using them for real-world decision-making.

5.1.2 Spatial Distribution and Cellular Composition Analysis

The second objective acted as a bridge between image segmentation and survival analysis. After generating the nerve segmentation masks using the best model, I quantified the spatial distribution of nerve structures. Specifically, I counted how many nerves were associated with tumor tissue and how many with stroma. I also analyzed their cellular composition. For the latter, I applied the same approach as for tissue compartments, but instead counted nerves based on the types of cells present within each segment according to the cell masks.

Although this objective was successfully completed from a technical standpoint, its biological interpretation is limited by the uncertainties in both the nerve segmentation masks discussed in Objective 1 (Subsection 5.1.1) and the cell masks used for the cellular composition analysis. Since cell masks were also produced by a deep learning model, it is quite possible that they contain inaccuracies, adding another potential source of error. If

some nerves were missed or non-nerve structures were mistakenly included, the resulting measurements may not fully reflect the true organization of the tissues. Nevertheless, the features extracted at this step served as the input for the final objective - the survival analysis.

5.1.3 Survival Analysis

The third objective was to investigate whether the presence and spatial distribution of nerve structures is associated with patient survival. For this purpose, I used Cox proportional hazards regression and nerve-related features derived from the segmentation masks. These features included total nerve counts, counts by tissue compartment (tumor vs. stroma), and measures of cellular composition within nerve regions. I evaluated these variables both individually and in combination with clinical predictors (tumor diameter, histological grade, and age at diagnosis).

The results suggest that nerve-related features alone do not have prognostic value in this cohort. Across all nerve-only models (models 1, 2, 5, and 6), hazard ratios were close to 1, confidence intervals included 1, and p-values exceeded 0.05, indicating no significant association with breast cancer-specific survival. Within the current dataset and segmentation quality, the number of nerve fiber elements was not found to be a reliable predictor of survival.

One possible explanation is that, according to co-supervisors, the peripherin marker stained all nerve fibers regardless of subtype. As a result, any subtype-specific effects may have been concealed. The literature on this topic is conflicting: some studies report that sensory nerves enhance breast cancer invasion and metastasis, while others point to a more complex picture in which sympathetic nerves promote progression, whereas parasympathetic and sensory nerves have a predominantly anti-tumor effect [30, 50, 22, 25]. So, if nerve subtypes truly have opposing effects and are combined together in a single "nerve count" variable, the final effect might appear neutral. If there were a possibility to distinguish subtypes of nerves in the provided IMC data, hypotheses concerning specific subtypes could be tested.

The addition of clinical variables such as tumor diameter, histological grade, and age at diagnosis to the nerve-related models (models 3, 4, 7, and 8) moderately improved performance. Tumor diameter and histological grade showed statistically significant associations with survival in all cases, whereas nerve-related variables remained non-significant.

Combining nerve counts with clinical variables did not change their lack of prognostic value. It is possible that segmentation errors, the small sample size, or the lack of biological specificity of the nerve variables may have limited their ability to improve prediction when strong clinical predictors were already included.

The most notable findings emerged when looking at the cellular composition of nerve-associated regions (models 10 and 11). These models achieved the highest predictive performance, with C-index values of 0.82 and 0.77. Interestingly, statistically significant predictors differed depending on whether the cell-type data came from loose masks or from nerve masks predicted by U-Net. In the model using cell-type predictors from loose masks, CD163+ macrophages appeared highly predictive, whereas in the model based on my predicted masks, endothelial cells were the significant predictors.

It is quite possible that this discrepancy in the results is related to the large number of predictors. Perhaps some of the predictors are correlated with each other, resulting in multicollinearity. If I had more time, I would perform statistical diagnostics for multicollinearity and check whether removing redundant variables would help.

In conclusion, the third objective was achieved. I examined the relationship between the spatial distribution of nerve structures and patient survival. The analysis showed no significant association in this cohort when considering nerve-related features. However, incorporating cell-type features showed some predictive value.

5.2 Strengths and Limitations

5.2.1 Strengths

A notable strength of this work is its focus on nerve-cell type interactions. In the studies I reviewed, researchers looked at nerve subtypes but did not pay attention to the specific cell types they interacted with. In this work, I used the available cell masks to examine these interactions.

Another strength is the use of an explainable approach. Although this does not apply to the nerve segmentation stage, since it relies on a deep learning model that functions as a "black box", it does apply to the survival analysis stage. With high-precision segmentation masks, spatial distribution measurements can be used to draw

accurate and transparent conclusions from survival analysis through p-values and hazard ratios.

Finally, this work includes a comprehensive evaluation of preprocessing methods to address the challenges of IMC images. In addition to existing approaches for noise reduction, contrast enhancement, and normalization, I tested a custom intensity clipping method. This method achieved competitive results, ranking second among all tested ones in terms of DSC (Table 4.1).

5.2.2 Limitations

Certainly, the most significant limitation is that the nerve segmentation masks used for model training and evaluation came from a semi-automated pipeline and served only as loose ground truth. Any errors in these masks, such as missing nerves or inclusion of non-nerve structures, would directly affect both U-Net model performance and subsequent analysis.

Second, the cell masks used for cellular composition analysis were also generated using a deep learning approach and may therefore contain inaccuracies. Since the second and third objectives relied on these masks, such errors could have affected the survival analysis results of models using features derived from them.

Third, the dataset size is relatively small, with only 427 patients in total. This posed challenges for both image segmentation and survival analysis. During segmentation model selection, only samples containing nerves were used, which represents 71.2% of the dataset. For survival analysis, the training and validation sets were merged, as the validation set had already been used multiple times during segmentation model selection. Ultimately, only 58 breast cancer-specific events among 384 patients were available for training survival models. Besides, as the test subset was used more than once to evaluate survival models, it increased the risk of overfitting. In essence, the test set did not function as a true independent test set, which ideally should be used only once. And the main reason for this was the limited size of the dataset.

Fourth, only one neural channel was used for nerve detection, which did not distinguish between neural subtypes. As a result, any subtype-specific effects on survival could not be assessed.

Finally, deaths from other causes were treated as censored observations, so potential differences in cause-specific hazards were not examined.

5.3 Future Work

1. Using multiple neural markers for subtype-specific survival analysis If the antibodies for neural markers had functioned as intended, it would have been possible to investigate whether information about specific nerve subtypes improves the predictive power of survival analysis. In particular, the dataset used in this work included TH (tyrosine hydroxylase) for sympathetic nerves and VAChT (vesicular acetylcholine transporter) for parasympathetic nerves. Had these channels not been so noisy, it would have been possible to look at their overlap with peripherin-positive fibers and conduct survival analysis for each nerve subtype separately.

2. Competing risks survival analysis In the survival analysis I conducted, I treated deaths from causes other than breast cancer as censored observations. Given more time, it would be interesting to perform survival analysis using competing risks models to see whether the cause-specific hazards differ.

3. Unsupervised segmentation approaches There is no point in training other types of supervised models without proper ground truth masks. However, it would be interesting to see if unsupervised image segmentation methods, such as the W-Net model, can give good results in identifying neural structures without labeled data.

4. Testing on a different cohort Another possible direction is to apply the developed methods to a different, perhaps larger, cohort of patients. This could reveal whether the lack of association found here is specific to the dataset used in this work or is a more general trend.

5. Iterative refinement pipeline Future work could also explore an iterative refinement approach, where existing masks are combined with model predictions and the model is updated in successive iterations. At certain stages, a person with a neurological background could review the results, indicating what is correct and what is wrong, and then, based on this feedback, it would probably be possible to train improved models. Although this would be time-consuming during the development phase, such a pipeline could save a significant amount of time in the long run if used on an ongoing basis.

Bibliography

[1] Nicolo Cosimo Albanese. How to Evaluate Survival Analysis Models, May 2022. **URL:** https://medium.com/data-science/how-to-evaluate-survival-analysis-models-dd67bc10caae. Accessed: 19-05-2025.

[2] Ylva B. Almquist. Cox regression in short. **URL:** https://statsapplied.com/part-ii-regression-analysis/cox-regression/the-cox-regression-model/cox-regression-in-short/. Accessed: 20-07-2025.

[3] Matthijs Baars, Neeraj Sinha, Mojtaba Amini, Annelies Pieterman-Bos, Stephanie Dam, Maroussia Ganpat, Miangela Laclé, Bas Oldenburg, and Yvonne Vercoulen. MATISSE: a method for improved single cell segmentation in imaging mass cytometry. *BMC Biology*, 19, May 2021.

[4] Rebecca Bevans. Understanding Confidence Intervals — Easy Examples & Formulas, June 2023. **URL:** https://www.scribbr.com/statistics/confidence-interval/. Accessed: 15-07-2025.

[5] Kimberley M. Bird, Xujiong Ye, Alan M. Race, and James M. Brown. Pushing the limits of cell segmentation models for imaging mass cytometry. In *2024 IEEE International Symposium on Biomedical Imaging (ISBI)*, page 1–5. IEEE, May 2024.

[6] Britney. ELIF: The Dice Similarity Coefficient, September 2023. **URL:** https://medium.com/the-131217net/elif-the-dice-similarity-coefficient-6355b5e0422d. Accessed: 23-04-2025.

[7] Nithin Buduma, Nikhil Buduma, and Joe Papa. *Fundamentals of Deep Learning*. O'Reilly Media, Sebastopol, CA, 2022. ISBN 9781492082156. 2nd ed.

[8] Karina Cereceda, Roddy Jorquera, and Franz Villarroel-Espindola. Advances in mass cytometry and its applicability to digital pathology in clinical-translational cancer research. *Advances in Laboratory Medicine*, 3, November 2021.

[9] Fong Chun Chan. The Basics of Survival Analysis, May 2016.
URL: https://tinyheero.github.io/2016/05/12/survival-analysis.html. Accessed: 17-07-2025.

[10] D. R. Cox and D. Oakes. *Analysis of Survival Data*. Chapman & Hall/CRC, New York, 1st edition, 1984. ISBN 9781315137438.

[11] S. Eloranta, K. E. Smedby, P. W. Dickman, and T. M. Andersson. Cancer survival statistics for patients and healthcare professionals – a tutorial of real-world data analysis. *Journal of Internal Medicine*, 289(1):12–28, January 2021.

[12] Sam Faulkner, Phillip Jobling, Brayden March, Chen Chen Jiang, and Hubert Hondermarck. Tumor Neurobiology and the War of Nerves in Cancer. *Cancer Discovery*, 9(6):702–710, June 2019. ISSN 2159-8274.

[13] GeeksforGeeks. What is Transposed Convolutional Layer?, April 2025.
URL: https://www.geeksforgeeks.org/machine-learning/what-is-transposed-convolutional-layer/. Accessed: 04-06-2025.

[14] GeeksforGeeks. How to Calculate x score of Confidence Interval, July 2025.
URL: https://www.geeksforgeeks.org/maths/how-to-calculate-z-score-of-confidence-interval/. Accessed: 14-07-2025.

[15] Brandon George, Samantha Seals, and Inmaculada Aban. Survival analysis and regression models. *Journal of nuclear cardiology*, 21(4):686–694, May 2014.

[16] Charlotte Giesen, Hao AO Wang, Denis Schapiro, Nevena Zivanovic, Andrea Jacobs, Bodo Hattendorf, Peter J Schüffler, Daniel Grolimund, Joachim M Buhmann, Simone Brandt, et al. Highly multiplexed imaging of tumor tissues with subcellular resolution by mass cytometry. *Nature methods*, 11(4):417–422, March 2014.

[17] Brenda Gillespie. Checking Assumptions in the Cox Proportional Hazards Regression Model, October 2006.
URL: https://mwsug.org/proceedings/2006/stats/MWSUG-2006-SD08.pdf. Accessed: 25-07-2025.

[18] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[19] Noah Greenwald, Geneva Miller, Erick Moen, Alex Kong, Adam Kagel, Christine Fullaway, Brianna McIntosh, Ke Leow, Morgan Schwartz, Thomas Dougherty, Cole Pavelchek, Sunny Cui, Isabella Camplisson, Omer Bar-Tal, Jaiveer Singh, Mara

Fong, Gautam Chaudhry, Zion Abraham, Jackson Moseley, and David Valen. Whole-cell segmentation of tissue images with human-level performance using large-scale data annotation and deep learning. *Nature Biotechnology*, 40:555–565, April 2021.

[20] Mina Haider, Ammar Baghdadi, and Mohammed Almukhtar. Normalized-UNet Segmentation for COVID-19 Utilizing an Encoder-Decoder Connection Layer Block. *Journal of Image and Graphics*, 12:66–75, August 2024.

[21] Frank E. Harrell Jr., Kerry L. Lee, and Daniel B. Mark. Multivariable prognostic models: issues in developing models, evaluating assumptions and adequacy, and measuring and reducing errors. *Statistics in Medicine*, 15(4):361–387, February 1996.

[22] Jianming Hu, Wuzhen Chen, Lesang Shen, Zhigang Chen, and Jian Huang. Crosstalk between the peripheral nervous system and breast cancer influences tumor progression. *Biochimica et Biophysica Acta (BBA) - Reviews on Cancer*, 1877(6):188828, November 2022.

[23] Marieke Ijsselsteijn, Antonios Somarakis, Boudewijn Lelieveldt, Thomas Höllt, and Noel de Miranda. Semi-automated background removal limits data loss and normalises imaging mass cytometry data. *Cytometry Part A*, 99(12), June 2021.

[24] Lee Dong Kyu In Junyong. Survival analysis: part II – applied clinical data analysis. *Korean J Anesthesiol*, 72(5):441–457, 2019.

[25] Jia-feng Wang, Meng-chuan Wang, Lei-lei Jiang, and Neng-ming Lin. The neuroscience in breast cancer: Current insights and clinical opportunities. *Heliyon*, 11(3), February 2025. ISSN 2405-8440.

[26] Frank E. Harrell Jr. *Regression Modeling Strategies*. Springer Series in Statistics. Springer, New York, NY, 2nd edition, 2015.

[27] Mitchell H. Katz and Walter W. Hauck. Proportional hazards (Cox) regression. *Journal of General Internal Medicine*, 8:702–711, 1993.

[28] Manpreet Kaur, Jasdeep Kaur, and Jappreet Kaur. Survey of Contrast Enhancement Techniques based on Histogram Equalization. *International Journal of Advanced Computer Science and Applications*, 2(7), 2011.

[29] David G. Kleinbaum and Mitchel Klein. *Introduction to Survival Analysis*. Springer New York, New York, NY, 2012. ISBN 978-1-4419-6646-9.

[30] Thanh Le, Samantha Payne, Maia Buckwald, Lily Hayes, Christopher Burge, and Madeleine Oudin. Sensory nerves enhance triple-negative breast cancer migration and metastasis via the axon guidance molecule PlexinB3. *npj Breast Cancer*, 8, November 2022.

[31] M. Le Rochais, P. Hemon, J. O. Pers, and A. Uguen. Application of High-Throughput Imaging Mass Cytometry Hyperion in Cancer Research. *Frontiers in Immunology*, 13:859414, March 2022.

[32] Xiaoya Li, Xiaofei Sun, Yuxian Meng, Junjun Liang, Fei Wu, and Jiwei Li. Dice Loss for Data-imbalanced NLP Tasks. *CoRR*, 2019.

[33] Tsung-Yi Lin, Priya Goyal, Ross B. Girshick, Kaiming He, and Piotr Dollár. Focal Loss for Dense Object Detection. *CoRR*, 2017.

[34] Bo Lindqvist. Slides 12: Cox regression. STK4080 Survival and Event History Analysis, Department of Mathematical Sciences, Norwegian University of Science and Technology (NTNU), 2019.
URL: `https://bo.folk.ntnu.no/STK4080/STK4080-Slides12-2019.pdf`. Accessed: 24-07-2025.

[35] Bo Lindqvist. Slides 12: Proportional hazards modeling and Cox regression. TMA4275 Lifetime Analysis, Department of Mathematical Sciences, Norwegian University of Science and Technology (NTNU), 2020.
URL: `https://bo.folk.ntnu.no/TMA4275/2020v/TMA4275-Slides12-2020.pdf`. Accessed: 23-07-2025.

[36] Peng Lu, Karolyn Oetjen, Diane Bender, Marianna Ruzinova, Daniel Fisher, Kevin Shim, Russell Pachynski, W Nathaniel Brennen, Stephen Oh, Daniel Link, and Daniel Thorek. IMC-Denoise: a content aware denoising pipeline to enhance Imaging Mass Cytometry. *Nature Communications*, 14, March 2023.

[37] Abdullah Al Mamun. *Life History of Cardiovascular Disease and Its Risk Factors: Multistate Life Table Approach and Application to the Framingham Heart Study.* PhD thesis, University of Groningen, 2003.

[38] Carlo Manco, Delia Righi, Guido Primiano, Angela Romano, Marco Luigetti, Luca Leonardi, Nicola De Stefano, and Domenico Plantone. Peripherin, A New Promising Biomarker in Neurological Disorders. *European Journal of Neuroscience*, 61(4), February 2025.

[39] Mathias Jesussek, Hannah Volk-Jesussek. *Statistics made easy*. DATAtab e.U., Graz, Austria, 5th edition edition, 2024.

[40] Lorenz Meuli and Christoph Kuemmerli. The Hazard of Non-proportional Hazards in Time to Event Analysis. *European Journal of Vascular and Endovascular Surgery*, 62(3):495–498, September 2021.

[41] Fausto Milletari, Nassir Navab, and Seyed-Ahmad Ahmadi. V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation. June 2016.

[42] Vladan Milosevic. Different approaches to Imaging Mass Cytometry data analysis. *Bioinformatics Advances*, 3(1), April 2023.

[43] Minitab, LLC. Interpret the key results for Fit Cox Model in a Counting Process Form.
URL: `https://support.minitab.com/en-us/minitab/help-and-how-to/statistical-modeling/reliability/how-to/cox-regression/counting-process-form/interpret-the-results/key-results/`. Accessed: 20-07-2025.

[44] Akif Mustafa. Censoring in Survival Analysis: Not as Scary as It Sounds, August 2023.
URL: `https://medium.com/@akif.iips/censoring-in-survival-analysis-not-as-scary-as-it-sounds-9bc244c594d8`. Accessed: 12-07-2025.

[45] Dominik Müller, Iñaki Soto-Rey, and Frank Kramer. Towards a Guideline for Evaluation Metrics in Medical Image Segmentation, February 2022.

[46] Ramzi W. Nahhas. *Introduction to Regression Methods for Public Health Using R*. CRC Press, 2024.

[47] Ying-Hwey Nai, Bernice W. Teo, Nadya L. Tan, Sophie O'Doherty, Mary C. Stephenson, Yee Liang Thian, Edmund Chiong, and Anthonin Reilhac. Comparison of metrics for the evaluation of medical segmentations using prostate MRI dataset. *Computers in Biology and Medicine*, 134, 2021.

[48] Keiron O'Shea and Ryan Nash. An Introduction to Convolutional Neural Networks, December 2015.

[49] Ozgur Ozdemir and Elena Sőnmez. Weighted Cross-Entropy for Unbalanced Data with Application on COVID X-ray images. October 2020.

[50] Veena Padmanaban, Isabel Keller, Ethan S. Seltzer, Benjamin N. Ostendorf, Zachary Kerner, and Sohail F. Tavazoie. Neuronal substance-P drives breast cancer growth and metastasis via an extracellular RNA-TLR7 axis. *bioRxiv*, 2024.

[51] Huu-Tai Thai Quang Du Nguyen. Crack segmentation of imbalanced data: The role of loss functions. *Engineering Structures*, 297:116988, December 2023.

[52] Reza Azad, Moein Heidary, Kadir Yilmaz, Michael Hüttemann, Sanaz Karimijafarbigloo, Yuli Wu, Anke Schmeink, Dorit Merhof. Loss Functions in the Era of Semantic Segmentation: A Survey and Outlook, December 2023.

[53] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional Networks for Biomedical Image Segmentation. May 2015.

[54] Samar Abd ElHafeez, Graziella D'Arrigo, Daniela Leonardis, Maria Fusaro, Giovanni Tripepi, Stefanos Roumeliotis. Methods to Analyze Time-to-Event Data: The Cox Regression Analysis. *Oxidative Medicine and Cellular Longevity*, November 2021.

[55] Samia Akhtar, Shabib Aftab, Oualid Ali, Munir Ahmad, Muhammad Adnan Khan, Sagheer Abbas, and Taher M. Ghazal. A deep learning based model for diabetic retinopathy grading. *Scientific Reports*, 15:1–20, January 2025.

[56] Matthias Schmid, Marvin Wright, and Andreas Ziegler. On the use of Harrell's C for clinical risk prediction via random survival forests, July 2016.

[57] Sebastian Schneeweiss, Philip S. Wang, Jerry Avorn, Malcolm Maclure, Raia Levin, and Robert J. Glynn. Consistency of Performance Ranking of Comorbidity Adjustment Scores in Canadian and U.S. Utilization Data. *Journal of General Internal Medicine*, 19(5p1):444–450, May 2004.

[58] Aryaman Sharda. Understanding Gaussian Blurs. *Digital Bunker*, September 2020. **URL:** `https://digitalbunker.dev/understanding-gaussian-blurs/`. Accessed: 25-04-2025.

[59] Antonios Somarakis, Vincent Van Unen, Frits Koning, Boudewijn Lelieveldt, and Thomas Höllt. ImaCytE: Visual Exploration of Cellular Micro-Environments for Imaging Mass Cytometry Data. *IEEE Transactions on Visualization and Computer Graphics*, 27(1):98–110, January 2021.

[60] Carole H. Sudre, Wenqi Li, Tom Vercauteren, Sébastien Ourselin, and M. Jorge Cardoso. Generalised Dice overlap as a deep learning loss function for highly unbalanced segmentations. *CoRR*, July 2017.

[61] Akshay Swaminathan. Implementing the Cox model in R, April 2021.
URL: https://medium.com/analytics-vidhya/implementing-the-cox-model-in-r-b1292d6ab6d2. Accessed: 16-07-2025.

[62] Juan Terven, Diana M Cordova-Esparza, Alfonso Ramirez-Pedraza, Edgar A Chavez-Urbiola, and Julio A Romero-Gonzalez. Loss Functions and Metrics in Deep Learning. April 2025.

[63] Lu Tian. STAT331: Cox's Proportional Hazards Model. Stanford University, January 2014.
URL: https://web.stanford.edu/~lutian/coursepdf/unitcox1.pdf. Accessed: 05-05-2025.

[64] Vijay Vignesh. Gaussian Blurring — A Gentle Introduction, June 2023.
URL: https://pub.towardsai.net/gaussian-blurring-a-gentle-introduction-e34aca1d9bbd. Accessed: 25-04-2025.

[65] Francesco Visin Vincent Dumoulin. A guide to convolution arithmetic for deep learning, January 2018.

[66] W. W. M. Abeysekera, M. R. Sooriyarachchi. Use of Schoenfeld's global test to test the proportional hazards assumption in the Cox proportional hazards model: an application to a clinical study. *Journal of the National Science Foundation of Sri Lanka*, 37(1), 2009.

[67] Yu Wang, Qian Chen, and Baeomin Zhang. Image Enhancement Based on Equal Area Dualistic Sub-Image and Non-parametric Modified Histogram Equalization Method. *IEEE Transactions on Consumer Electronics*, 45(1):68–75, February 1999.

[68] Aston Zhang, Zachary C. Lipton, Mu Li, and Alexander J. Smola. *Dive into Deep Learning*. Cambridge University Press, 2023.

Appendix A

Loss Curves

The loss curves are shown on the following pages.

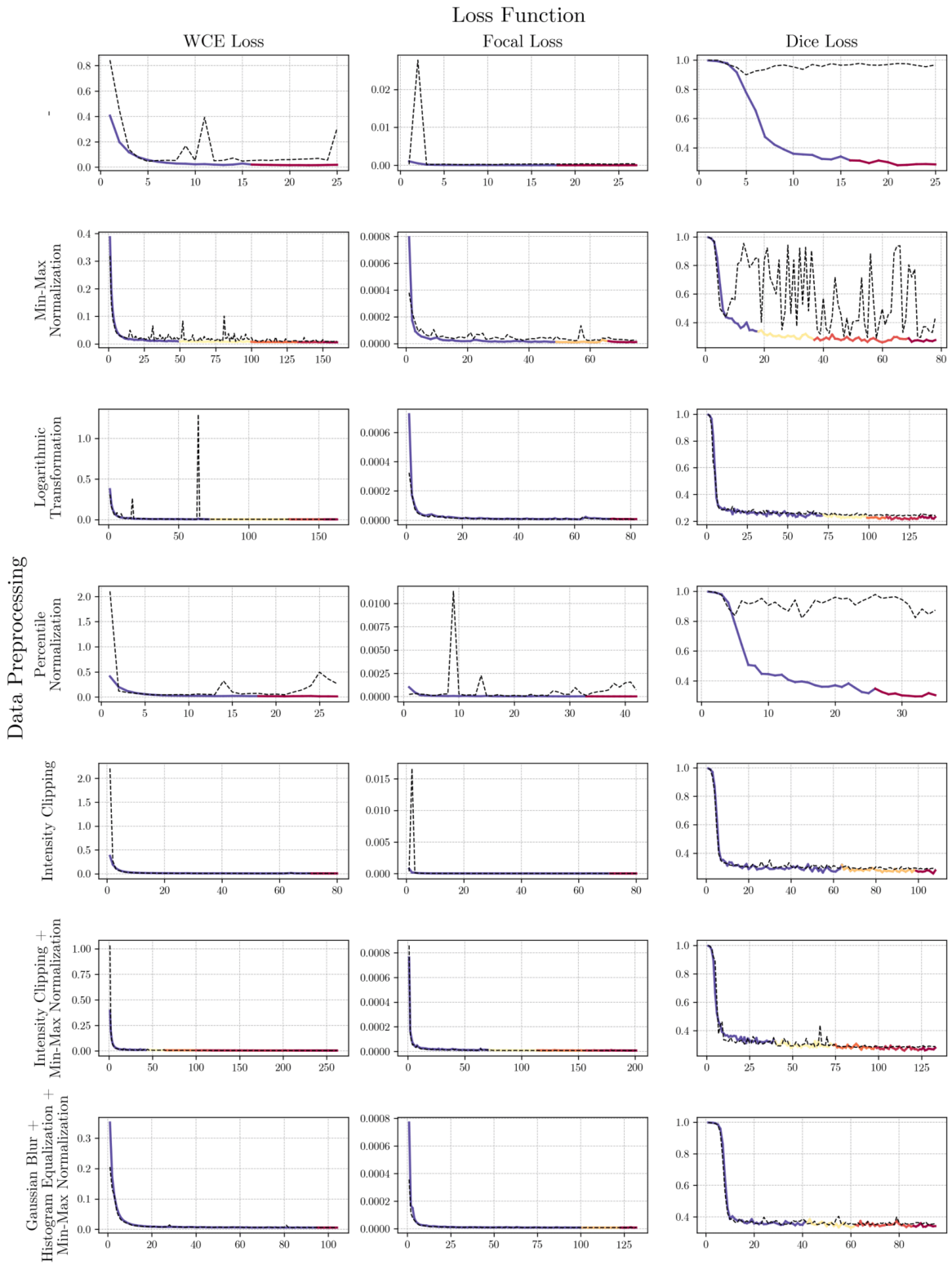


Figure A.1: Training and validation loss curves for U-Net models trained with an initial learning rate of 1×10^{-3}. Training loss lines are color-coded based on learning rate changes triggered by the scheduler. Validation loss is represented by a dashed black line.

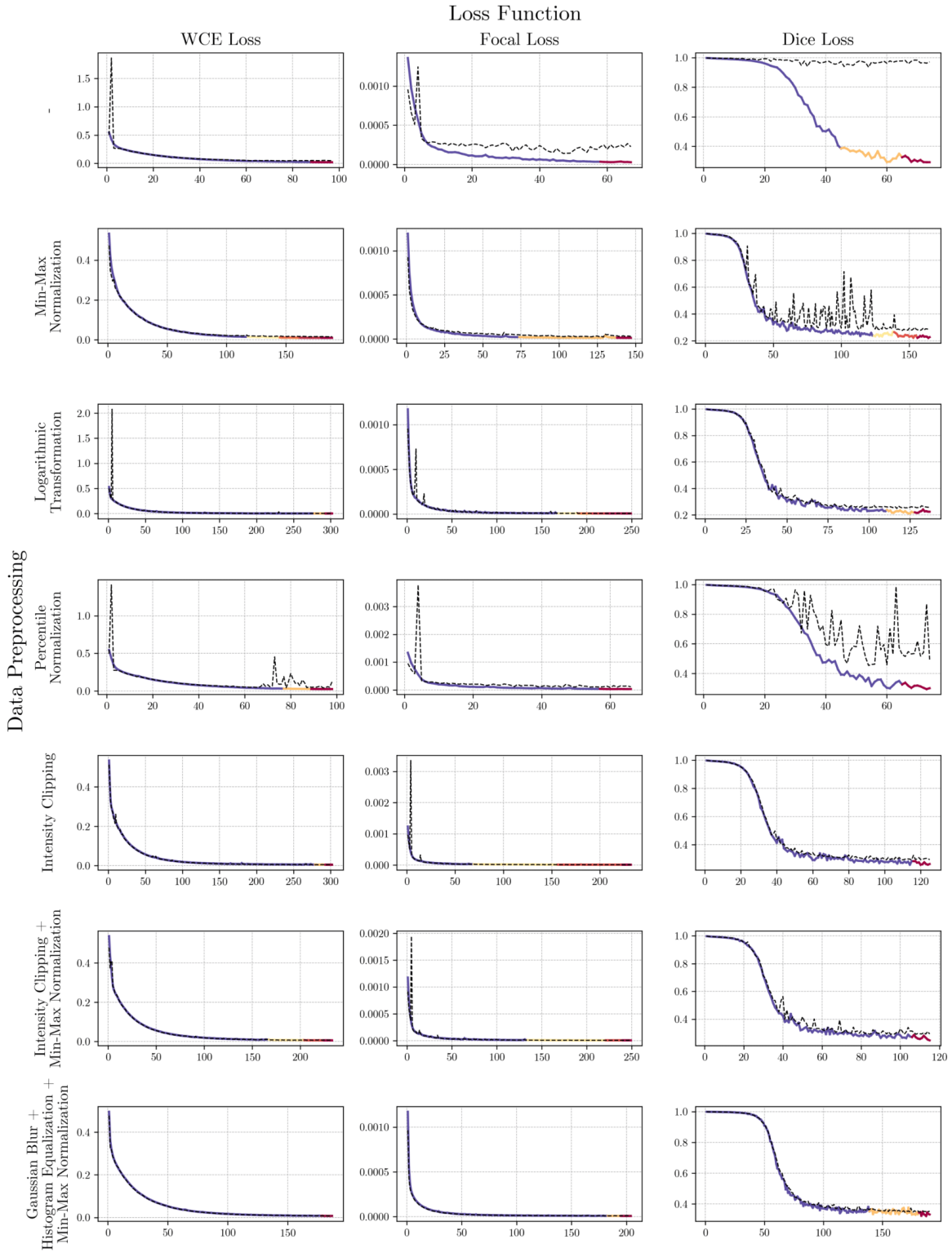


Figure A.2: Training and validation loss curves for models trained with an initial learning rate of 1×10^{-4}.

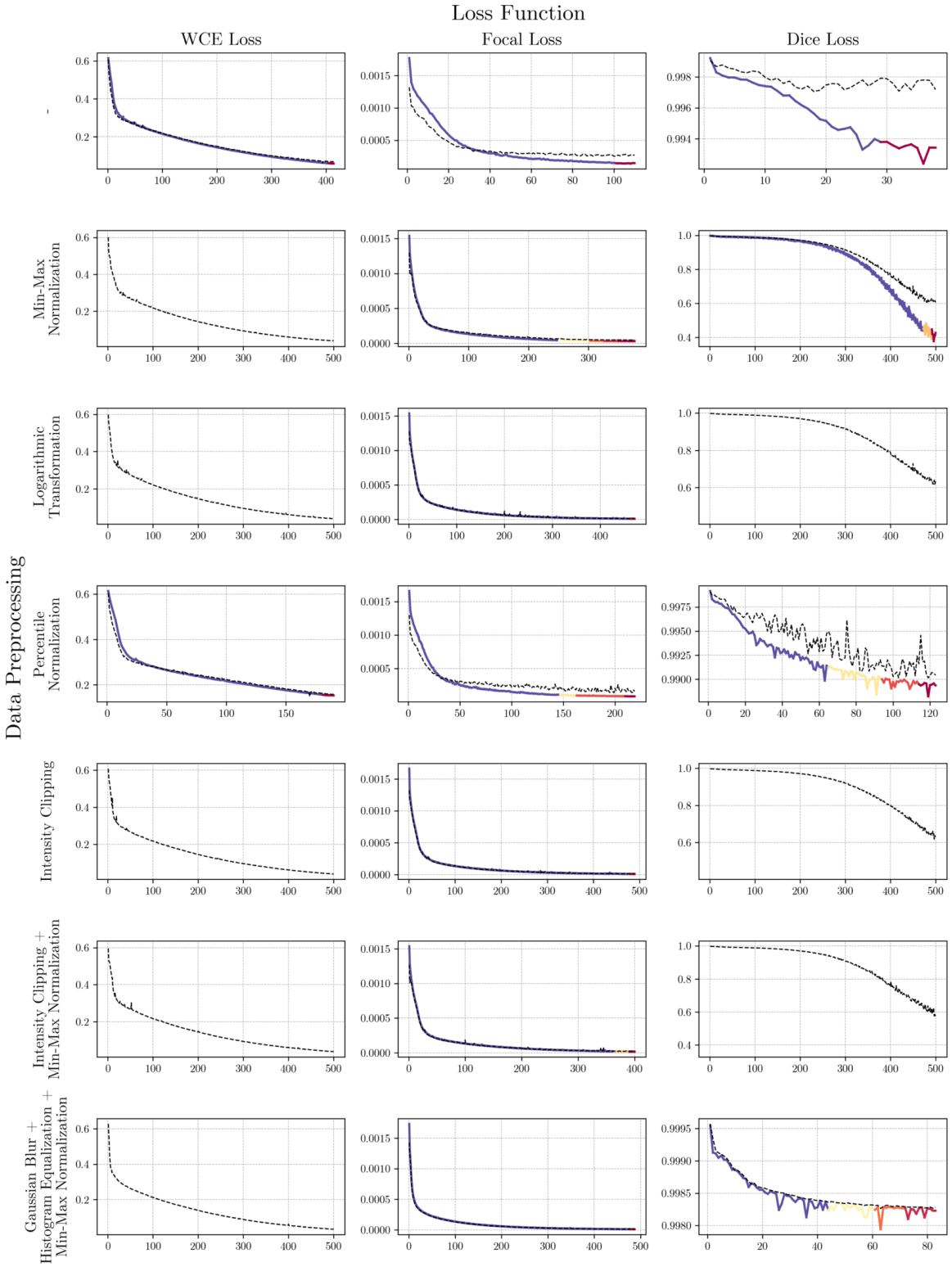


Figure A.3: Training and validation loss curves for models trained with an initial learning rate of 1×10^{-5}.

Appendix B

Cox Regression Model Outputs

Predictor	Coef (SE)	HR	*p*-value	95% CI	C-index (SE)
nerve_count	-0.0041 (0.0066)	0.9959	0.532	[0.9831, 1.009]	0.484 (0.039)

Table B.1: Cox regression results for Model 1 using the exact method for handling ties. SE = Standard Error, HR = Hazard Ratio, CI = Confidence Interval.

Predictor	Coef (SE)	HR	*p*-value	95% CI	C-index (SE)
n_stroma	-0.0071 (0.0144)	0.9930	0.623	[0.9653, 1.021]	0.498 (0.039)
n_tumor	-0.0008 (0.0149)	0.9992	0.959	[0.9705, 1.029]	

Table B.2: Cox regression results for Model 2.

Predictor	Coef (SE)	HR	*p*-value	95% CI	C-index (SE)
nerve_count	-0.0025 (0.0063)	0.9975	0.6928	[0.9854, 1.010]	0.677 (0.034)
age_of_diagnosis	0.0211 (0.0223)	1.0213	0.3434	[0.9777, 1.067]	
tumor_diameter	0.0334 (0.0068)	1.0340	<0.0001	[1.0203, 1.048]	
histologic_grade	0.4101 (0.1769)	1.5070	0.0204	[1.0654, 2.132]	

Table B.3: Cox regression results for Model 3.

Predictor	Coef (SE)	HR	*p*-value	95% CI	C-index (SE)
n_stroma	-0.0002 (0.0135)	0.9998	0.9906	[0.9738, 1.027]	0.677 (0.034)
n_tumor	-0.0050 (0.0152)	0.9950	0.7420	[0.9659, 1.025]	
age_of_diagnosis	0.0213 (0.0223)	1.0215	0.3408	[0.9778, 1.067]	
tumor_diameter	0.0335 (0.0069)	1.0341	<0.0001	[1.0203, 1.048]	
histologic_grade	0.4115 (0.1771)	1.5091	0.0202	[1.0665, 2.136]	

Table B.4: Cox regression results for Model 4. SE = Standard Error, HR = Hazard Ratio, CI = Confidence Interval.

Predictor	Coef (SE)	HR	*p*-value	95% CI	C-index (SE)
pred_nerve_count	-0.0026 (0.0050)	0.9974	0.604	[0.9876, 1.007]	0.483 (0.039)

Table B.5: Cox regression results for Model 5.

Predictor	Coef (SE)	HR	*p*-value	95% CI	C-index (SE)
pred_n_stroma	-0.0028 (0.0115)	0.9972	0.809	[0.9750, 1.020]	0.481 (0.039)
pred_n_tumor	-0.0025 (0.0066)	0.9975	0.701	[0.9846, 1.010]	

Table B.6: Cox regression results for Model 6.

Predictor	Coef (SE)	HR	*p*-value	95% CI	C-index (SE)
pred_n_stroma	0.0034 (0.0104)	1.0034	0.7445	[0.9831, 1.024]	0.674 (0.035)
pred_n_tumor	-0.0034 (0.0071)	0.9966	0.6333	[0.9828, 1.011]	
age_of_diagnosis	0.0212 (0.0223)	1.0215	0.3412	[0.9778, 1.067]	
tumor_diameter	0.0337 (0.0068)	1.0343	<0.0001	[1.0205, 1.048]	
histologic_grade	0.4160 (0.1775)	1.5159	0.0191	[1.0705, 2.147]	

Table B.7: Cox regression results for Model 8.

Predictor	**Coef (SE)**	**HR**	***p*-value**	**95% CI**	**C-index (SE)**
age_of_diagnosis	0.0212 (0.0223)	1.0215	0.3414	[0.9777, 1.067]	0.677 (0.035)
tumor_diameter	0.0337 (0.0068)	1.0343	<0.0001	[1.0207, 1.048]	
histologic_grade	0.4083 (0.1768)	1.5043	0.0209	[1.0637, 2.127]	

Table B.8: Cox regression results for Model 9. SE = Standard Error, HR = Hazard Ratio, CI = Confidence Interval.

Predictor	**Coef (SE)**	**HR**	***p*-value**	**95% CI**	**C-index (SE)**
B_cells	-1.0320 (1.1428)	0.3563	0.3665	[0.0379, 3.346]	0.588 (0.039)
Cancer_cell	-0.0411 (0.0313)	0.9597	0.1885	[0.9027, 1.020]	
Endothelial	0.3096 (0.7741)	1.3628	0.6892	[0.2989, 6.214]	
Immune_other	-0.0451 (0.7430)	0.9559	0.9516	[0.2228, 4.101]	
Macrophages_CD163+	0.7107 (0.3108)	2.0353	0.0222	[1.1067, 3.743]	
Macrophages_CD163-	0.6675 (0.6669)	1.9494	0.3168	[0.5276, 7.203]	
Stromal	-0.0113 (0.0793)	0.9888	0.8869	[0.8465, 1.155]	
T_cytotoxic	0.2070 (0.9594)	1.2300	0.8292	[0.1876, 8.064]	
T_helper	-0.0133 (0.0669)	0.9868	0.8431	[0.8655, 1.125]	
T_regulatory	0.1735 (0.2379)	1.1895	0.4657	[0.7462, 1.896]	

Table B.9: Cox regression results for Model 10.

Predictor	**Coef (SE)**	**HR**	***p*-value**	**95% CI**	**C-index (SE)**
B_cells	-0.9307 (0.8785)	0.3943	0.2894	[0.0705, 2.206]	0.620 (0.038)
Cancer_cell	-0.0383 (0.0273)	0.9625	0.1614	[0.9123, 1.015]	
Endothelial	1.0573 (0.4232)	2.8787	0.0125	[1.2560, 6.598]	
Immune_other	-0.4263 (0.8994)	0.6529	0.6356	[0.1120, 3.806]	
Macrophages_CD163+	0.3969 (0.2381)	1.4873	0.0954	[0.9327, 2.372]	
Macrophages_CD163-	0.1373 (0.5896)	1.1472	0.8158	[0.3612, 3.643]	
Stromal	-0.0439 (0.0796)	0.9571	0.5815	[0.8189, 1.119]	
T_cytotoxic	0.0514 (0.8265)	1.0527	0.9504	[0.2083, 5.319]	
T_helper	0.0787 (0.0744)	1.0819	0.2901	[0.9351, 1.252]	
T_regulatory	0.2400 (0.3610)	1.2713	0.5061	[0.6266, 2.579]	

Table B.10: Cox regression results for Model 11. SE = Standard Error, HR = Hazard Ratio, CI = Confidence Interval.

Appendix C

Schoenfeld Residuals Plots

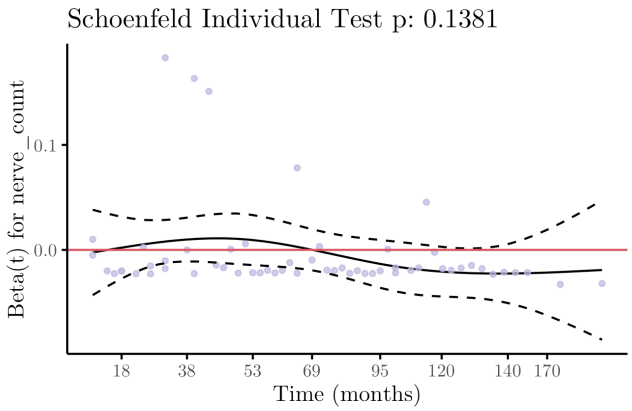


Figure C.1: Schoenfeld residuals for Model 1. The pink horizontal line at zero represents the null hypothesis of no time-dependent effect. The solid black line is a LOESS smoothing curve of the residuals, while the dashed lines are the 95% confidence bands around the smoothed fit.

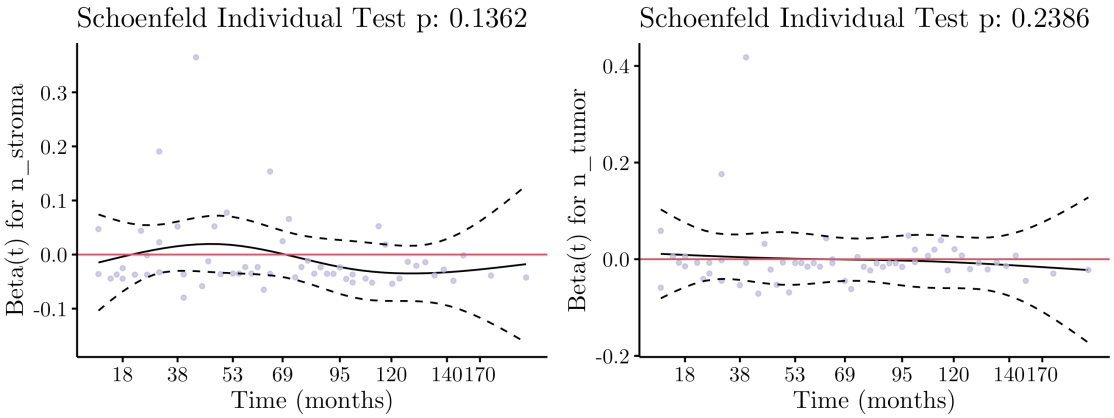


Figure C.2: Schoenfeld residuals for Model 2.

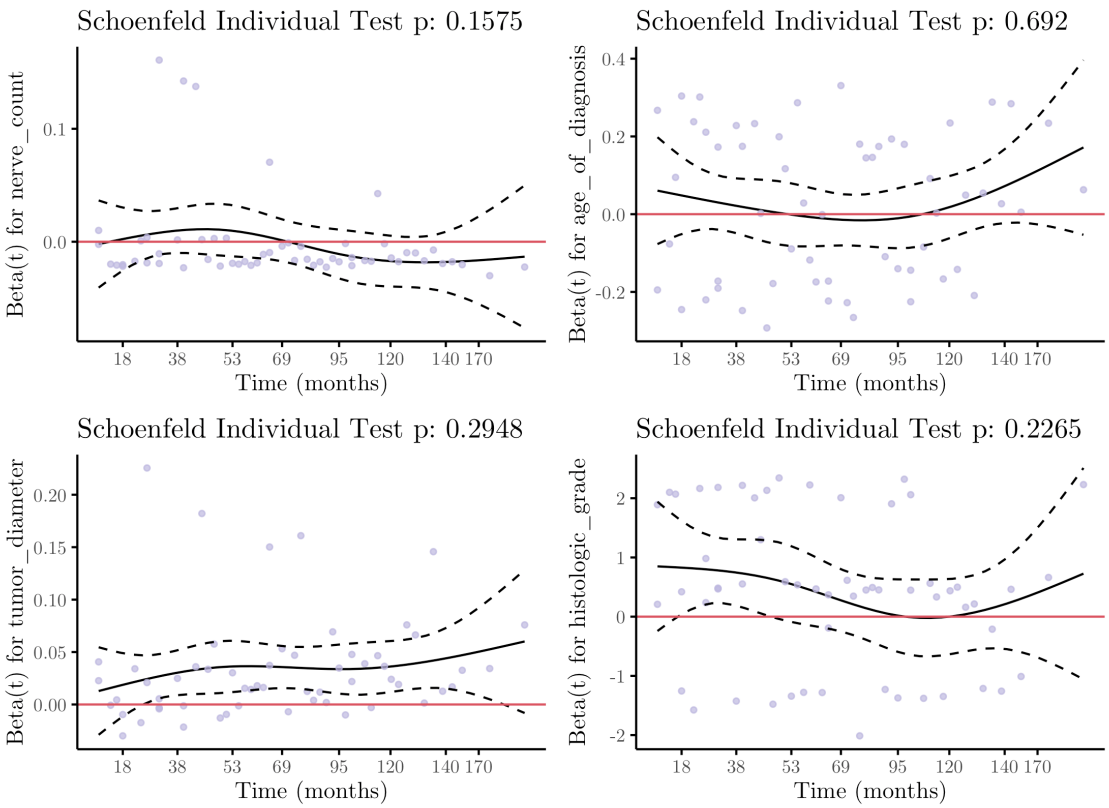


Figure C.3: Schoenfeld residuals for Model 3.

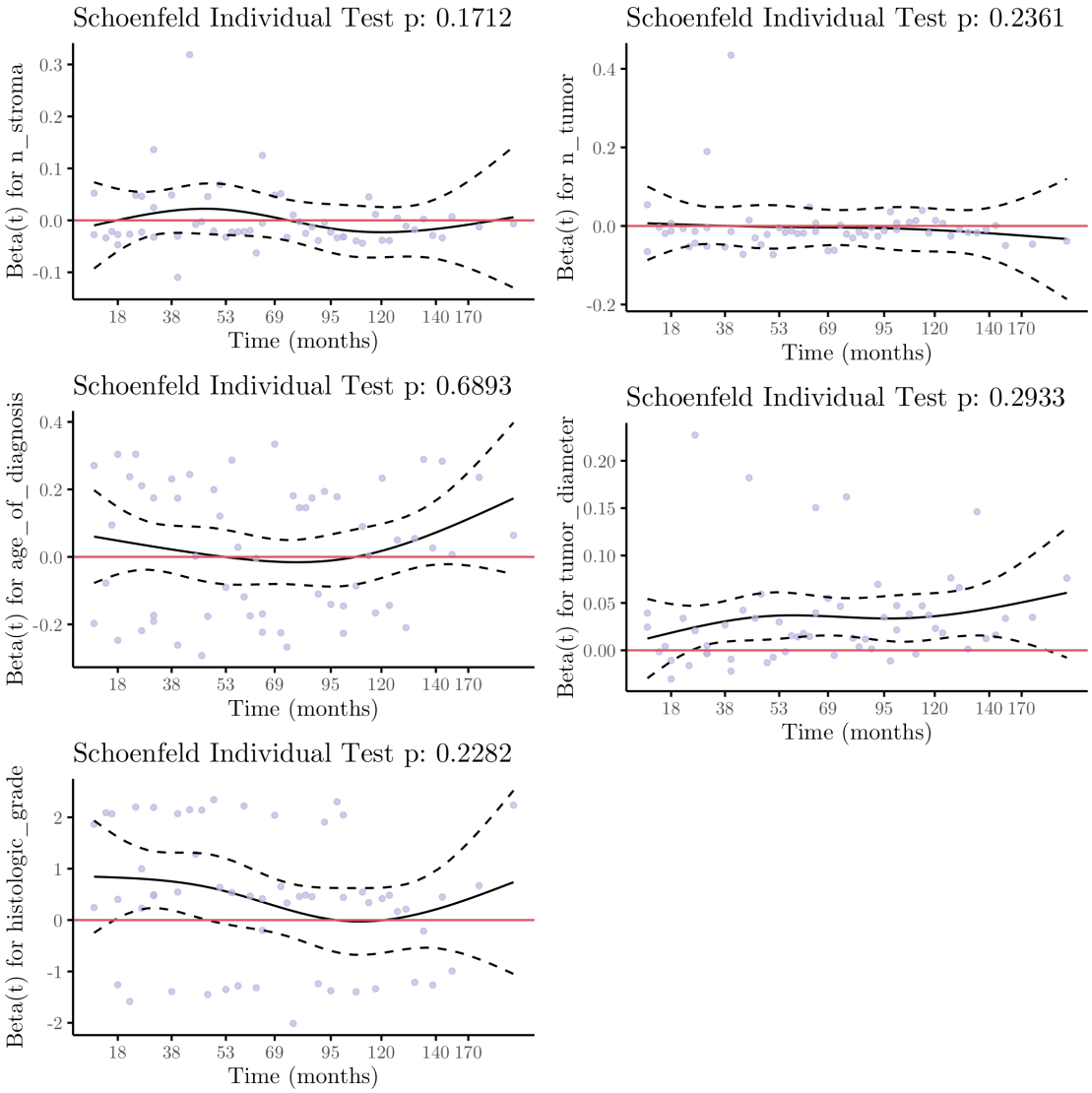


Figure C.4: Schoenfeld residuals for Model 4.

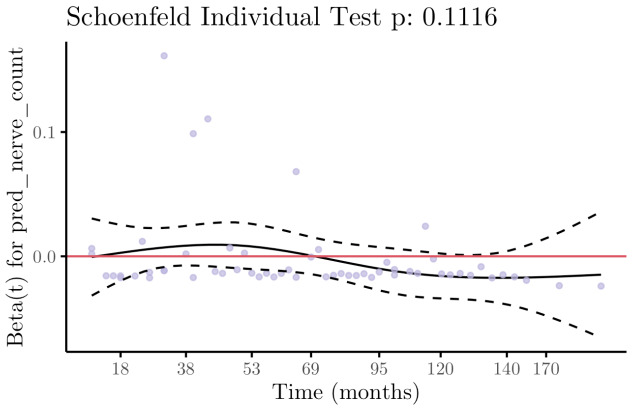


Figure C.5: Schoenfeld residuals for Model 5.

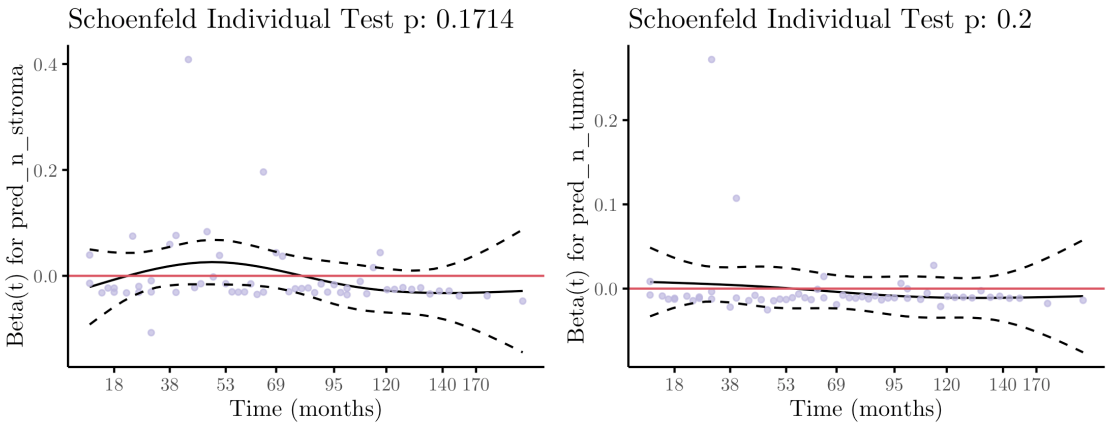


Figure C.6: Schoenfeld residuals for Model 6.

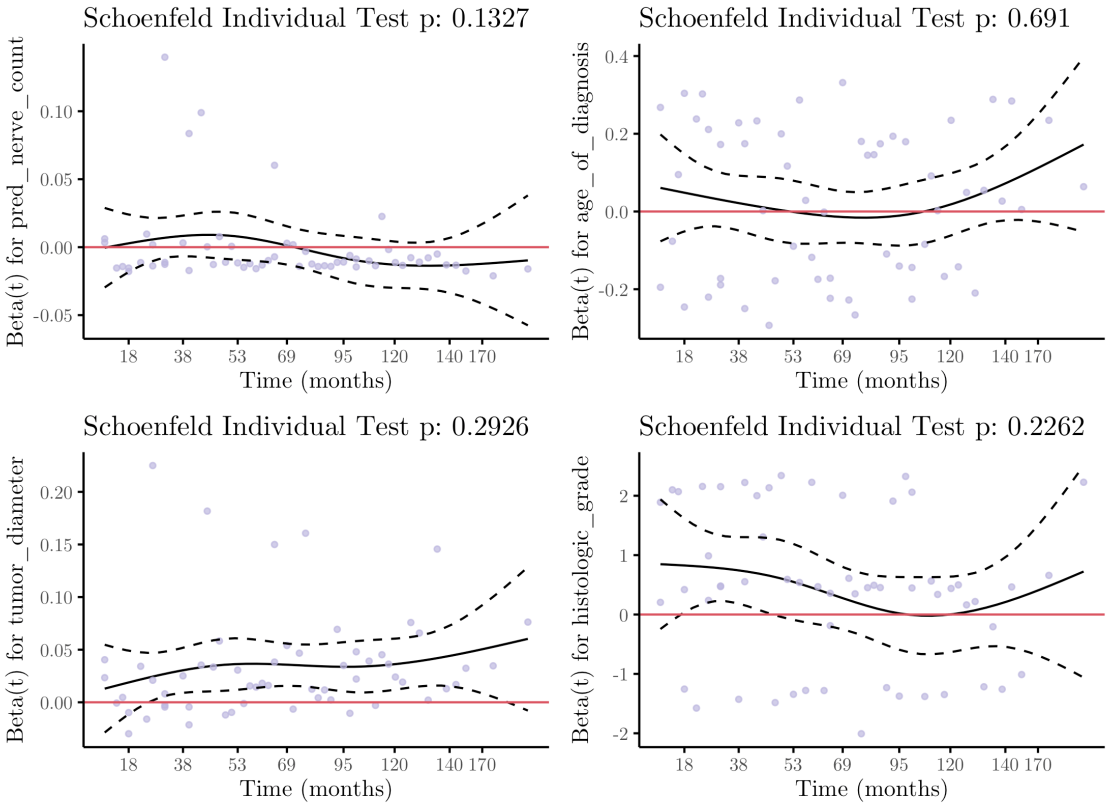


Figure C.7: Schoenfeld residuals for Model 7.

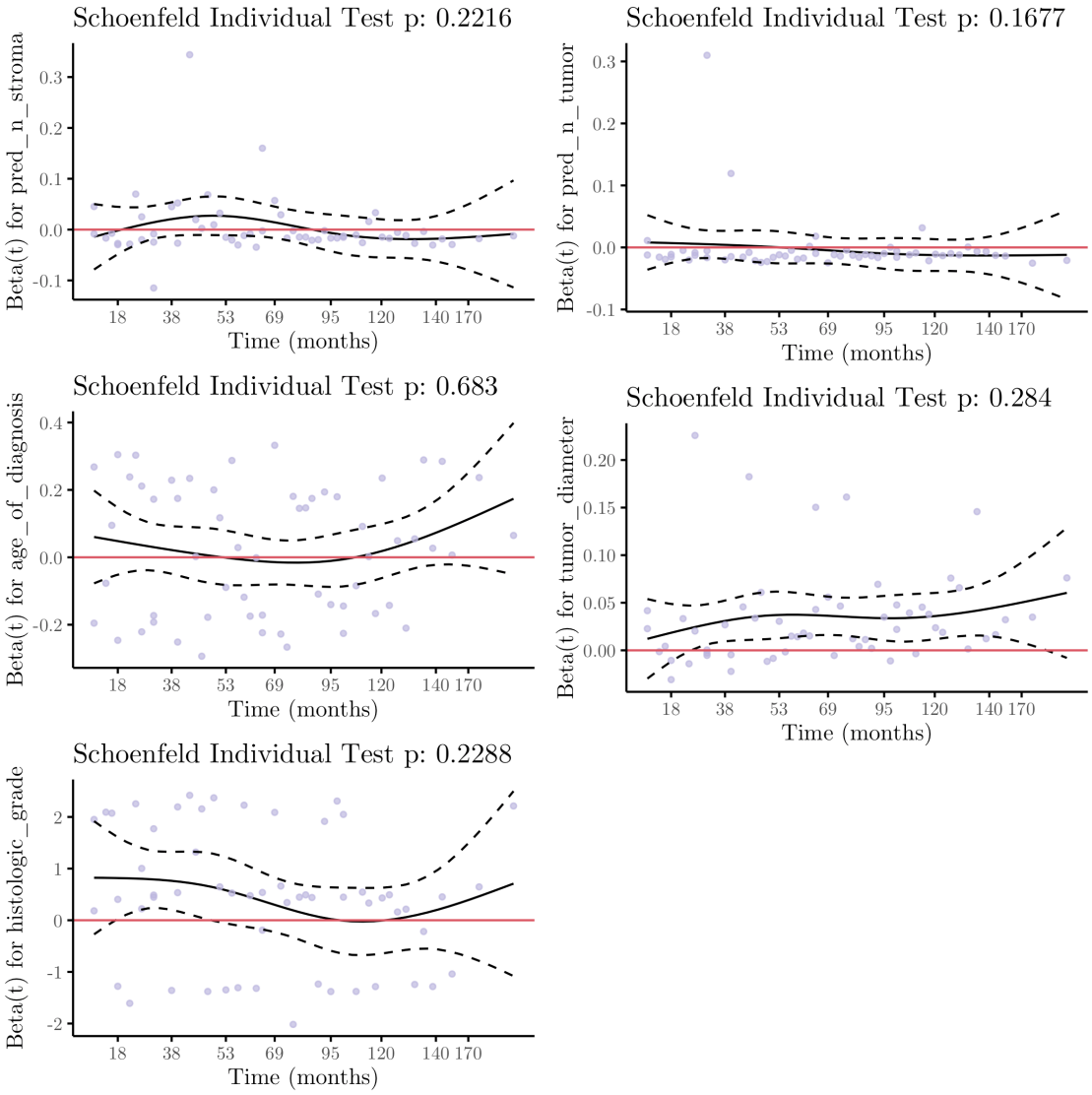


Figure C.8: Schoenfeld residuals for Model 8.

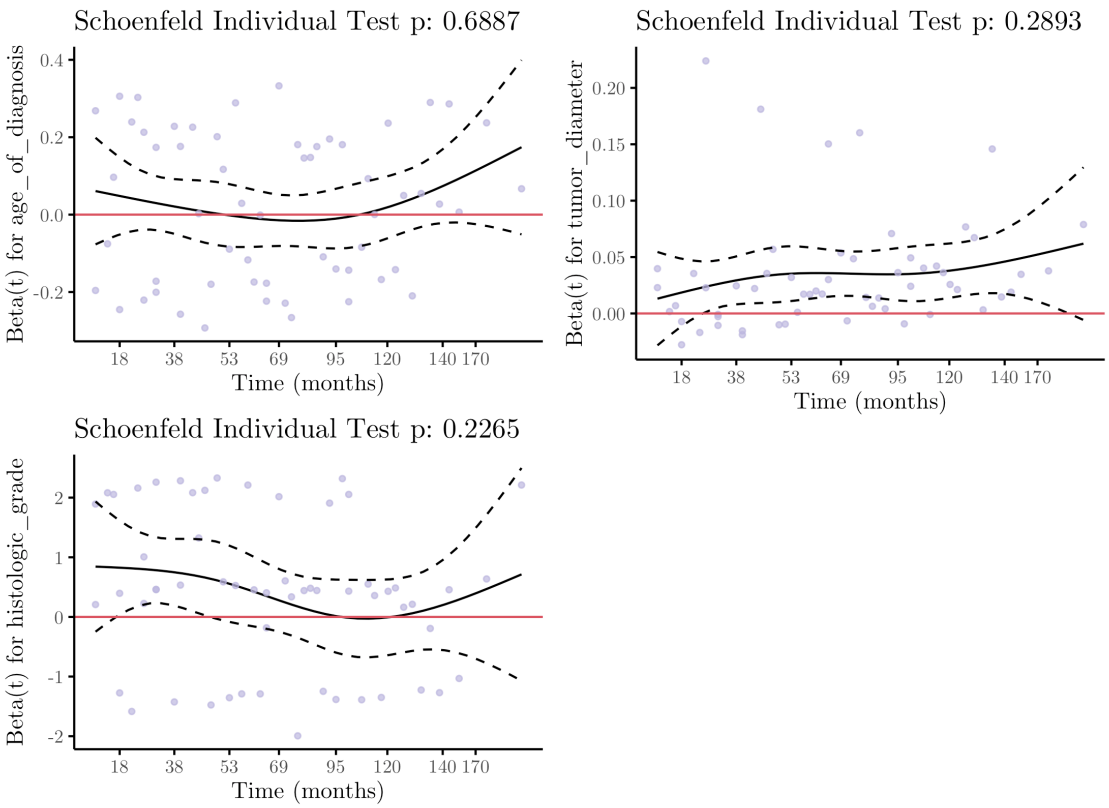


Figure C.9: Schoenfeld residuals for Model 9.

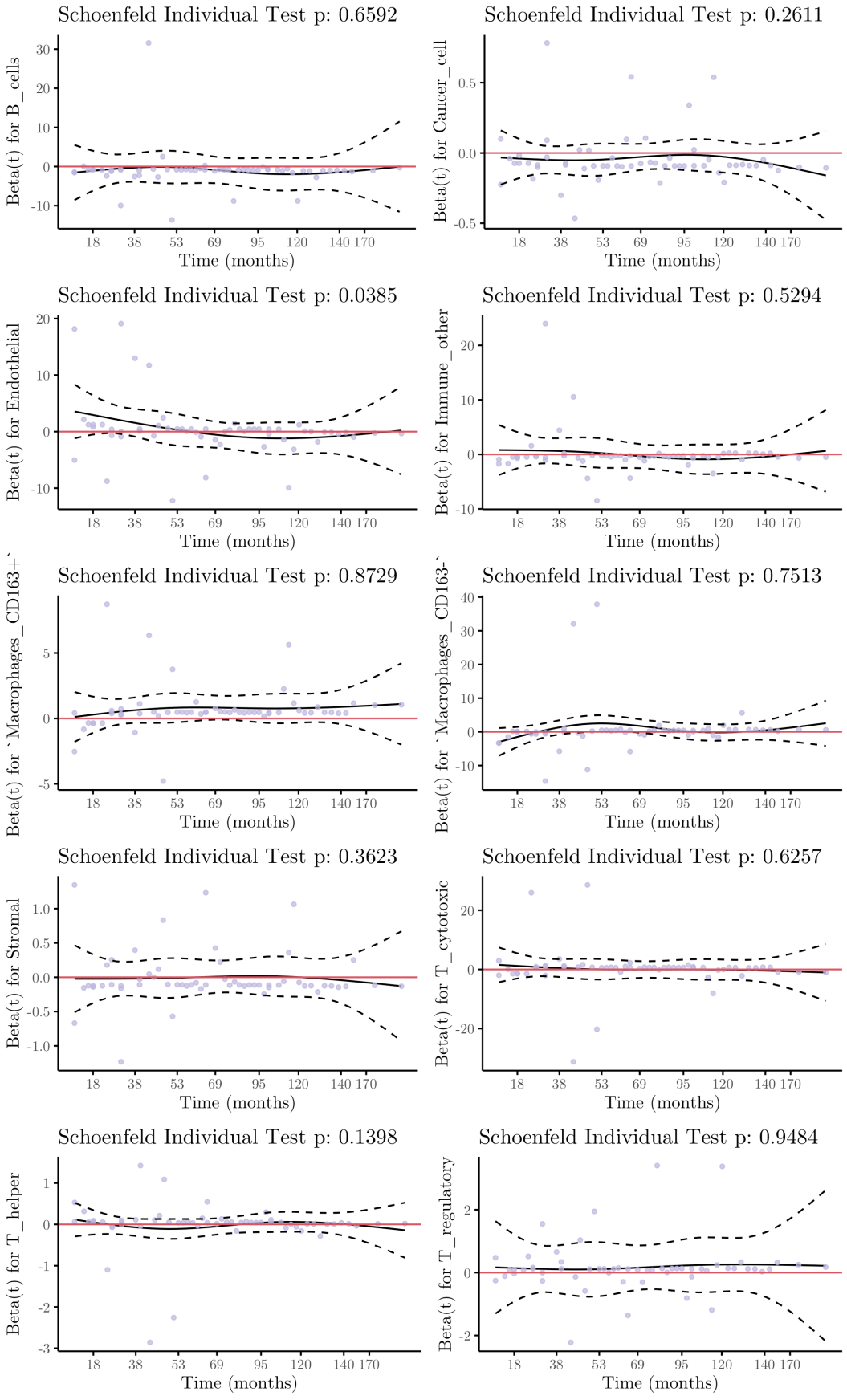


Figure C.10: Schoenfeld residuals for Model 10.

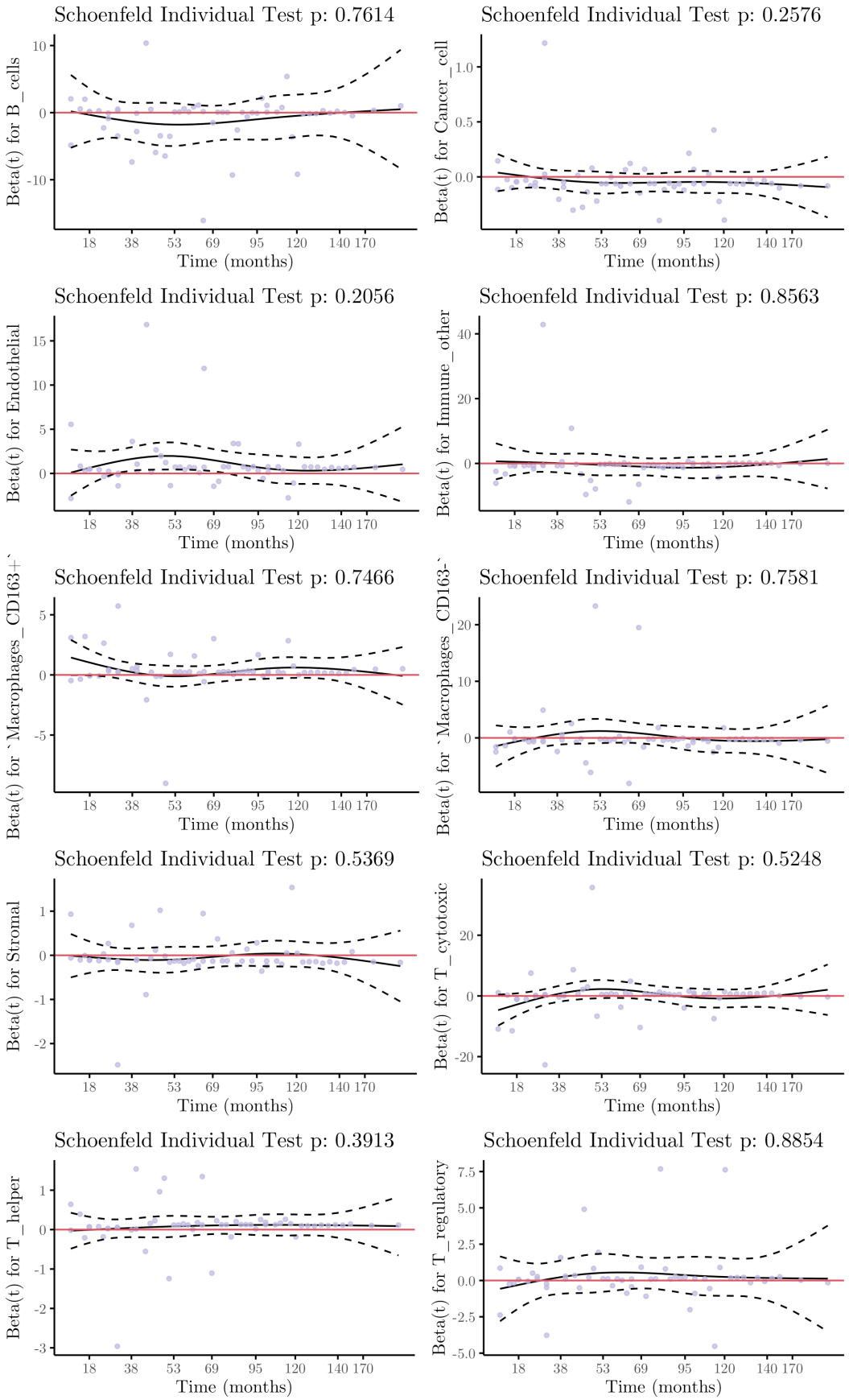


Figure C.11: Schoenfeld residuals for Model 11.

Appendix D

Code

All scripts developed for this thesis are publicly available at `https://github.com/kuimetra/breast-cancer-nerve-analysis`.